SIMULINK

E. Cabrol

Création **07/10/2016**
Dernière MAJ **11/12/2022**

Ce document compile des notes personnelles prises au cours du temps, il ne s'agit donc pas d'un user manual exhaustif (la doc officielle est imbattable).

Ce document est sous licence Creative Commons BY-NC-SA. Pour consulter une copie de la licence, visitez https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode.fr

1 INTRODUCTION 3

2 MEILLEURES RESSOURCES 4

3 ENVIRONNEMENT 4

4 CONSTRUCTION D'UN MODELE 7

5 ARCHITECTURE D'UN MODELE 33

6 GESTION DES DONNEES 51

7 STATEFLOW 55

8 INTERFACES 56

9 SOLVEURS 69

10 SIMULINK CODER – GENERATION DE CODE (SINCE 2011A) 71

11 EMBEDDED CODER (SINCE 2011A) 78

12 SIMULINK COMPILER (SINCE 2020A) 79

13 SIMULINK PROJECT 81

14 ANALYSE D'UN MODELE 82

15 VERIFICATION ET VALIDATION 94

16 REPORT GENERATOR 120

17 DEPLOIEMENT D'APPLICATIONS 128

18 BLOCKSETS ET TOOLBOXES 130

19 MACHINE LEARNING 134

20 COMMENT FAIRE POUR … 135

21 EVOLUTIONS – RELEASES 142

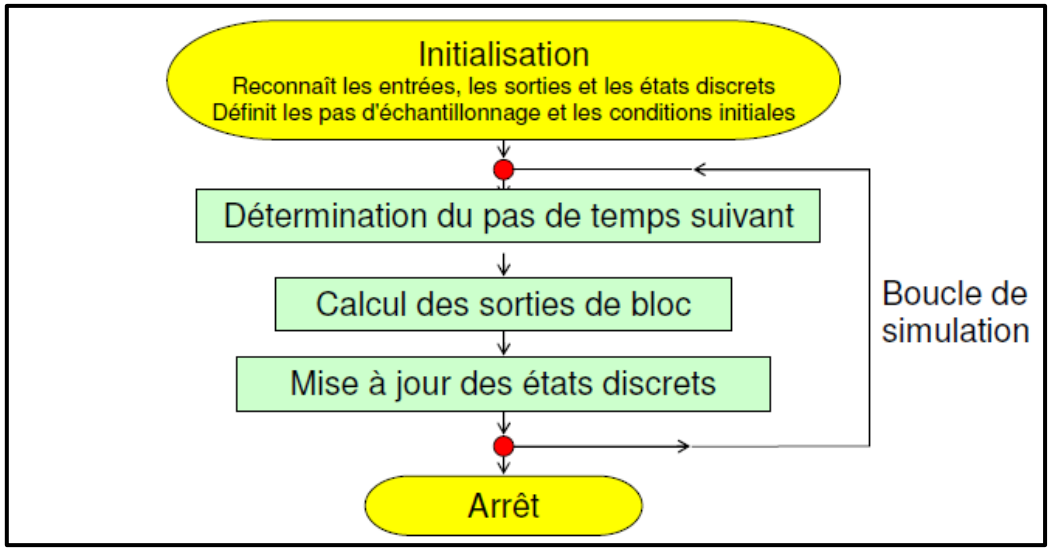
22 EXTENSIONS DE FICHIER 144

1 INTRODUCTION

Simulink est un environnement (graphique) de modélisation et un solveur dédié à la simulation des systèmes dynamiques.

Simulink a été créé dans les années 1990 pour faire de l'automatique, des lois de commande …

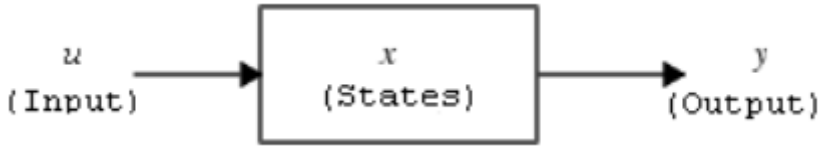
Simulink permet également de faire de la génération automatique de code (C, etc …) pour des systèmes embarqués.



Exemple : simulation de systèmes discrets

La brique élémentaire d'un modèle Simulink s'appelle un bloc. Il s'agit soit d'un composant élémentaire présent dans une bibliothèque (ou une toolbox), soit d'une interface qui permet de faire l'abstraction d'un sous-système.

Un bloc est caractérisé par des entrées, des variables d'état, et des sorties :



On peut écrire les équations suivantes, avec x = [xc ; xd]

$$y = f_0(t, x, u) \qquad \text{(Outputs)}$$

$$\dot{x} = f_d(t, x, u) \qquad \text{(Derivatives)}$$

$$x_{d_{k+1}} = f_u(t, x_c, x_{d_k}, u), \qquad \text{(Update)}$$

2 MEILLEURES RESSOURCES

Matlab Central File Exchange, dans la catégorie Simulink

https://fr.mathworks.com/matlabcentral/fileexchange/?category%5B%5D=simulink%2Findex

Le site de Michael Burke :

https://mburkeonmbd.com/

Le blog de Guy Rouleau :

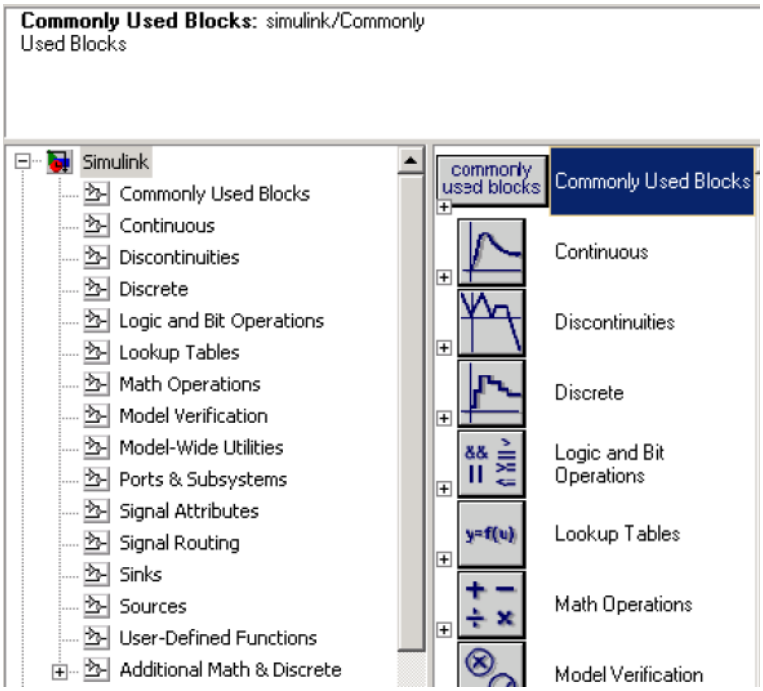
https://blogs.mathworks.com/simulink/

Les playlists de la chaine Mathworks :

- Systems Engineering (2020), par l'excellent Brian Douglas
- MATLAB and Simulink Racing Lounge (2014-2021)
- Tips and tricks for Matlab and Simulink (2019-2021)
- "How To" with MATLAB and Simulink (2019-2021)

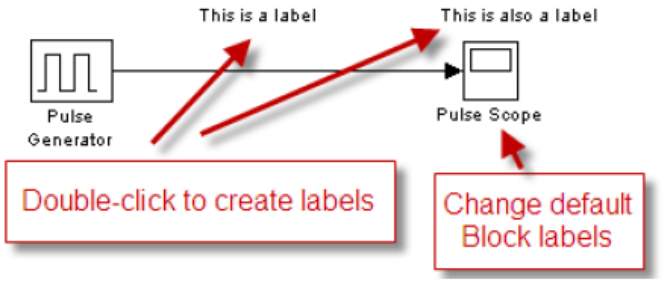
3 ENVIRONNEMENT

3.1 BIBLIOTHEQUE(S)



3.2 ESPACE DE TRAVAIL

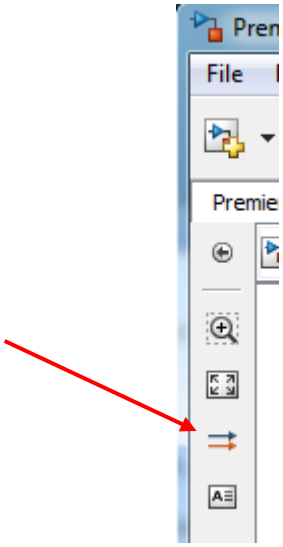
3.2.1 LABELS/COMMENTAIRES



3.2.2 ZOOM ETENDU

Space

3.2.3 AFFICHER LES TEMPS D'ECHANTILLONNAGE



Ou clic-droit sur le modèle / Sample Time Display

Ou menu Display / Sample Time

3.3 RACCOURCIS ET ASTUCES

3.3.1 COPIER UN BLOC

clic droit + glisser

3.3.2 INSERER UN BLOC

le copier sur le connecteur déjà existant

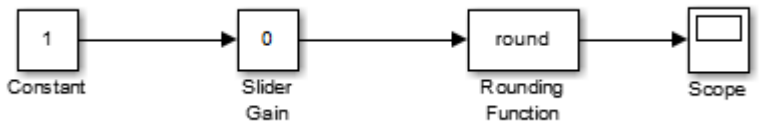
3.3.3 DECONNECTER UN BLOC

clic droit + Maj, puis déplacer le bloc

3.3.4 AJOUTER UN EMBRANCHEMENT

clic droit sur le signal

3.3.5 GENERER UN NOMBRE DISCRET DE CONFIGS



4 CONSTRUCTION D'UN MODELE

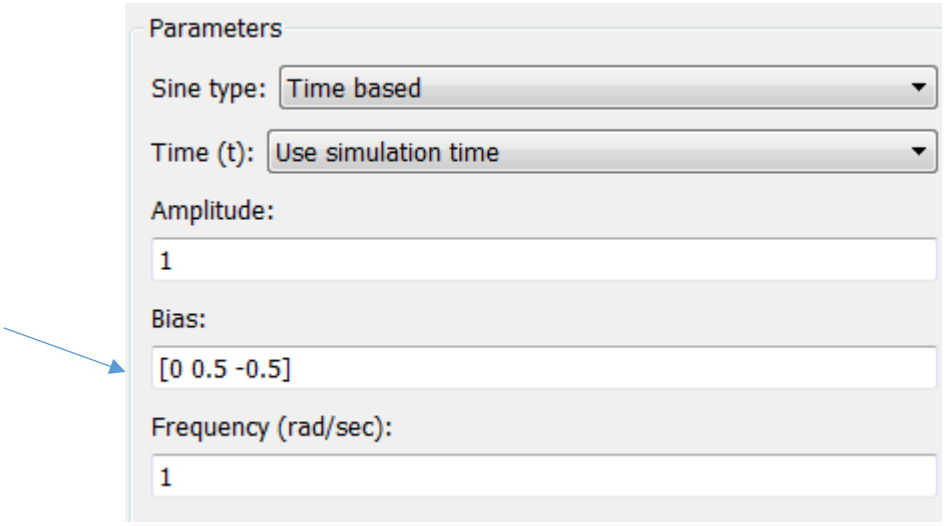
4.1 SIGNAUX

4.1.1 SIGNAUX TEMPORELS

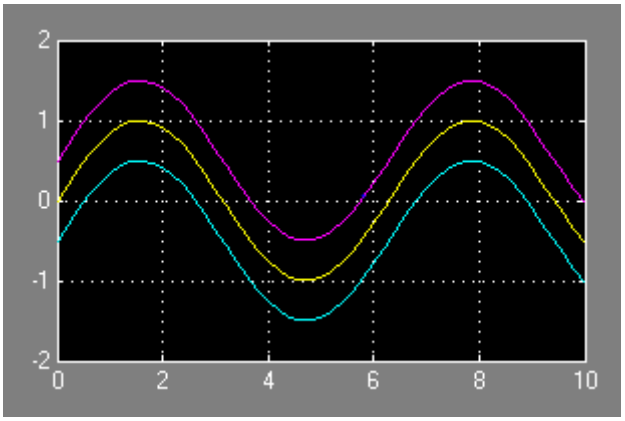
From Workspace : requiert des données en colonnes (temps en 1e colonne, etc …)

From file : requiert des données en ligne !!!

On peut générer plusieurs signaux (dans le même bus) de même type en utilisant des vecteurs pour certaines entrées :



Permet d'obtenir ceci :

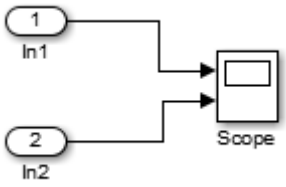


4.1.2 SIGNAL BUILDER

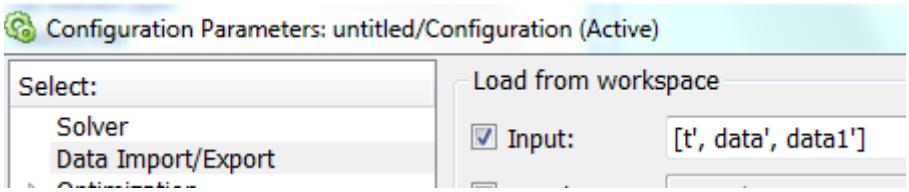
Shift+clic pour ajouter un point

4.1.3 INPUTS GENERIQUES

On peut utiliser des input ports, même à l'échelle d'un modèle (et pas d'un sous-système) :



Puis dans **Simulation / Model Configuration Parameters**, on spécifie les Inputs dans Data/Import :



4.1.4 RAMPE

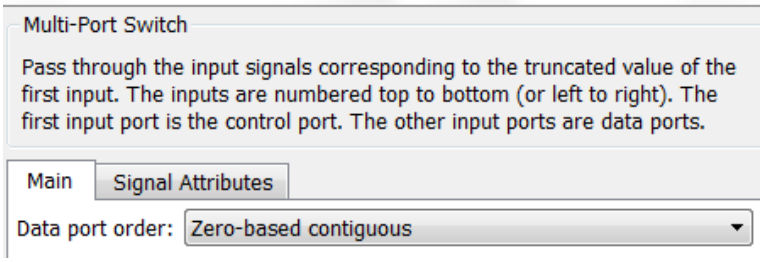


Sample Time	Inherited from driven block
Scalar Expansion	Yes
Dimensionalized	Yes
Zero-Crossing Detection	Yes

4.1.5 SIGNAL ROUTING

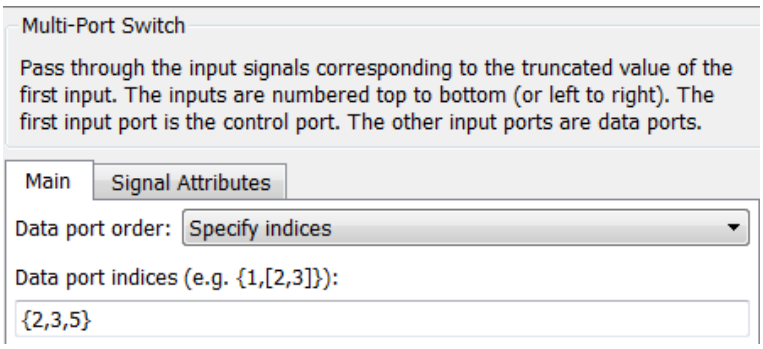
Multi-port switch

On peut spécifier une liste de « configs » en entrée qui débutent à 0 :



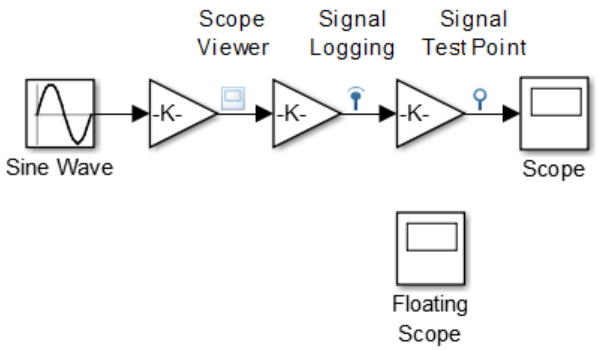
... ou encore à 1 (« *One-based contiguous* »)

… ou bien spécifier soi-même la liste des sélecteurs, entre accolades :



4.1.6 SCOPE

[Scope Blocks and Scope Viewer Overview]



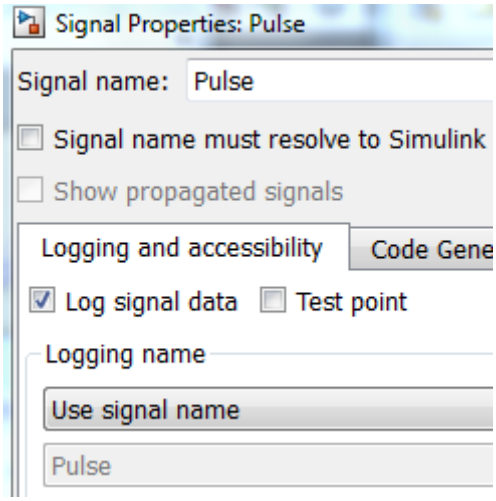
4.1.6.1 VIEWER SUR UN SIGNAL

Clic droit sur un signal / Create & Connect Viewer / Scope



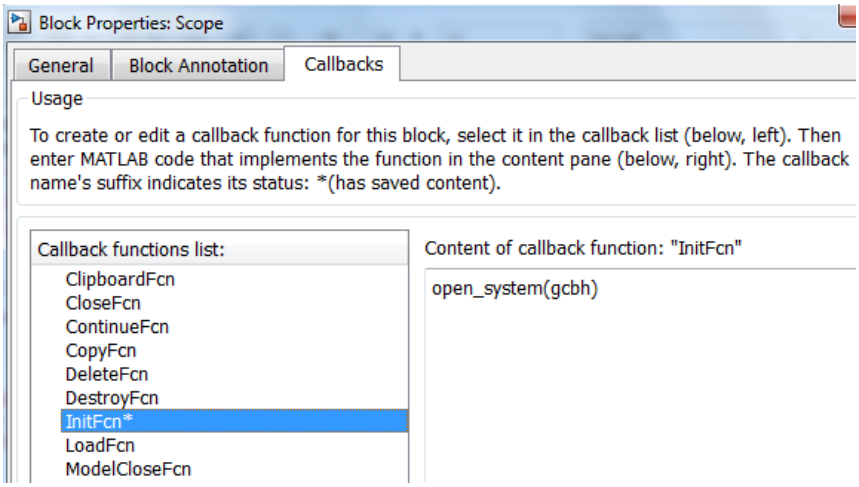
4.1.6.2 LOGGER UN SIGNAL

Clic droit / Properties, cocher Log signal data



4.1.6.3 OUVERTURE AUTOMATIQUE D'UN SCOPE

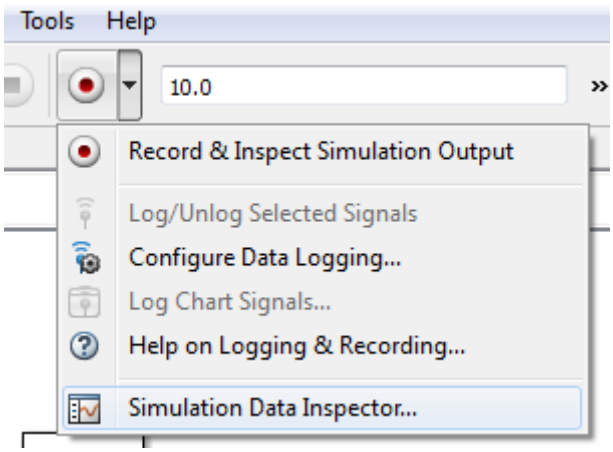
Il faut y ajouter un callback InitFcn :



PS : gcbh pour Get Current Block Handle

4.1.6.4 SIMULATION DATA INSPECTOR

Permet de visualizer les signaux loggés



4.1.7 BUS OU MUX ?

Mux signals must have the same data type and numeric type

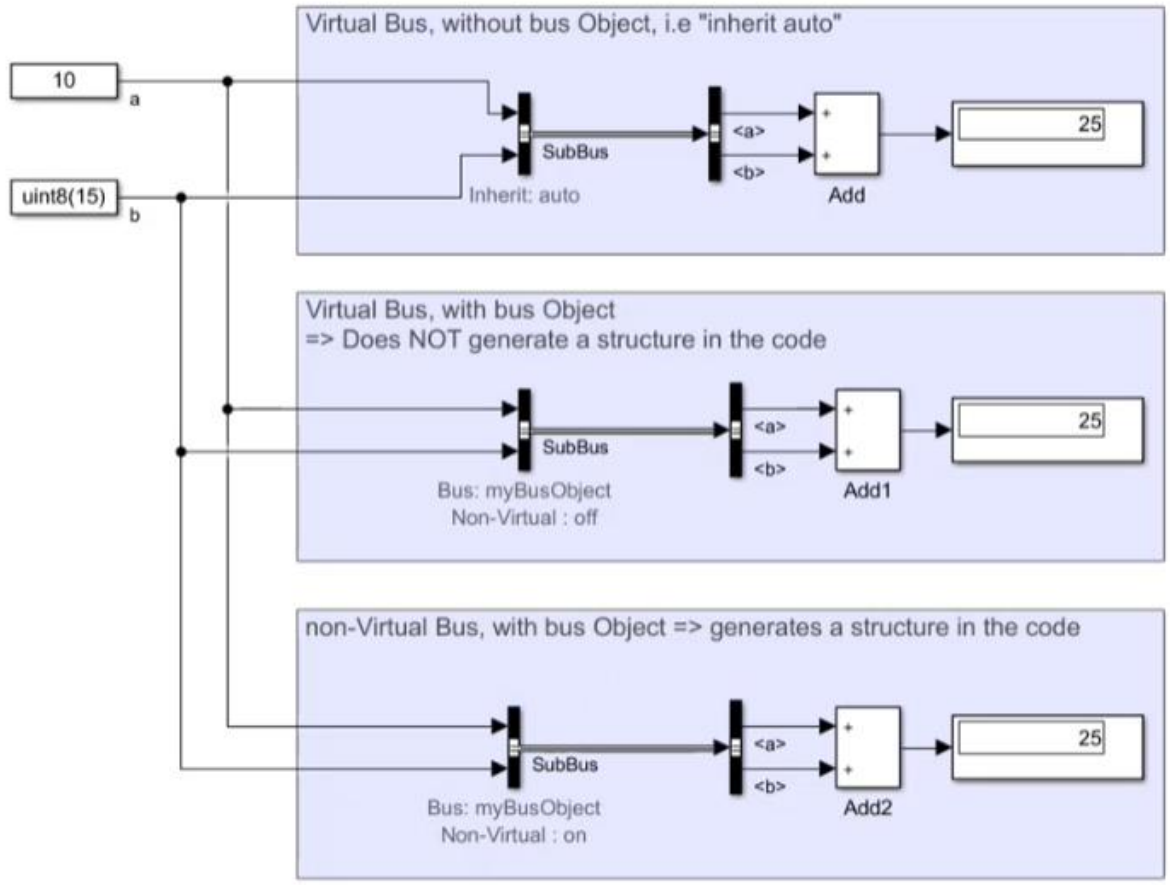
Line Style	Composite Signal Type
	Virtual bus
	Nonvirtual bus
	Nonscalar signal when the **Nonscalar Signals** information overlay is enabled (includes index-based composite signals)
	Index-based composite signal that contains nonvirtual buses

https://fr.mathworks.com/matlabcentral/answers/97787-what-is-the-difference-between-the-mux-demux-and-bus-creator-bus-selector-blocks-in-simulink

- The main difference between mux signals and bus signals is that the components of a bus signal can include more than one data type and can be hierarchical, while the components of a mux signal must all have the same data type and cannot be hierarchical
- Buses can be realized as structures in generated code, but muxes cannot
- MathWorks strongly discourages mixing muxes and buses. For example, do not use Mux blocks as Bus Creators
- The Demux block has a Bus Selector mode
- Using a Bus Selector block, you can change the order of signals. This cannot be done with a Demux block.
- Although the Demux block cannot rearrange the order of muxed signals, the Signal Selector block can

4.1.8 BUS VIRTUELS

Un bus virtuel est un élément purement graphique. Il ne se « voit pas » dans le code généré par exemple.



On utilise le type Inherit :auto pour définir un bus virtuel (c'est le choix par défaut comme Output data type)

NB : un Bus object peut être créé interactivement, via le menu Edit / Bus Editor

NB #2 : on peut ensuite sauver l'objet Bus généré en utilisant Simulink.saveVars

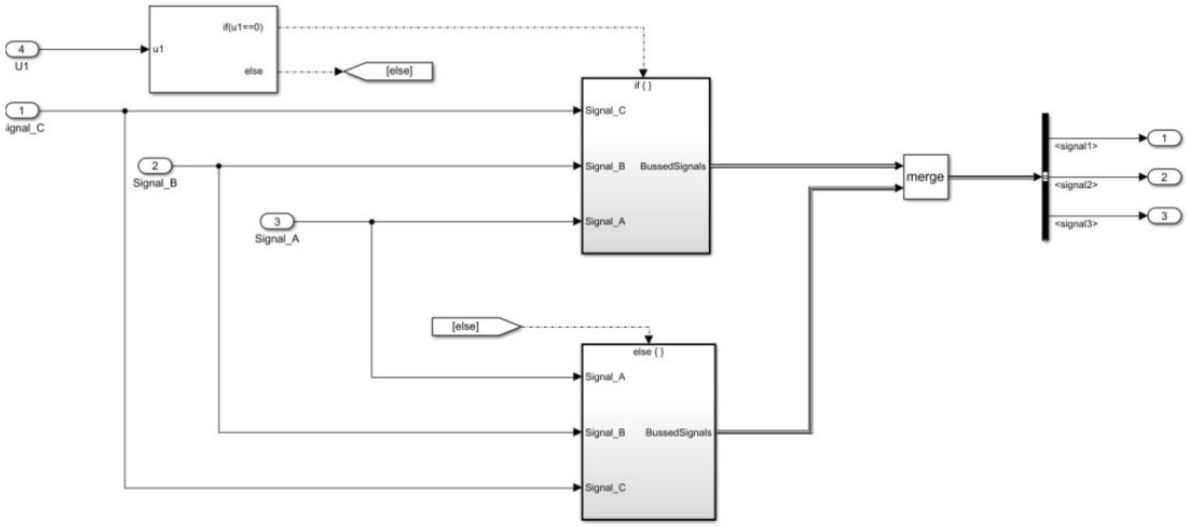
C'est en cochant l'option suivante qu'on pourra créer un bus non-virtuel



https://mburkeonmbd.com/2017/10/03/5-tips-for-more-readable-simulink-models/

Use of Virtual busses for conditionally executed subsystems

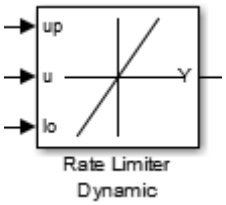
Merging data from conditionally executed subsystems requires the use of the Merge block. When the subsystems have multiple output ports routing the data quickly becomes cumbersome. This can be addressed by creating a virtual bus to pack and then unpack the signals.



Note: Using a virtual bus will allow Simulink/Embedded Coder to optimize the memory placement of the signals. If a structure is desired, then a bus object should be defined and used.

4.1.9 PASSAGE EN DISCRET

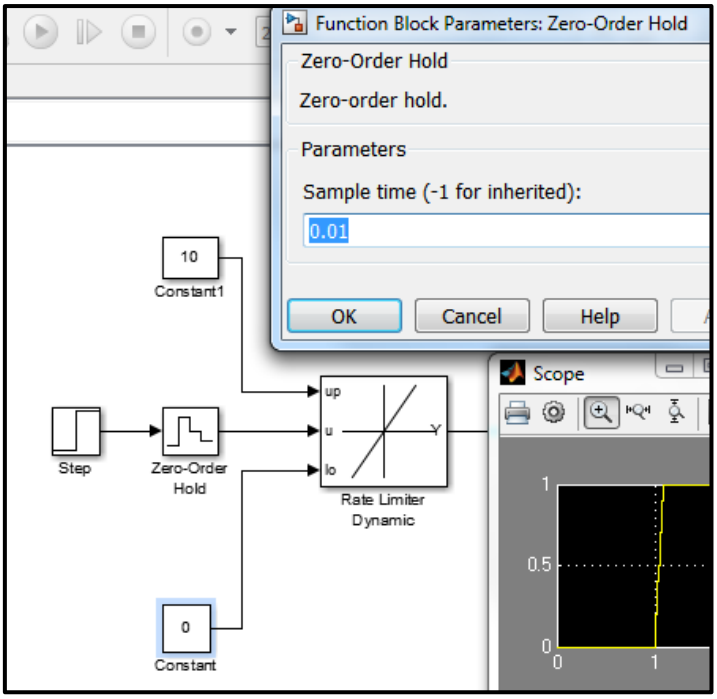
Certains signaux d'origine continu ne peuvent être traités par certains blocs qui s'appuient sur une nature discrète (par exemple le bloc **Rate Limiter Dynamic**)



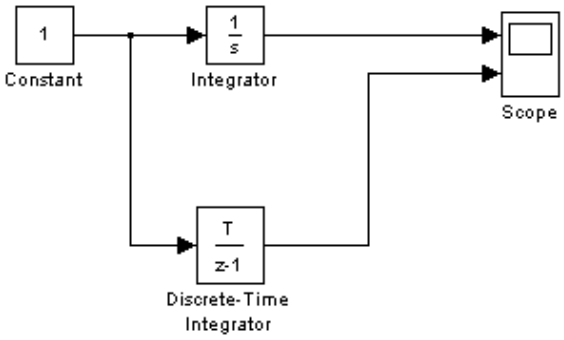
Si on essaie de l'utiliser tel quel sur une source type Step, on obtient le message suivant :

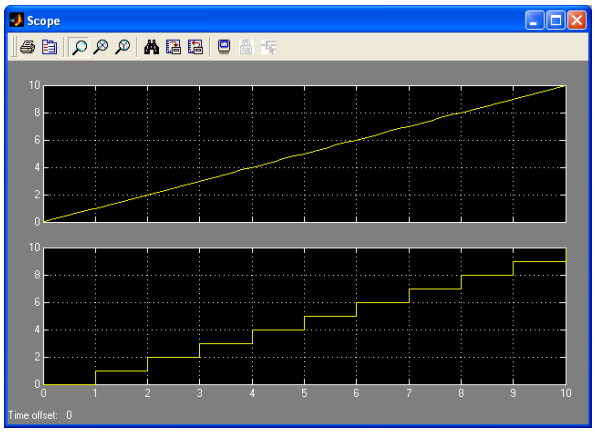
```
Continuous sample time is not supported by block 'untitled/Rate
Limiter Dynamic/sample time'
```

Il faut discrétiser le signal d'entrée, par exemple avec un bloc **Zero-Order Hold** (ZOH) :



Voir également **Rate Transition**, qui peut se comporter comme un bloc ZOH selon les options cochées.





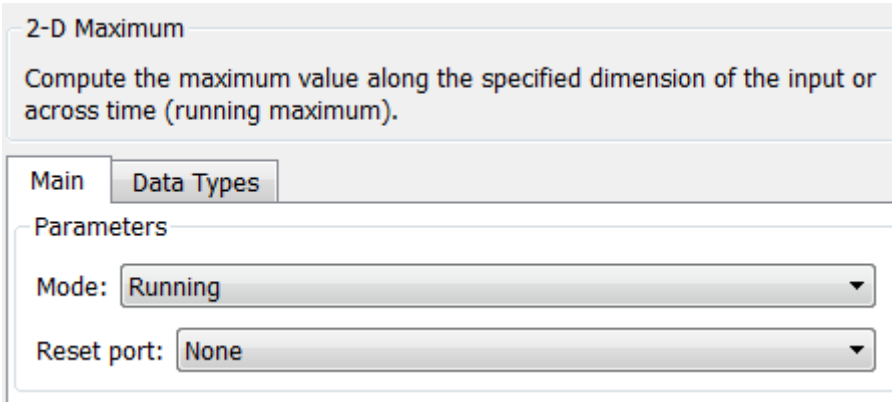
Analysis / Control Design / Model Discretizer

4.1.10 DETECTER LE MAX D'UN SIGNAL TEMPOREL

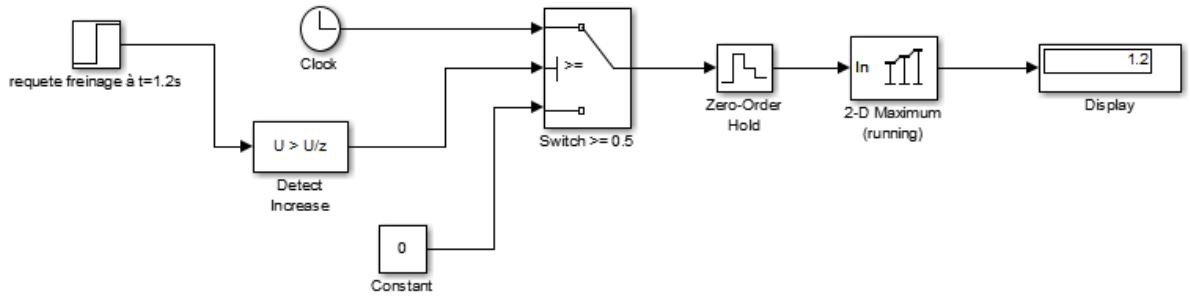
Bloc 2D Maximum



Paramétré sur « Running »



4.1.11 DETECTER L'INSTANT DE VARIATION D'UN SIGNAL FAÇON STEP



… avec des blocs paramétrés comme suit :

Detect Increase (mask) (link)

If the input is strictly greater than its previous value, then output TRUE, otherwise output FALSE. The initial condition determines the initial value of the previous input U/z.

Parameters

Initial condition: 0.0

Input processing: Inherited

Output data type: uint8

Zero-order hold.

Parameters

Sample time (-1 for inherited): 0.01

2-D Maximum

Compute the maximum value along the specified dimension of the input or across time (running maximum).

Main | Data Types

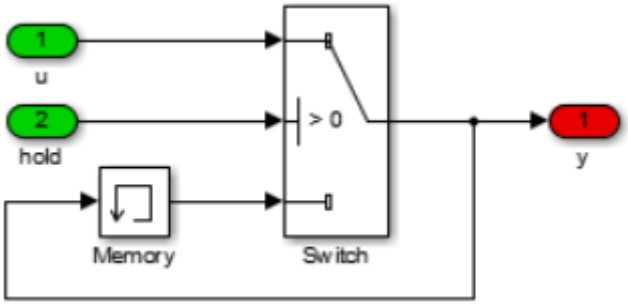
Parameters

Mode: Running

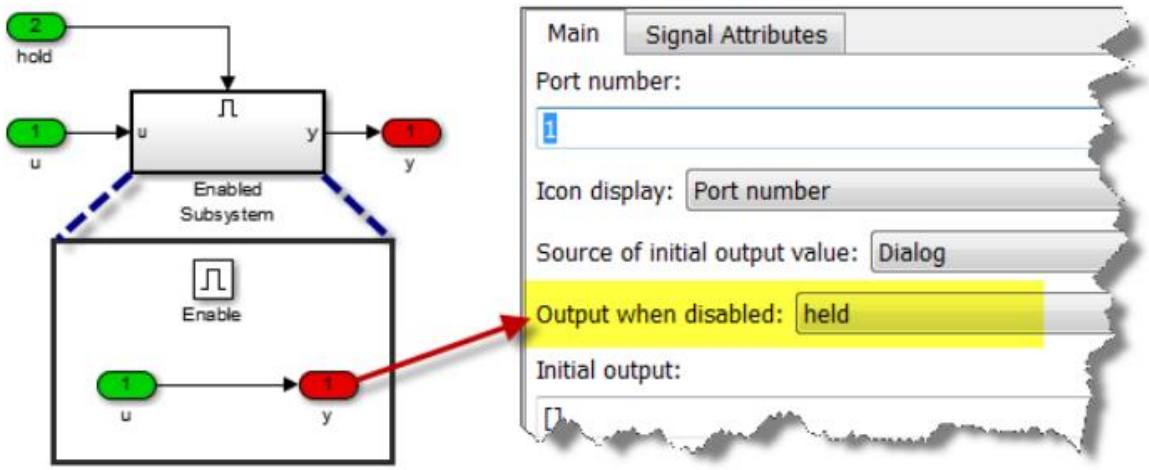
Reset port: None

4.1.12 CONSERVER LA VALEUR D'UN SIGNAL LORSQU'UN TRIGGER EST DECLENCHE

https://blogs.mathworks.com/simulink/2014/08/06/how-do-you-hold-the-value-of-a-signal/

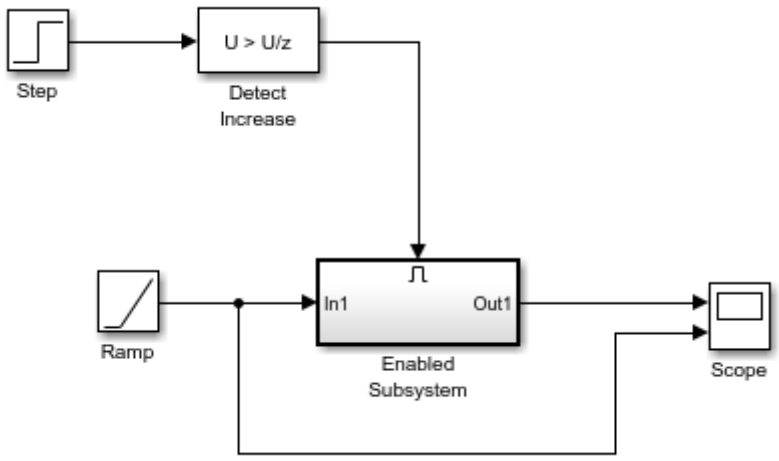


Autre méthode assez élégante, un subsystem enabled :



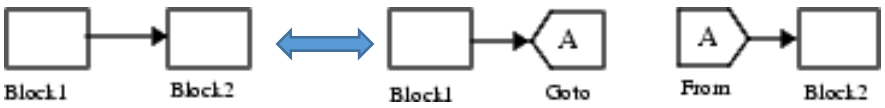
Sample and Hold

NB : cette méthode ne fonctionne que si le déclencheur est de type dirac. Sinon il faut ajouter un detect increase :



4.2 FROM/GOTO

Permettent de passer des signaux d'un bloc à l'autre sans les connecter, ce qui permet d'une part d'alléger la représentation graphique d'un modèle, et surtout de passer aisément des signaux d'un subsystem à un autre.



A Goto block can pass its input signal to more than one From block, although a From block can receive a signal from only one Goto block

The Tag Visibility parameter determines whether the location of From blocks that access the signal is limited:

- **local**, the default, means that From and Goto blocks using the same tag must be in the same subsystem. A local tag name is enclosed in brackets ([]).
- **scoped** means that From and Goto blocks using the same tag must be in the same subsystem or at any level in the model hierarchy below the Goto Tag Visibility block that does not entail crossing a nonvirtual subsystem boundary, i.e., the boundary of an atomic, conditionally executed, or function-call subsystem or a model reference. A scoped tag name is enclosed in braces ({}).
- **global** means that From and Goto blocks using the same tag can be anywhere in the model except in locations that span nonvirtual subsystem boundaries.

Nota : dans la plupart des cas, le MAAB recommande d'utiliser des From/Goto locaux uniquement.

4.3 SIMULINK PARAMETERS



set_simulink_param.m

```
myGain = Simulink.Parameter;
myGain.Value = 2;
```

Use a `Simulink.Parameter` object to:

- Share a value among multiple block parameters.
- Represent an engineering constant or a tunable calibration parameter.
- Separate a parameter value from its data type and other properties.
- Configure parameter data for code generation.

https://fr.mathworks.com/help/simulink/slref/parameter.html

On peut aussi utiliser une expression pour affecter la valeur :

```
myParam.Value = slexpr('myVar + myOtherVar')
```

4.4 PROTECTED MODELS

Depuis 2018b

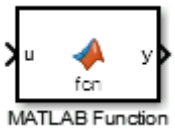
4.5 INTEGRATION DE CODE

4.5.1 MATLAB

Sources :

- Create Model That Uses MATLAB Function Block

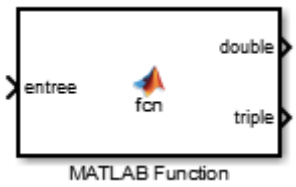
On peut intégrer du code avec le bloc **MATLAB Function** (library **User-defined functions**)



Editer le bloc pour ajouter une ligne d'entête. Ici une fonction qui calcule le double et le triple d'une entrée :

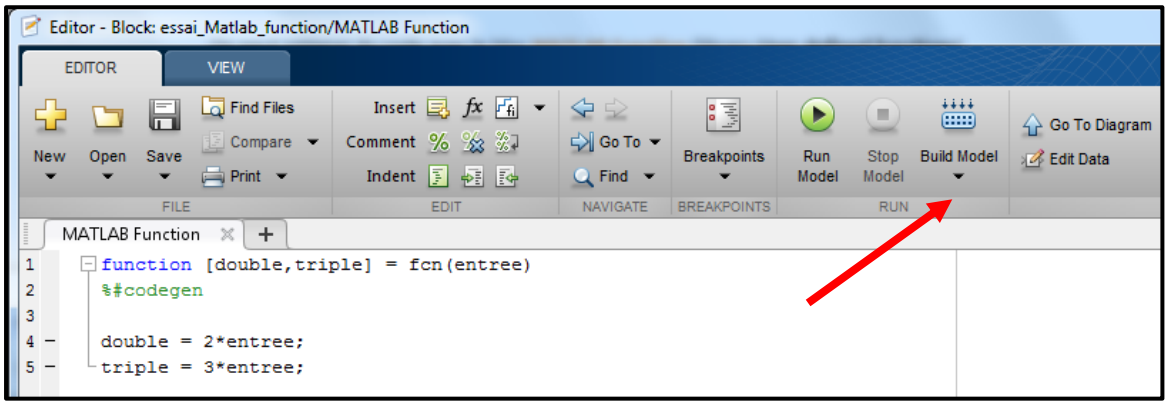
```
function [double,triple] = fcn(entree)
```

Si on enregistre le bloc à ce moment on peut constater que les inputs et outputs ont été mis à jour :

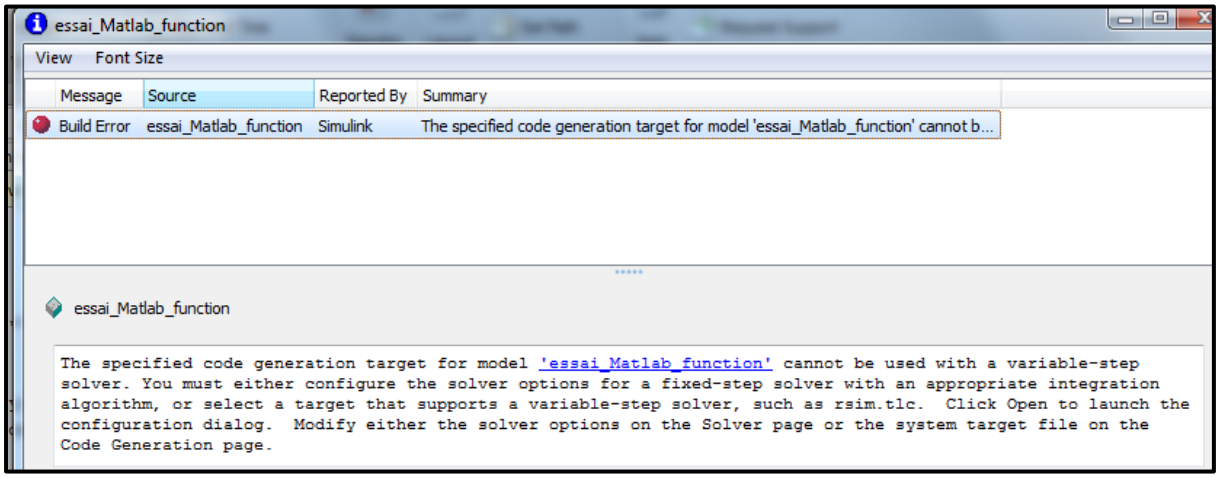


Une fois le bloc complété avec le code de la fonction, il faut ensuite :

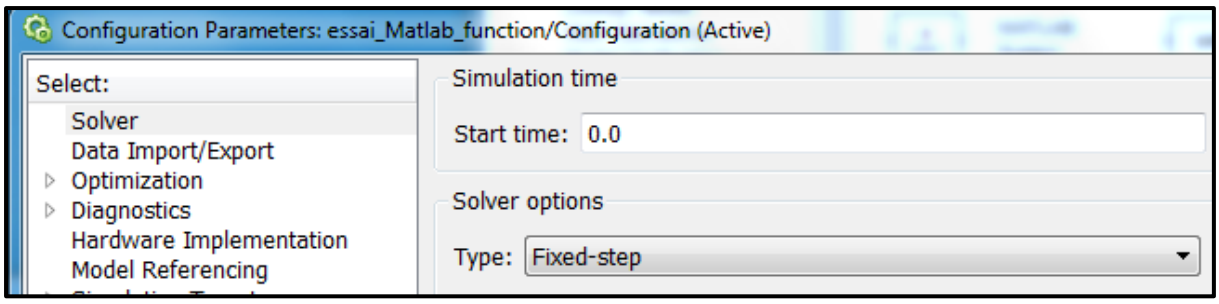
- Spécifier le compilateur si ça n'a pas été fait (`mex -setup` sous Matlab)
- **Build model / Build** (dans l'éditeur du bloc)



Ici, sans précaution particulière, on obtient un message d'erreur pour ne pas avoir sélectionné un solveur compatible :



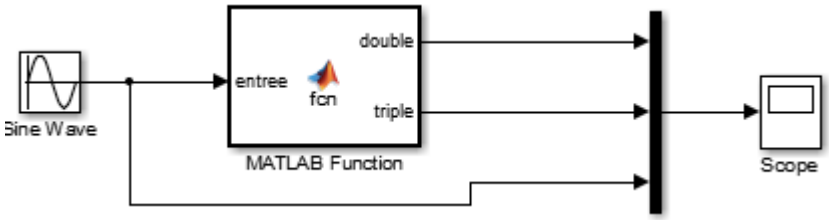
On corrige donc cela en choisissant un solveur à pas fixe :

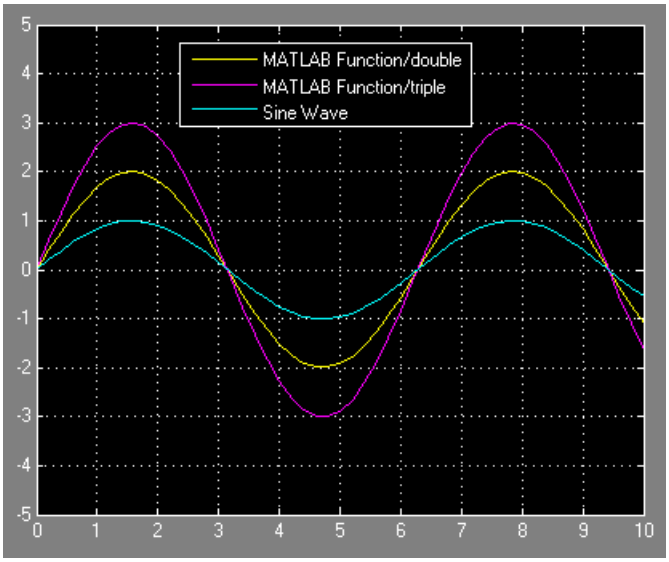


On recommence le Build et patiente jusqu'à ce que la console affiche

```
### Created: ..\essai_Matlab_function.exe
### Successfully generated all binary outputs.
### Successful completion of build procedure for model: essai_Matlab_function
>>
```

On peut ensuite utiliser ce bloc comme n'importe quel autre :





4.5.2 FONCTIONS MATLAB SYMBOLIQUES

On ne peut pas utiliser directement de variables symboliques ni de fonctions anonymes dans un bloc Matlab Function. En revanche on peut créer un .m « externe » et l'appeler dans Simulink par coder.extrinsic :

https://stackoverflow.com/questions/42271181/how-to-use-syms-in-simulink-matlab-function-block

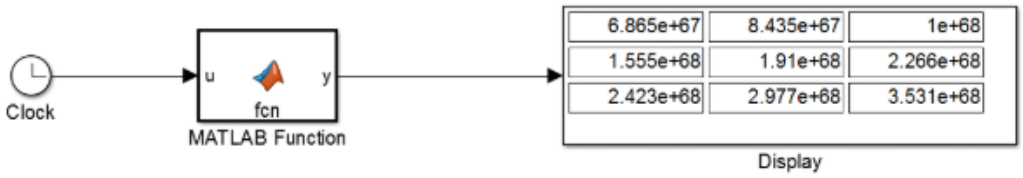
myIntegral.m

```
function out = myIntegral(in)
A = [1 2 3; 4 5 6; 7 8 9];
myfun = @(s) expm(A.*s);
out = integral(myfun,0,in,'ArrayValued',true);
end
```

Matlab Function block code:

```
function y = fcn(u)
%#codegen
coder.extrinsic('myIntegral');
y = zeros(3);
y = myIntegral(u);
```

It works:



Un autre exemple ici :

https://fr.mathworks.com/matlabcentral/answers/154695-how-to-use-symbolic-variables-and-functions-syms-in-a-simulink-matlab-function

Create a new .m file, e.g. solveSyms.m, put all your code there:

```
function out = solveSyms(in)  % add necessary inputs and outputs
  syms x y
  f = x^2*y + 5*x*sqrt(y);
  out = subs(f, x, in);
end
```

Now call this new file from the MATLAB Function block:

```
coder.extrinsic('solveSyms');
out = 0;
out = solveSyms(in);
```

Voir §10.6 pour coder.extrinsic

4.5.3 MATLABFUNCTIONBLOCK

Cette fonction permet depuis Matlab, à condition qu'un système soit ouvert, de créer un bloc à partir d'une fonction symbolique

https://fr.mathworks.com/help/symbolic/generate-matlab-function-blocks.html

```
syms x y
r = sqrt(x^2 + y^2);
matlabFunctionBlock('my_system/my_block', r)
```

On peut contrôler l'ordre des inports (qui sont par défaut rangés dans l'ordre alphabétique des noms de variables de la fonction)

```
syms x y
mu = sym('mu');
dydt = -x - mu*y*(x^2 - 1);
matlabFunctionBlock('my_system/vdp', dydt,'vars', [y mu x])
```

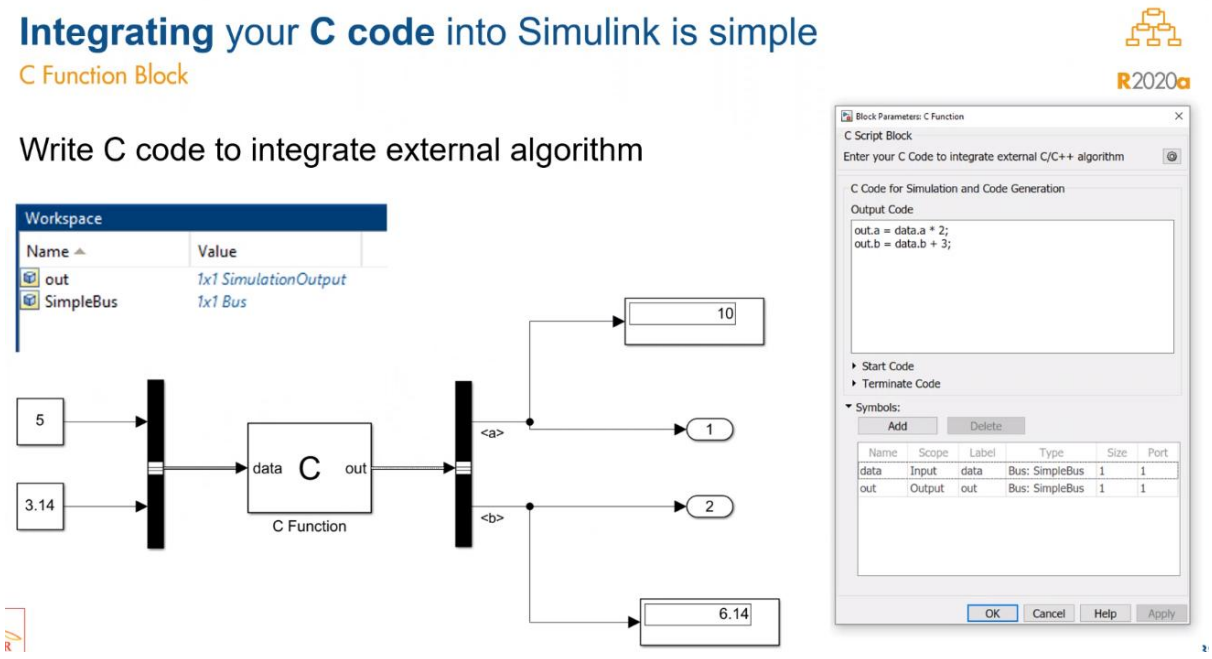
… et nommer les outports

```
syms x y
mu = sym('mu');
dydt = -x - mu*y*(x^2 - 1);
matlabFunctionBlock('my_system/vdp', dydt,'outputs',{'name1'})
```

4.5.4 CALLING SHARED LIBRARIES FROM SIMULINK

https://fr.mathworks.com/matlabcentral/fileexchange/44197-calling-shared-libraries-from-simulink

4.5.5 INTEGRATION DE CODE C (SINCE 2020)



4.6 S-FUNCTION

4.6.1 INTRO

http://fr.mathworks.com/help/simulink/s-function-basics.html

Une S-Function est un bloc qui permet d'intégrer du code externe (souvent du C) dans un modèle Simulink.

Les S-functions peuvent être compilées sous la forme de MEX-files par l'utilitaire `mex`.

Simulink supporte les pointeurs sur des entiers, des flottants mais pas des caractères. Pour importer une fonction C dans Simulink, celle-ci doit avoir un prototype avec des arguments valides constitués de pointeurs ou valeurs sur des types entiers ou flottants. Au besoin il faudra créer une fonction Wrapper acceptant des arguments valides qui appellera la fonction souhaitée utilisant des arguments non valides.

4.6.2 IMPLEMENTATIONS POSSIBLES D'UNE S-FUNCTION

Available S-Function Implementations

- A **Level-1 MATLAB S-function** provides a simple MATLAB interface to interact with a small portion of the S-function API. Level-2 MATLAB® S-functions supersede Level-1 MATLAB S-functions.
- A **Level-2 MATLAB S-function** provides access to a more extensive set of the S-function API and supports code generation. In most cases, use a Level-2 MATLAB S-function when you want to implement your S-function in MATLAB.
- A **handwritten C MEX S-function** provides the most programming flexibility. You can implement your algorithm as a C MEX S-function or write a wrapper S-function to call existing C, C++, or Fortran code. Writing a new S-function requires knowledge of the S-function API and, if you want to generate inlined code for the S-function, the Target Language Compiler (TLC).
- The **S-Function Builder** is a graphical user interface for programming a subset of S-function functionality. If you are new to writing C MEX S-functions, you can use the S-Function Builder to generate new S-functions or incorporate existing C or C++ code without interacting with the S-function API. The S-Function Builder can also generate TLC files for inlining your S-function during code generation with the Simulink® Coder™ product.
- The **Legacy Code Tool** is a set of MATLAB commands that helps you create an S-function to incorporate legacy C or C++ code. Like the S-Function Builder, the Legacy Code Tool can generate a TLC file to inline your S-function during code generation. The Legacy Code Tool provides access to fewer of the methods in the S-function API than the S-Function Builder or a handwritten C MEX S-function.

La création d'une S-function peut être automatisée par le biais d'un script à base du legacy code tool.

http://fr.mathworks.com/help/simulink/sfg/integrating-existing-c-functions-into-simulink-models-with-the-legacy-code-tool.html

4.6.3 C MEX S FUNCTIONS

Pour créer des S-functions à partir de code C, voir

https://fr.mathworks.com/help/simulink/sfg/example-of-a-basic-c-mex-s-function.html

Trois approches possibles :

- **Handwritten S-function** — You can write a C MEX S-function from scratch. (Basic C MEX S-Function provides a step-by-step example.) See Templates for C S-Functions for a complete skeleton implementation of a C MEX S-function that you can use as a starting point for creating your own S-functions.
- **S-Function Builder** — This block builds a C MEX S-function from specifications and code fragments that you supply using a graphical user interface. This eliminates the need for you to write S-functions from scratch. See Build S-Functions Automatically for more information about the S-Function Builder.
- **Legacy Code Tool** — This utility builds a C MEX S-function from existing C code and specifications that you supply using MATLAB® code. See Integrate C Functions Using Legacy Code Tool for more information about integrating legacy C code into Simulink® models.

Pour écrire une S-function "manuellement", on peut s'appuyer sur le template qui est expliqué ici :

http://fr.mathworks.com/help/simulink/sfg/example-of-a-basic-c-mex-s-function.html

Le Legacy Code Tool permet de simplifier ce processus (il effectue en quelque sorte lui-même le « wrapping » de la fonction C initiale avec les prérequis pour une utilisation sous Simulink.

Voir aussi : C MEX S-Function Examples

4.6.4 S-FUNCTION BUILDER

http://fr.mathworks.com/help/simulink/sfg/s-function-builder-dialog-box.html

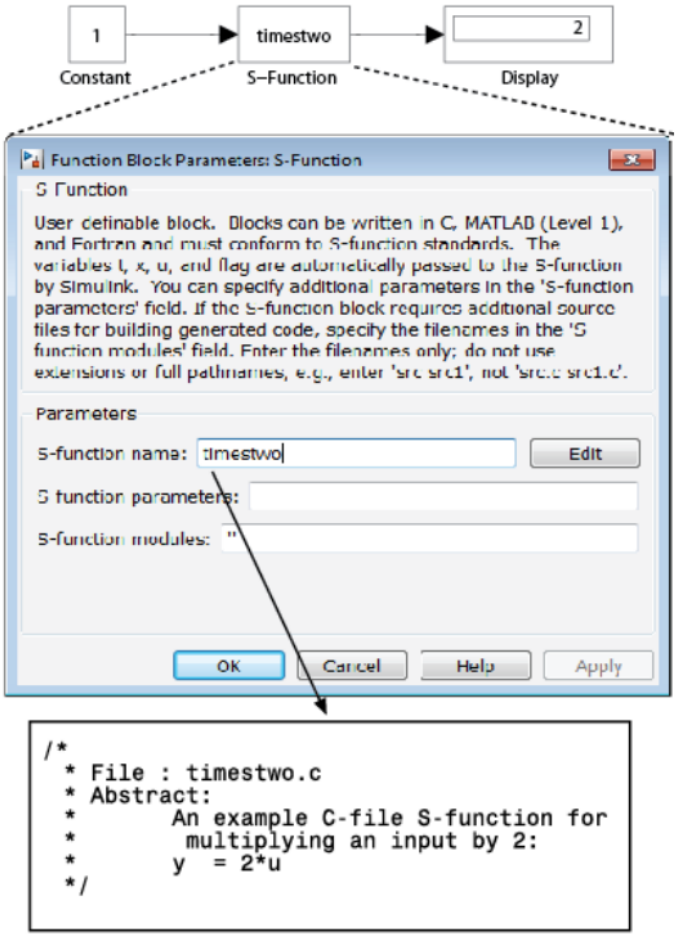
4.6.5 COMMENT CHOISIR ?

If you...	Then use...
Are a MATLAB® programmer with little or no C programming experience	A Level-2 MATLAB S-function, especially if you do not need to generate code for a model containing the S-function (see Write Level-2 MATLAB S-Functions).
Need to generate code for a model containing the S-function	Either a Level-2 MATLAB S-function or a C MEX S-functions. Level-2 MATLAB S-functions require that you write a Target Language Compiler (TLC) file for your S-function, before generating code. C MEX S-functions, however, automatically support code generation.
Need the simulation to run faster	A C MEX S-function, even if you do not need to generate code (see Creating C MEX S-Functions). For complicated systems, Level-2 MATLAB S-functions simulate slower than C MEX S-functions because they call out to the MATLAB execution engine.
Need to implement the S-function in C, but have no previous experience writing C MEX S-functions	The S-Function Builder.
Are incorporating legacy code into the model	Any S-function, with the exception of a Level-1 MATLAB S-function. Consider using the Legacy Code Tool if your legacy function calculates only outputs, not dynamic states (see Integrate C Functions Using Legacy Code Tool). Otherwise, consider using the S-Function Builder. If you need to call the legacy code during simulation, do not use a Level-2 MATLAB S-function because they call legacy code only through their TLC files.
Need to generate embeddable code for an S-function that incorporates legacy code	The Legacy Code Tool if your legacy function calculates only outputs. Otherwise, use a handwritten C MEX S-function or the S-Function Builder.

source

4.6.6 EXEMPLE

http://fr.mathworks.com/help/simulink/sfg/using-s-functions-in-models.html



4.6.7 DIRECT FEEDTHROUGH

Direct feedthrough means that the output (or the variable sample time for variable sample time blocks) is controlled directly by the value of an input port signal.

Il est important de positionner correctement le flag direct feedthrough.

If the simulation results for a model containing your S-function do not converge, or the simulation fails, you may have the direct feedthrough flag set incorrectly. Try turning on the direct feedthrough flag and setting the Algebraic loop solver diagnostic to warning (see the Algebraic loop option on the Diagnostics Pane: Solver reference page in Simulink Graphical User Interface). Subsequently running the simulation displays any algebraic loops in the model and shows if the engine has placed your S-function within an algebraic loop.

4.7 CALLBACKS

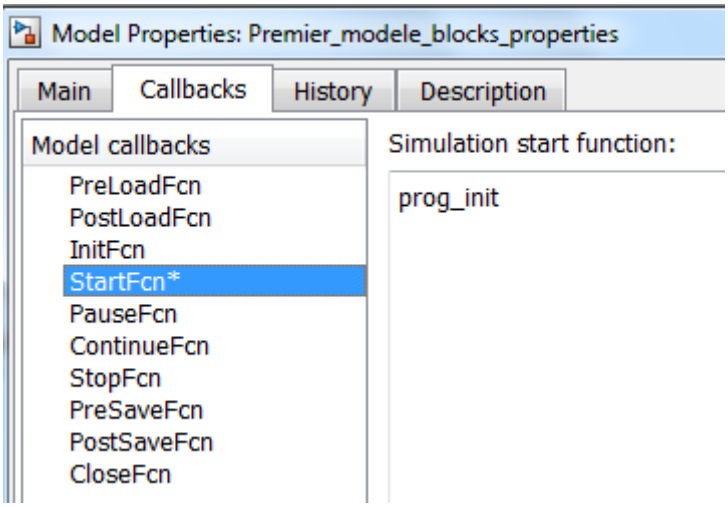
4.7.1 A L'ECHELLE DU BLOC

Clic droit sur un bloc / Properties

Définit les actions à réaliser lors de l'exécution de chaque bloc.

4.7.2 A L'ECHELLE DU MODELE

File / Model Properties



On voit avec un astérisque les fonctions auxquelles un callback est affecté.

Ici le fichier prog_init.m est associé au démarrage du modèle.

4.8 PRECONISATIONS

4.8.1 STYLE

Voir les précos du Mathworks Advisory Board (MAB) dont il existe une déclinaison automobile : Mathworks Automotive Advisory Board (MAAB)

https://fr.mathworks.com/solutions/mab-guidelines.html

4.8.2 CONSTRUCTION

https://mburkeonmbd.com/2017/10/03/5-tips-for-more-readable-simulink-models/

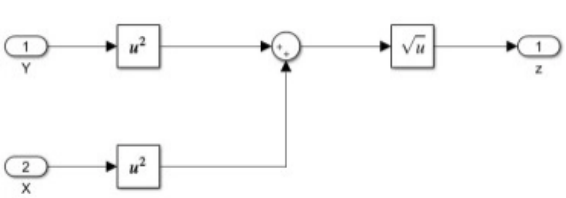
Utiliser des Matlab functions pour les équations :

1. Is there a built-in function for this equation (e.g., Integrators, Transfer Functions, table lookups)? *Use the built-in function*
2. Is the equation more than 3 operations long? *Use a MATLAB function.*

```
function z = Pythagorean(y,x)

z = sqrt(y^2+x^2);

end
```



In text form, the Pythagorean theorem is quickly recognized and understood. When written out as a Simulink equation it takes slightly longer to understand.

The rule of 40, 5 and 2

When I create a Simulink subsystem, I aim to have a limited number of active blocks in the subsystem (40). A limited number of used inputs (5) and a limited number of calculated outputs (2).

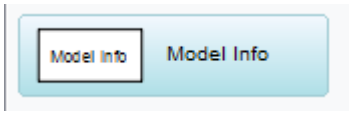
1. **Active blocks:** Any block that performs an operation. This count does not include inports, outports, mux, demux, bus…
    1. **Why**: When the total number of blocks goes above 40 the ability to understand what is going on in the subsystem decreases.
2. **Used inputs:** Bus signals can enter the subsystem, and only one signal off of the bus may be used. A “used signal” is one that is actively used as part of the subsystems calculations.

1. **Why**: The number of used inputs is a good metric for how “focused” the subsystem is in addressing a specific issue.

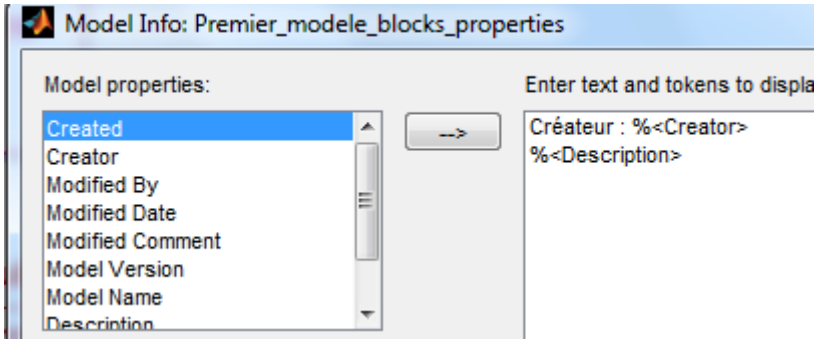
3. **The number of outputs:** directly relates to the first two metrics, e.g.h, how focused the subsystem is on a specific issue.

4.8.3 DOCUMENTATION / COMMENTAIRES

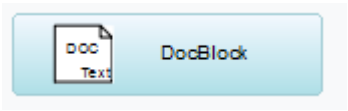
4.8.3.1 MODEL INFO



On peut ajouter des champs de description du modèle :



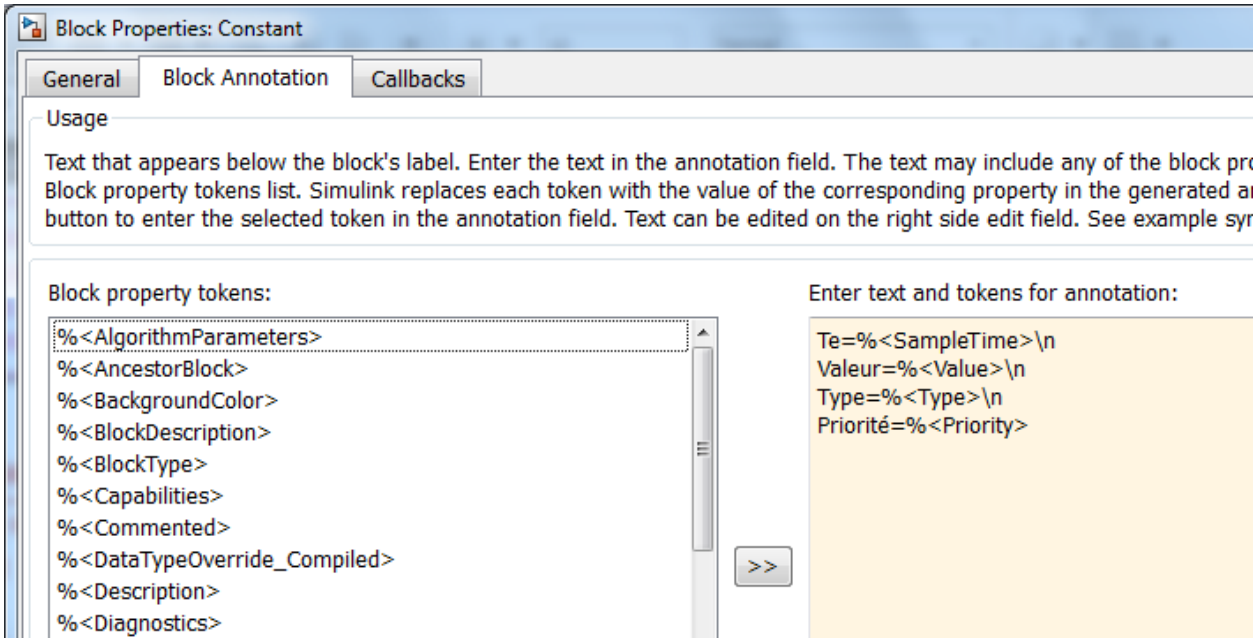
4.8.3.2 AJOUT DE DOCBLOCK



4.8.3.3 AJOUT DE BLOCK ANNOTATION

Clic droit sur un bloc / Properties

Puis on peut ajouter les « tokens » que l’on souhaite :


Block Properties: Constant
General
Block Annotation
Callbacks
Usage
Text that appears below the block's label. Enter the text in the annotation field. The text may include any of the block pr
Block property tokens list. Simulink replaces each token with the value of the corresponding property in the generated a
button to enter the selected token in the annotation field. Text can be edited on the right side edit field. See example syr
Block property tokens:
%<AlgorithmParameters>
%<AncestorBlock>
%<BackgroundColor>
%<BlockDescription>
%<BlockType>
%<Capabilities>
%<Commented>
%<DataTypeOverride_Compiled>
%<Description>
%<Diagnostics>
>>
Enter text and tokens for annotation:
Te=%<SampleTime>\n
Valeur=%<Value>\n
Type=%<Type>\n
Priorité=%<Priority>

5 ARCHITECTURE D'UN MODELE

5.1 SYSTEM COMPOSER (SINCE 2019A)

https://fr.mathworks.com/help/systemcomposer/getting-started-with-system-composer.html
https://fr.mathworks.com/products/system-composer.html
https://fr.mathworks.com/help/systemcomposer/gs/compose-an-architecture-model.html

5.2 MODEL REFERENCE

https://fr.mathworks.com/help/simulink/ug/overview-of-model-referencing-1.html

Model Reference is a standalone file that has very rigid architectural boundaries in both simulation and code generation. They can provide some benefits in simulation and code generation because the generated files don't need to be rebuilt every time. However, Model Reference has some challenges:

1. Signal properties must be specified at the boundaries and thus cannot be inherited from its connecting blocks.
2. Code customization is difficult due to the rigid constraints and limits on flexibility

Conversion advisor
https://fr.mathworks.com/help/simulink/slref/modelreferenceconversionadvisor.html

5.3 SUBSYSTEMS

5.3.1 VIRTUAL VS ATOMIC SUBSYSTEM

Les subsystems à bordure fine sont virtuels : ils ne changent que la représentation graphique du sketch, mais pas (notamment) l'execution order (menu Display / Blocks)

Les subsystems à bordure épaisse sont atomiques.

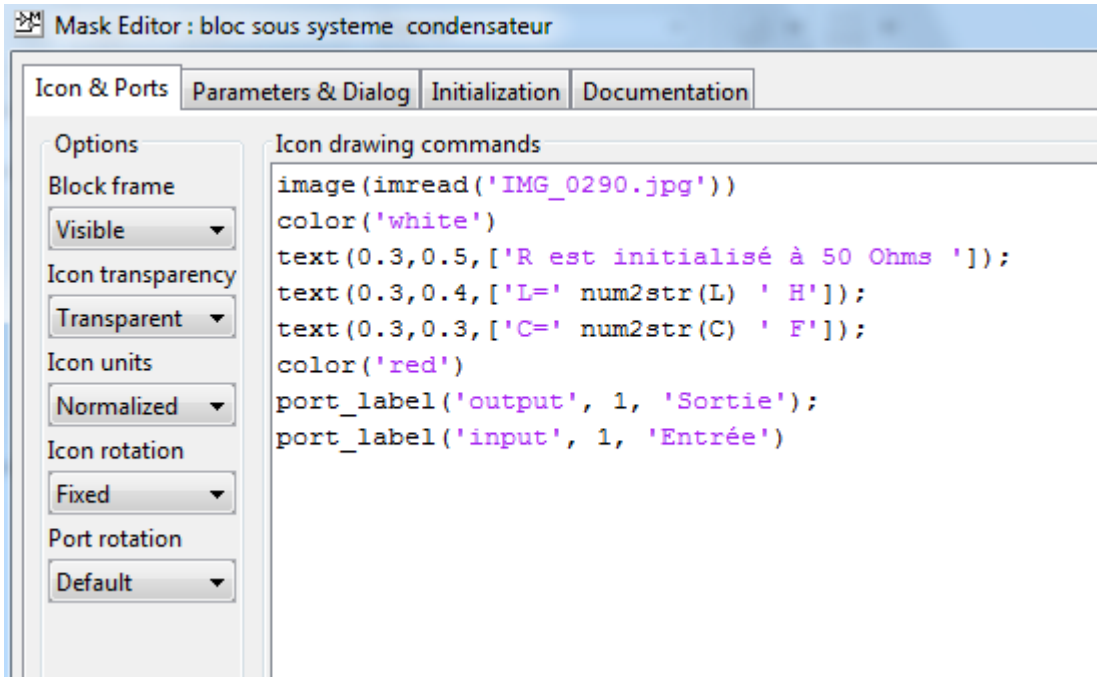
5.3.2 ENABLED SUBSYSTEM

N'accepte que des booléens en entrée.

Si on utilise un signal de pilotage (continu ou discret, peu importe) pour commander un subsystem, la conversion en booléen est nécessaire.

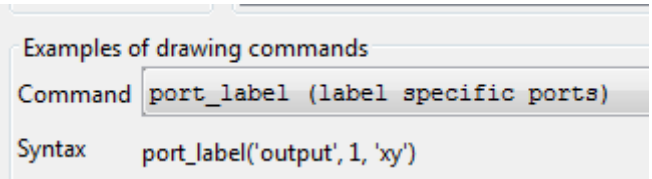
5.3.3 MASQUE

5.3.3.1 APPARENCE DU SUBSYSTEM



Il est conseillé de travailler en « Icon units = normalized »

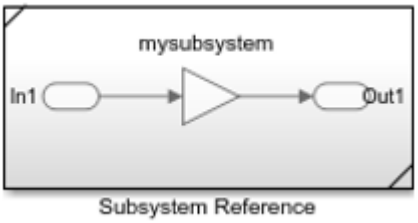
Les commandes disponibles se trouvent dans le menu déroulant situé en-dessous :



5.4 SUBSYSTEM REFERENCE (2019B)

5.4.1 INTRO

Identifié par des triangles dans les coins opposés



Apparu en 2019b

https://blogs.mathworks.com/simulink/2019/11/15/new-in-r2019b-subsystem-reference/

Subsystem reference allows you to save the contents of a subsystem in a separate SLX file and reference it using a Subsystem Reference block. You can create multiple instances referencing the same subsystem file

A subsystem file:

- Cannot be simulated.
- Does not have a configuration set.
- Does not have a model workspace.
- Does not have code generation capability.

5.4.2 AVANTAGES

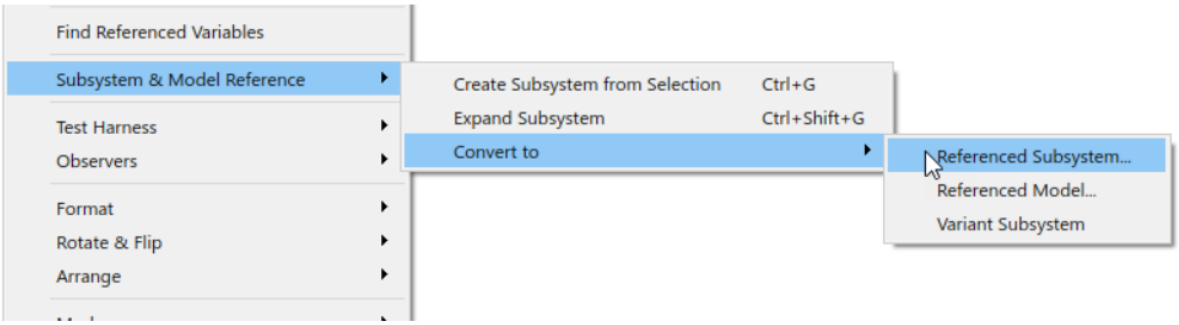
Fichier séparé => on peut faire du revision control

Moins rigide qu'un model reference (notamment en ce qui concerne la définition des signaux aux interfaces)

5.4.3 DIFFÉRENCE AVEC UNE LIBRAIRIE

Pas de “disable link”

5.4.4 CONVERTIR UN SUBSYSTEM EXISTANT



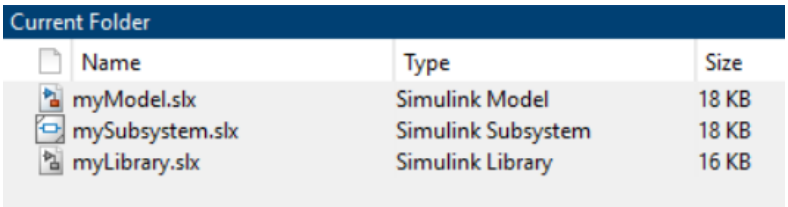
5.4.5 INTERACTION EN LIGNE DE COMMANDE

Création

```
new_system(subsystemfilename,'subsystem')
```

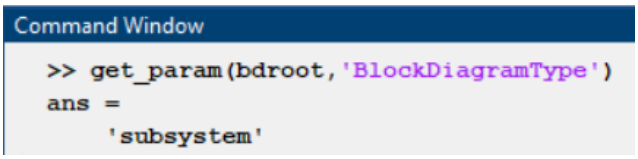
5.4.6 DISTINGUER UN MODELE D’UN SUBSYSTEM

On peut faire la différence entre un modèle et un sous-système (les 2 portent la même extension !) dans l’explorateur Matlab :



Ou en ligne de commande :

- si le .slx est ouvert



- sinon

```
>> Simulink.MDLInfo('mySubsystem.slx')
ans =
  MDLInfo with properties:

          BlockDiagramName: 'mySubsystem'
```

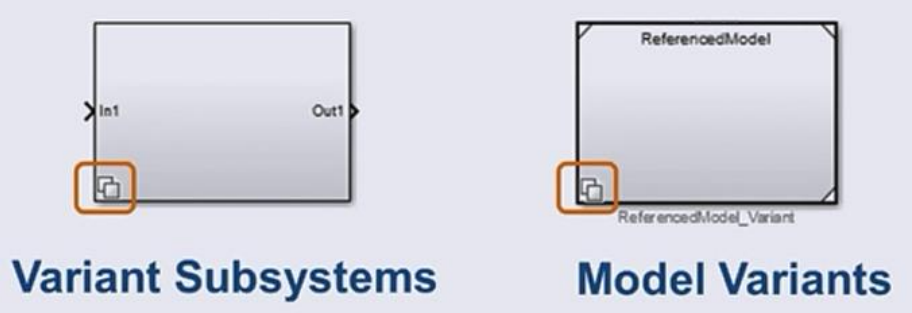
5.5 VARIANTES

5.5.1 INTRO

On peut créer un modèle "modulaire" contenant plusieurs variantes, sans avoir à les gérer manuellement sous la forme de .mdl distincts.

https://fr.mathworks.com/help/simulink/variant-systems.html

https://www.youtube.com/watch?v=TYP7t8dJJ_0



Un gestionnaire de configurations, ou **Variant Manager**, est disponible à cet effet.

On peut utiliser des **variant subsystems** pour construire un modèle "hiérarchique", ou bien des blocs **variant source** et **variant sink.**

	Variant Source and Variant Sink blocks	Variant Subsystem block	Model Variant block
Variant choice representation	Number of ports	Subsystem or Model block	Only Model block
Allows choice hierarchy	No	Yes	Yes
Mismatched number of inports and outports among variant choices	Simulink disables inactive ports	Simulink disables inactive ports	Requires manual resolution of disconnected signal lines
Option to specify default variant	Yes	Yes	No
Supports control ports	No	No	Yes
Can be saved as standalone file	No	No	Yes
Supports physical modeling connection ports	No	Partially	No
Comment choice	Yes	No	No

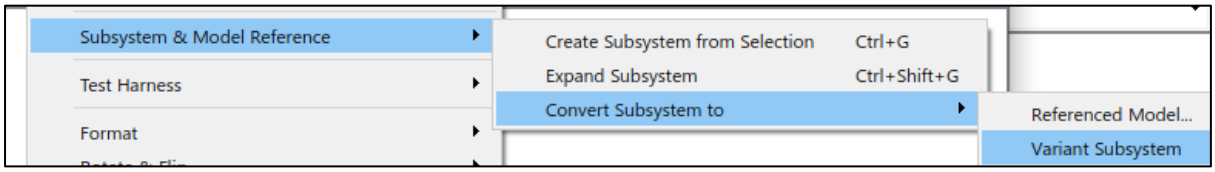
Chaque variante d'un modèle est associée à une expression conditionnelle qu'on appelle **variant control**. On peut évidemment spécifier une variante par défaut.

5.5.2 EXEMPLE SIMPLE

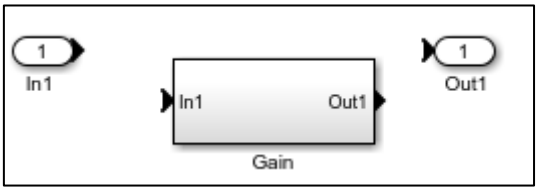
On part d'un modèle avec un subsystem contenant un simple gain :



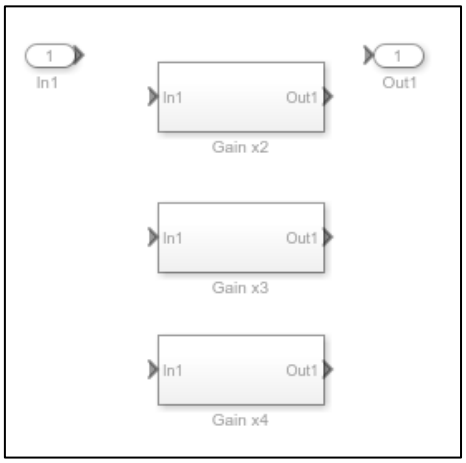
On convertit ce subsystem en "Variant …"



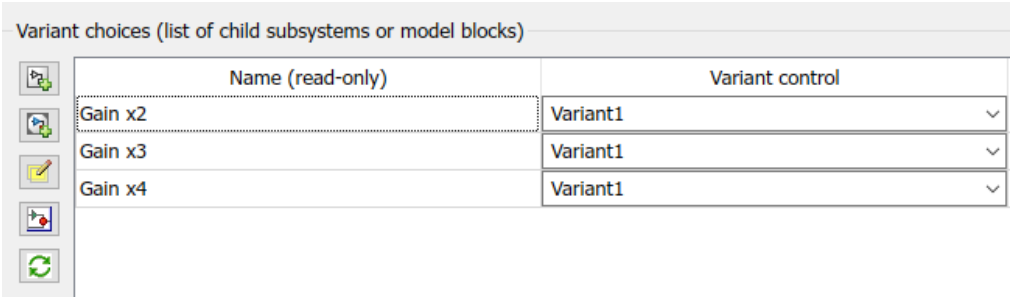
On obtient ceci, avec des ports déconnectés (c'est normal)



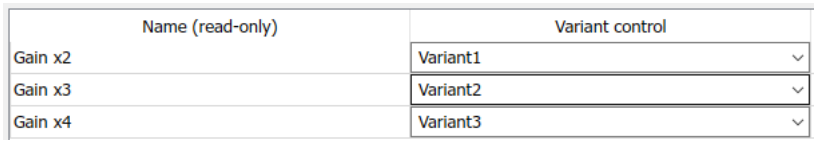
On duplique le subsystem, et on modifie les gains dans chacune des instances, de façon à obtenir ceci (les subsystems sont grisés car aucun n'est activé pour le moment - c'est toujours normal) :



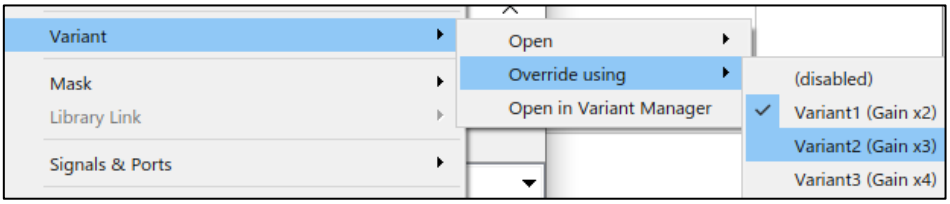
On remonte à l'étage au-dessus et clic-droit / Block Parameters. On voit que toutes les variantes portent pour l'instant le même nom :



Il faut modifier ceci pour obtenir par exemple :



De retour sur le sketch, clic-droit sur le subsystem puis "Override …"



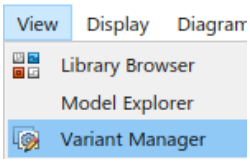
Si l'on choisit l'override avec la variante 2, on peut ensuite lancer la simu, on voit que la variante sélectionnée est affichée "dans" le subsystem, et … on obtient bien le résultat attendu :

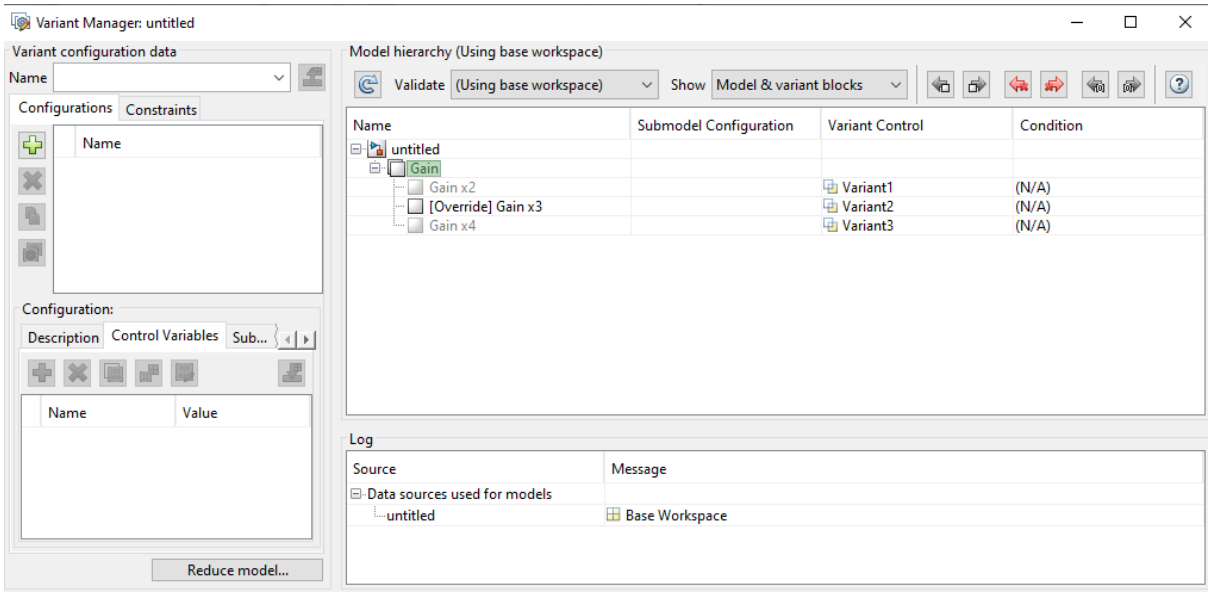


Cet exemple permet de choisir de manière interactive une variante que l'on souhaite utiliser pour un subsystem. Dès que le modèle se complexifie, il est souvent souhaitable de définir des configurations, et de les activer de manière programmatique. On passe pour cela par le variant manager.

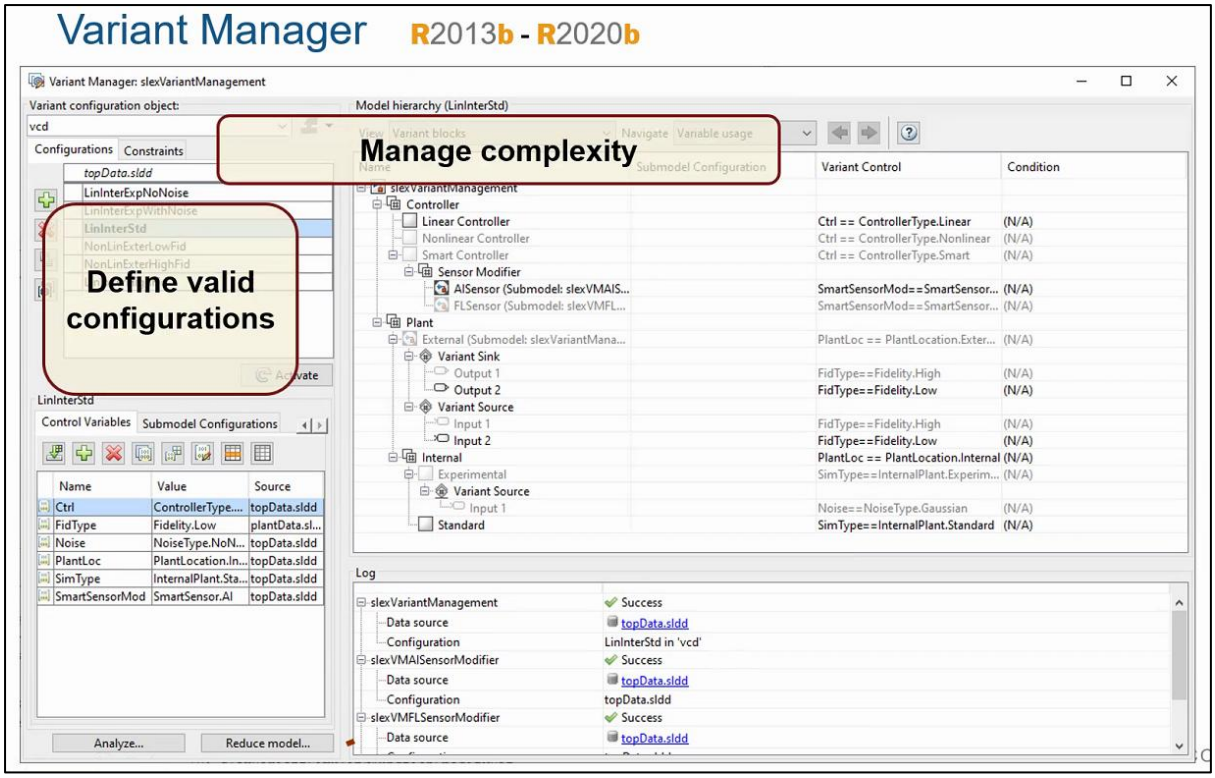
5.5.3 VARIANT MANAGER

En repartant du modèle précédent (un seul variant subsystem), on peut ouvrir le variant manager depuis le menu View :





Ci-dessous un extrait de doc Mathworks :



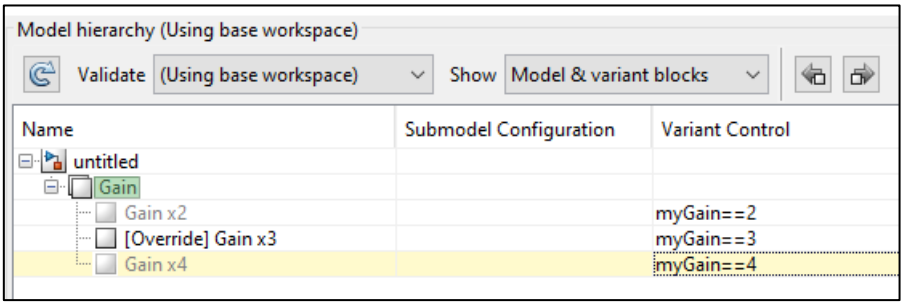
La variant manager permet de définir des configurations, qui correspondent à des combinaisons de variantes pour différents subsystems. Ces variantes peuvent être activées par des variables de contrôle, en utilisant notamment des expressions conditionnelles.

5.5.4 METHODE D'ACTIVATION DES VARIANTES

cf *Introduction to Variant Controls* dans la doc Simulink

Specification	Purpose	Example
Scalar variable	Rapid prototyping	`A == 1`
`Simulink.Variant` object	Reuse variant conditions	`LinearContoller = Simulink.Variant('FUEL==2 && EMIS==1');`
`Simulink.Parameter` object	Generate preprocessor conditionals for code generation	`mode == 1`, where `mode` is a `Simulink.Parameter` object
Enumerated type	Improved code readability because condition values are represented as meaningful names instead of integers	`LEVEL == Level.Advanced`

On peut illustrer la 1e de ces solutions en repartant de notre exemple simplifié. On définit nos variant controls de la sorte :



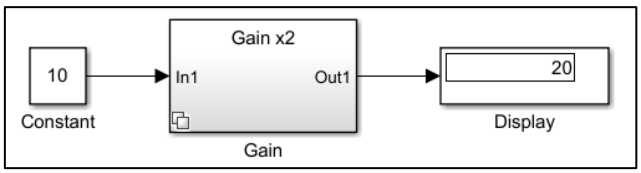
Si, dans le prompt Matlab, on définit

```
>> myGain=2
```

et qu'on relance la simu après avoir pris le soin de désactiver l'override précédemment défini :



on obtient bien :



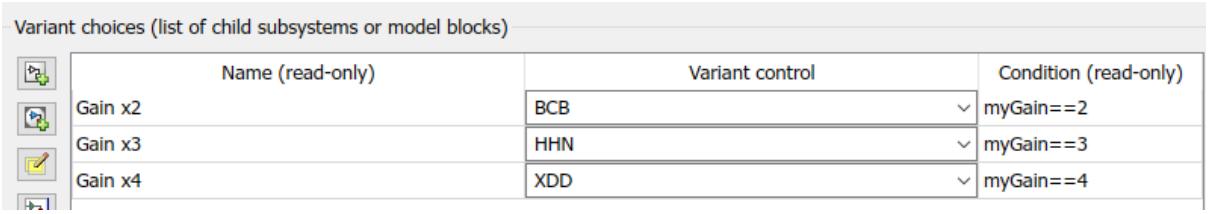
NB : The **Condition** field is N/A, because the variant control itself is the condition.

5.5.5 SIMULINK.VARIANT OBJECT

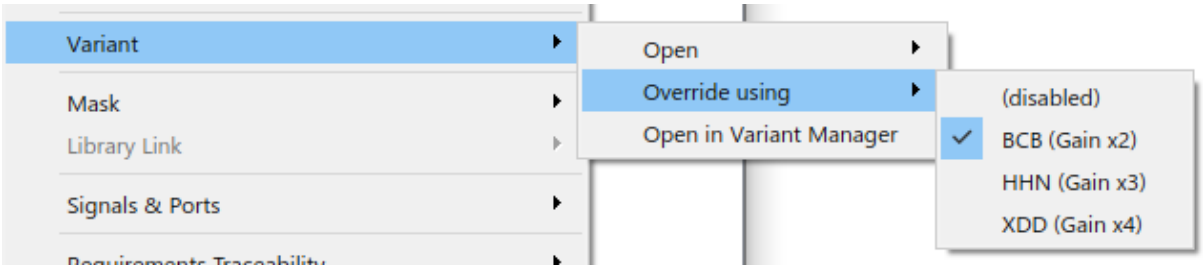
On peut définir des objects Simulink.Variant, ce qui permet de rendre les choses un peu plus explicites, en supposant par exemple que l'on utilise un gain par véhicule

```
BCB = Simulink.Variant('myGain==2')
HHN = Simulink.Variant('myGain==3')
XDD = Simulink.Variant('myGain==4')
```

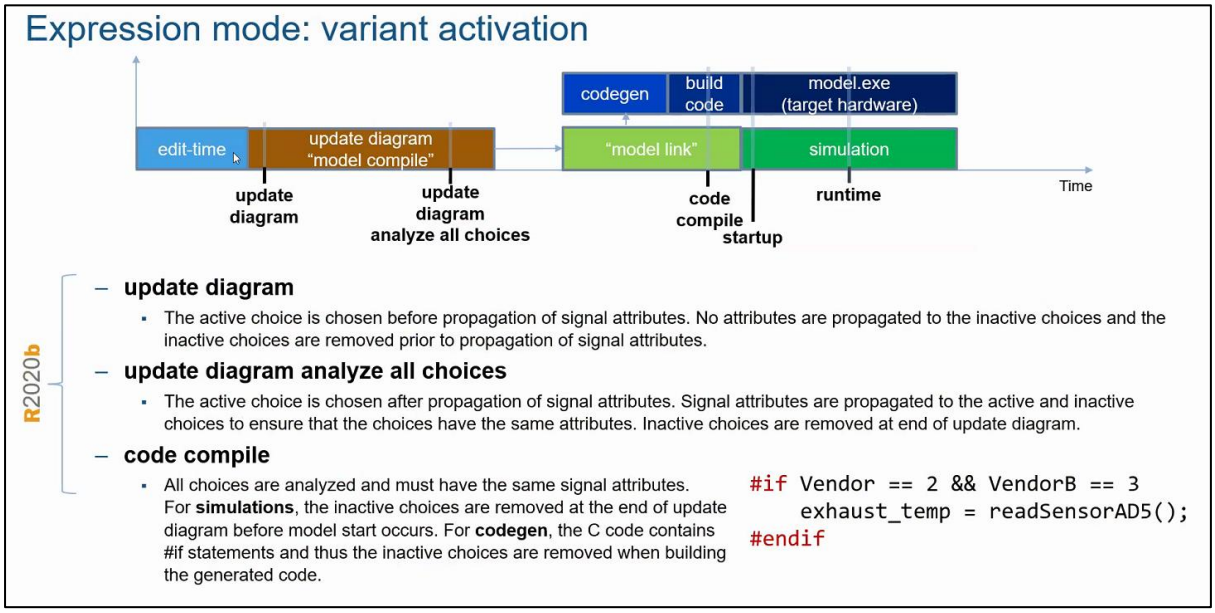
On comprend alors mieux la nuance entre Variant Control et Condition :

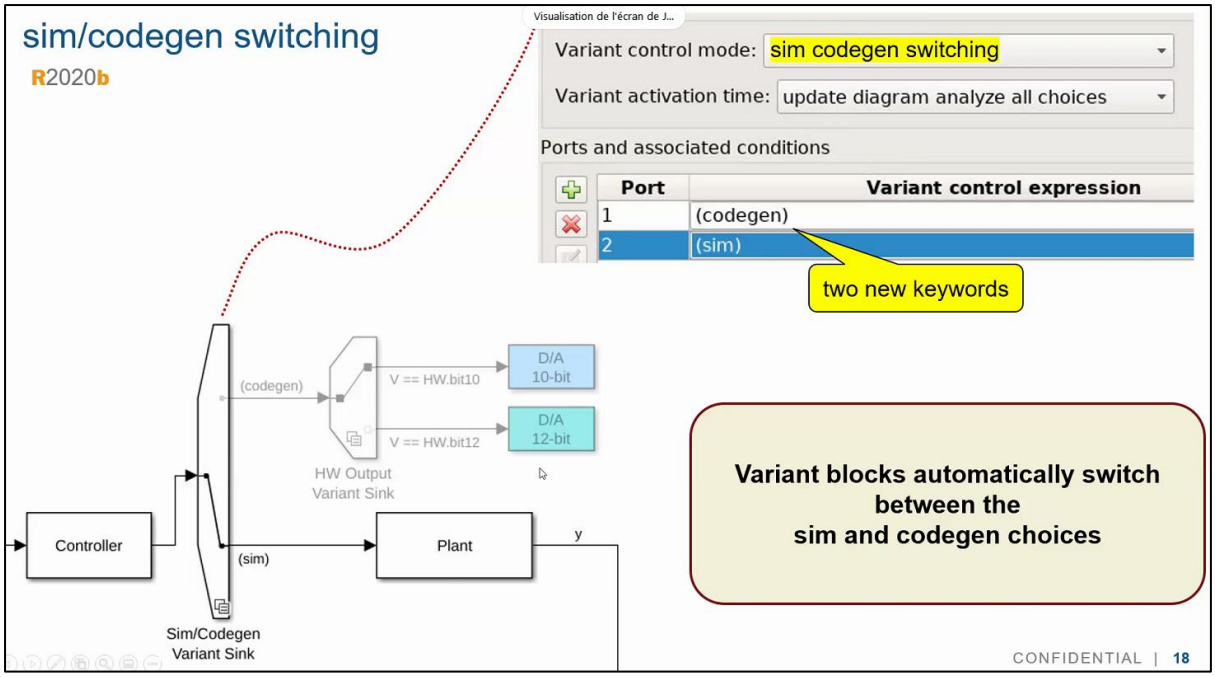


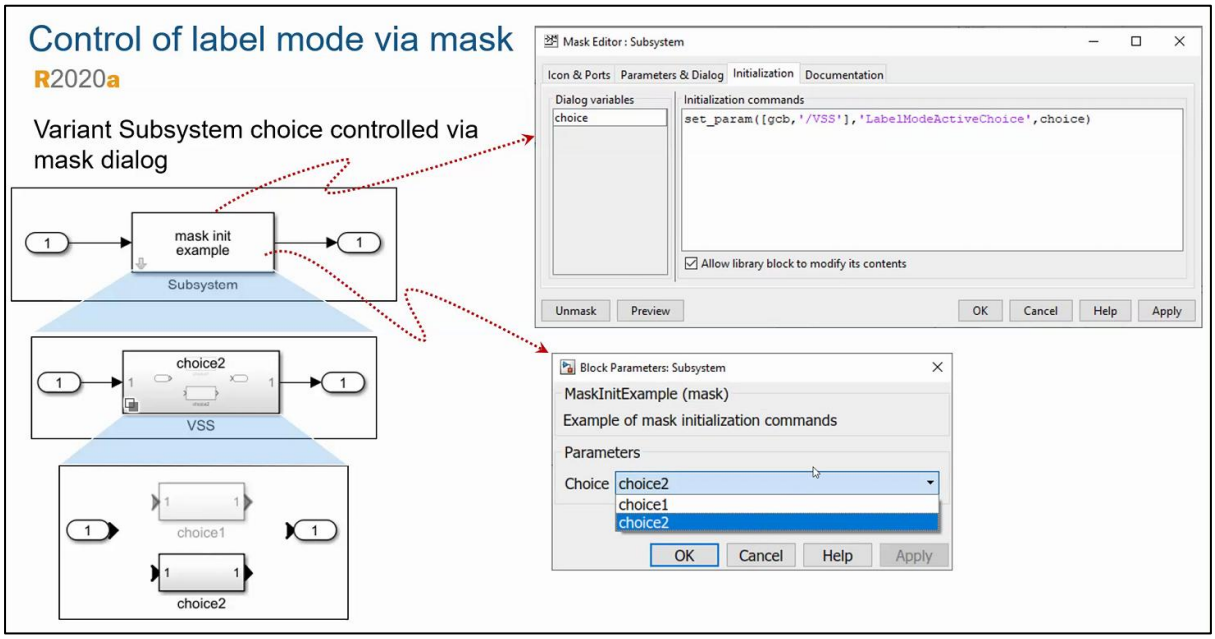
En interactif on a toujours :



5.5.6 INSTANT D'ACTIVATION DES VARIANTES



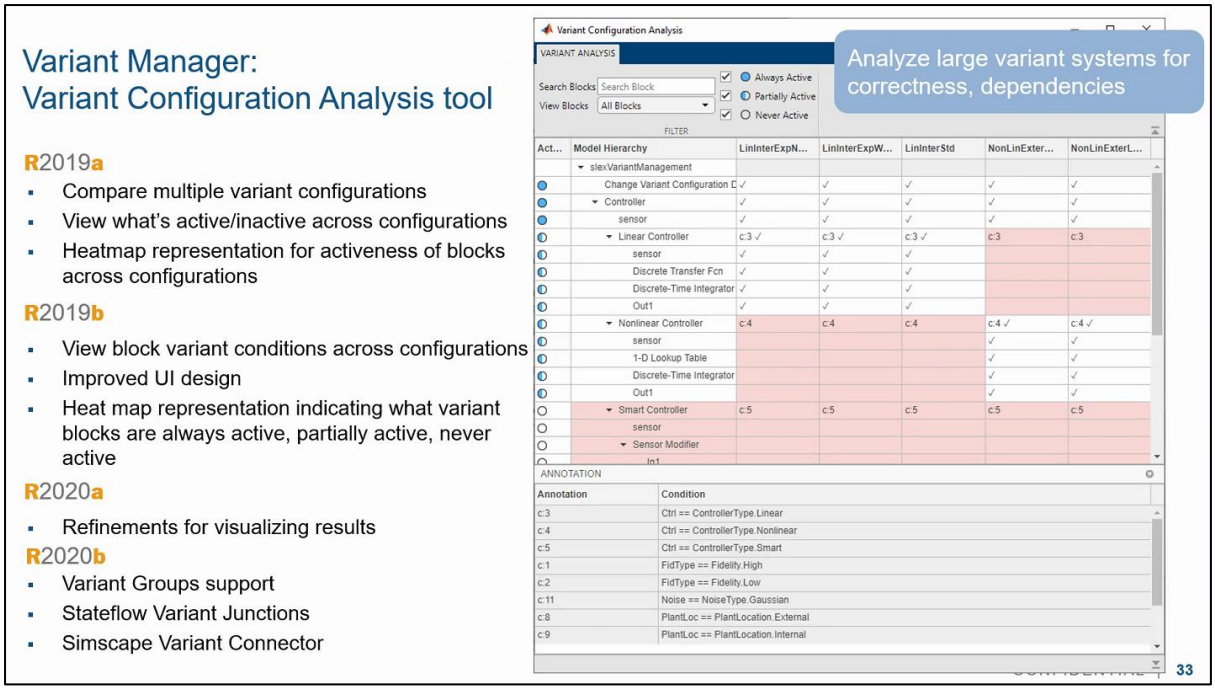




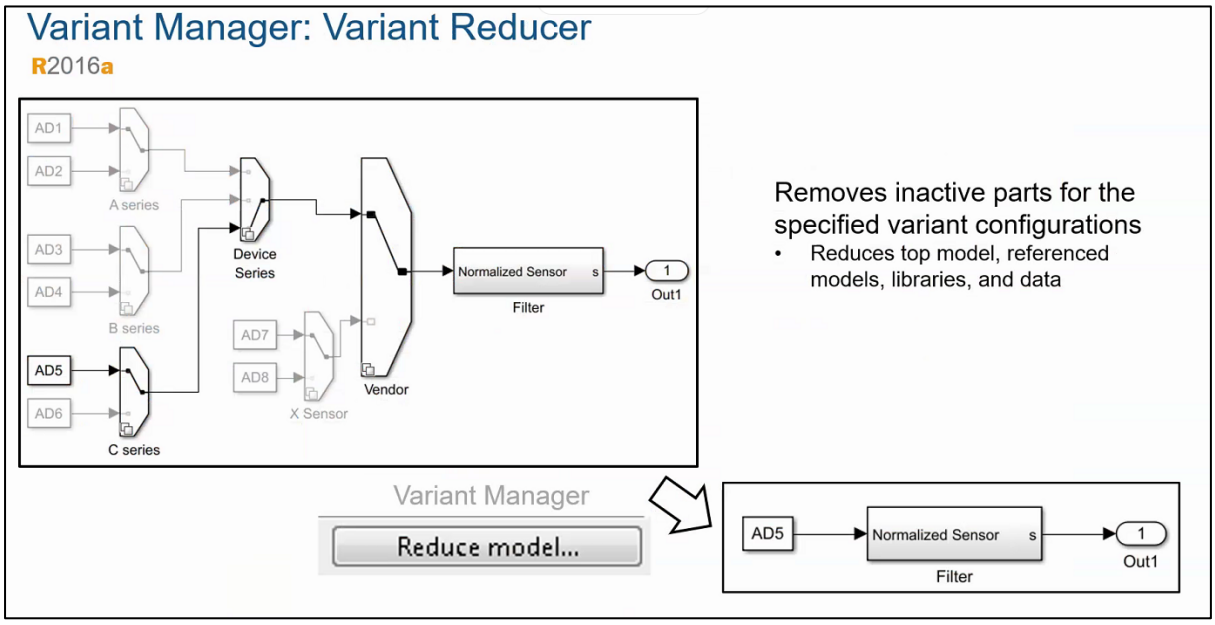
5.5.7 VARIANTES ET COMPILATION

https://fr.mathworks.com/help/ecoder/examples/variant-subsystems-controlled-by-c-preprocessor-conditionals.html

5.5.8 VARIANT ANALYSIS TOOL



5.5.9 VARIANT REDUCER



5.5.10 AVANTAGE DES VARIANTES SUR LES SUBSYSTEMS CONFIGURABLES

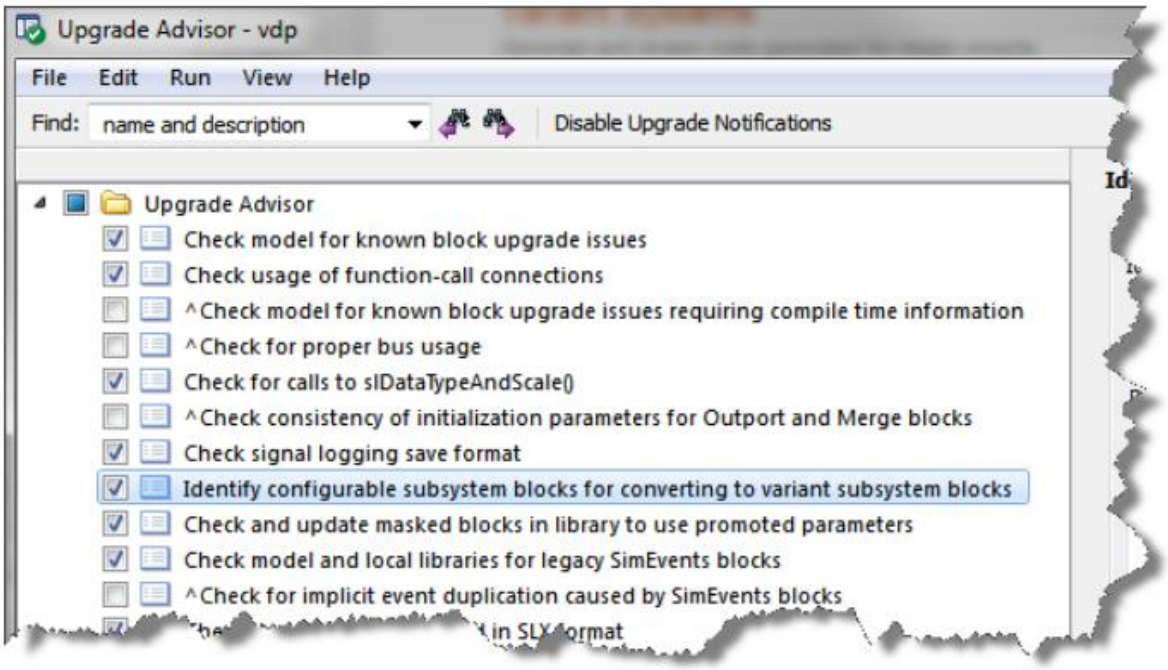
Les subsystems configurables étaient la solution "historique" de Simulink. Mais depuis plusieurs années il est fortement conseillé de passer aux variant subsystems :

https://blogs.mathworks.com/simulink/2013/03/18/time-to-convert-to-variant-subsystems/

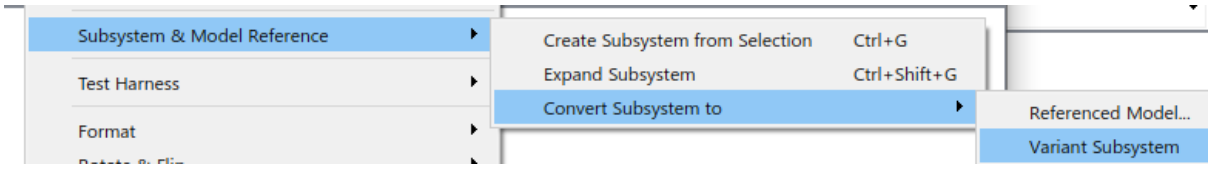
Simulink Variants provide increased functionality over Configurable Subsystems:

- They exist in two flavors: Model Variants and Subsystem Variants.
- They can be controlled programmatically via variables in the MATLAB workspace.
- They are in your model instead of a Simulink Library which simplifies model development.

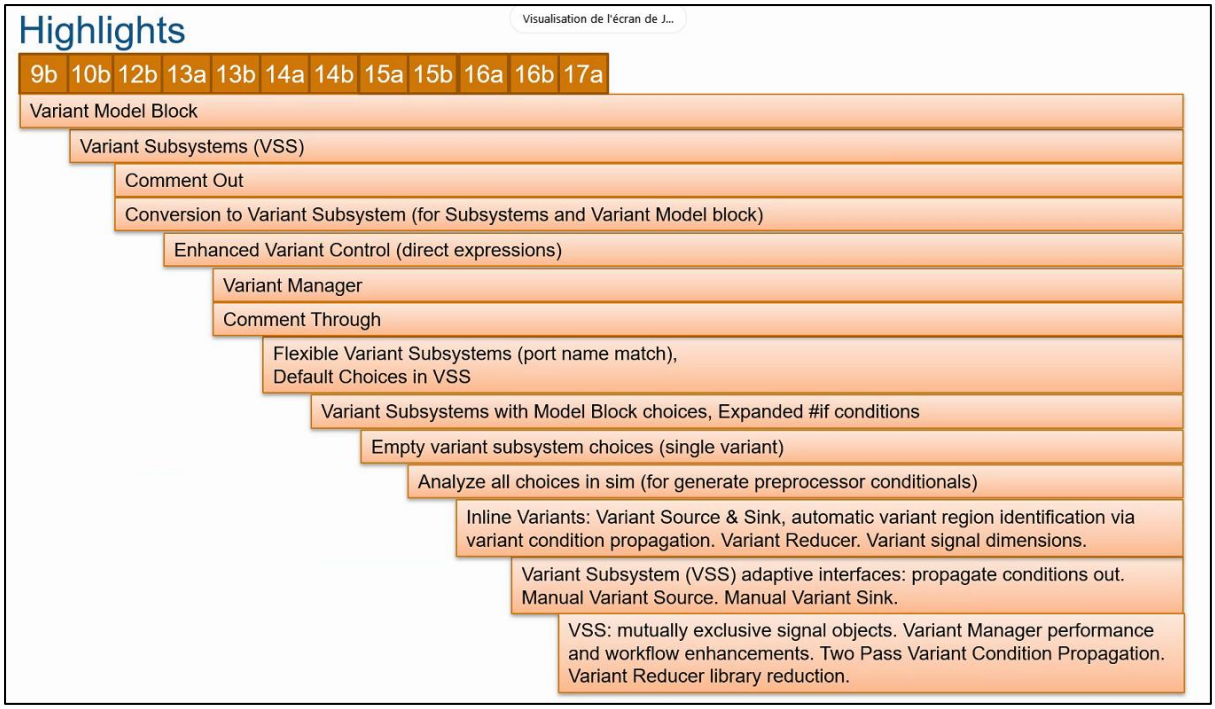
Upgrade advisor dès la 2012b : Identify configurable subsystem blocks for converting to variant subsystem blocks.



Right-click on the subsystem / Convert to Variant Subsystem



5.5.11 RELEASES


Visualisation de l'écran de J...
Highlights
9b
10b
12b
13a
13b
14a
14b
15a
15b
16a
16b
17a
Variant Model Block
Variant Subsystems (VSS)
Comment Out
Conversion to Variant Subsystem (for Subsystems and Variant Model block)
Enhanced Variant Control (direct expressions)
Variant Manager
Comment Through
Flexible Variant Subsystems (port name match),
Default Choices in VSS
Variant Subsystems with Model Block choices, Expanded #if conditions
Empty variant subsystem choices (single variant)
Analyze all choices in sim (for generate preprocessor conditionals)
Inline Variants: Variant Source & Sink, automatic variant region identification via variant condition propagation. Variant Reducer. Variant signal dimensions.
Variant Subsystem (VSS) adaptive interfaces: propagate conditions out. Manual Variant Source. Manual Variant Sink.
VSS: mutually exclusive signal objects. Variant Manager performance and workflow enhancements. Two Pass Variant Condition Propagation. Variant Reducer library reduction.

5.6 LIBRAIRIES

5.6.1 DISABLE/BREAK/RESOLVE

When you drag a library block into a model, you can't directly modify it inside that model. The idea is that the block should be modified in the library, and not in the model that references the library.

If you right-click the block in your model, however, you have the option to disable the link. When you do this, you are now able to modify the block in your model.

When you make changes to a disabled library block, it then becomes different from the original library block. If you select resolve, you can choose to push your local changes to the library or restore the block to how it originally was in the library.

Alternatively, you can break the link to a disabled block, which completely disconnects it from the library it originally came from. It is now a standalone block that has nothing to do with the library.

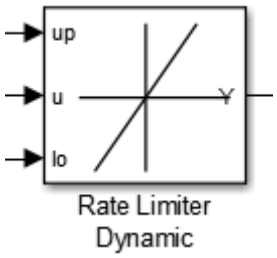
On peut afficher les liens dans le menu Display / Library Links

Link Badge	Status
Black links 	Active link
Gray separated links 	Inactive link
Black links with a red star icon 	Active and modified (parameterized link)
White links, black background 	Locked link

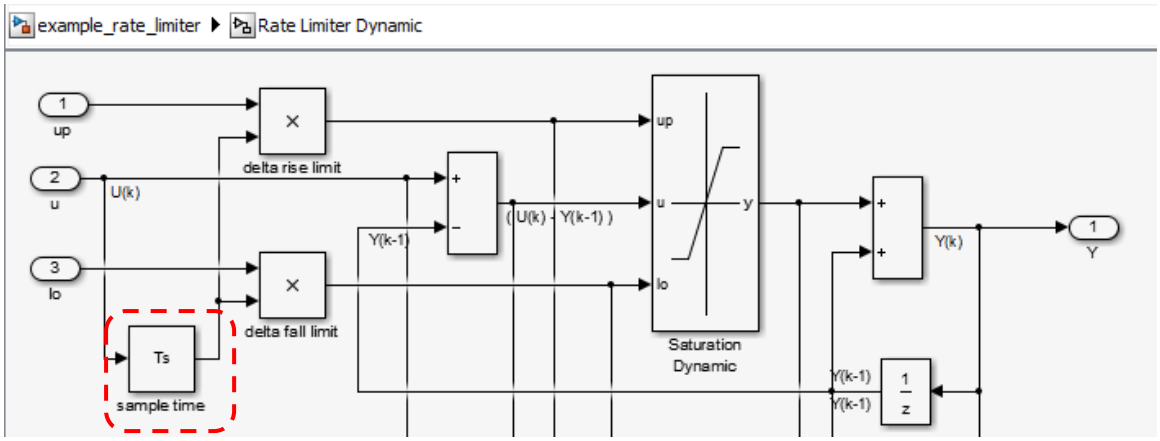
If you have a variant subsystem block inside a linked block, modifying the parameters on the variant subsystem creates instance data on the topmost linked block. If the link badge is visible, the presence of instance data is indicated by the red star on the link badge

5.6.2 RATE LIMITER DYNAMICS

Ce bloc venant de la lib Discontinuities permet de mettre des seuils (haut et bas) sur le taux de variation d'un signal.

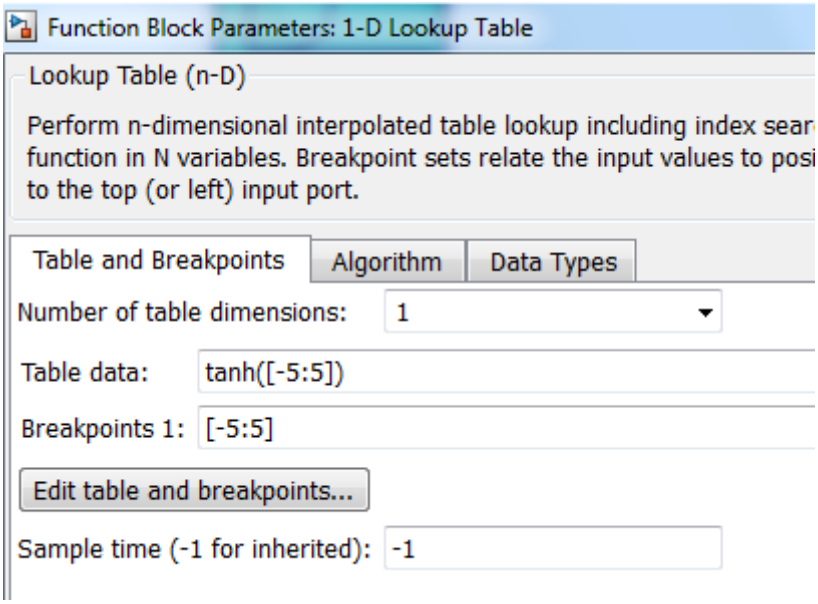


Il s'appuie sur le sample time Ts, et prend donc en entrée u des signaux discrets.



5.6.3 LOOKUP TABLE

Les breakpoints correspondent aux points de discrétisation de la table/cartographie :



6 GESTION DES DONNEES

6.1 INTRO

Il existe plusieurs solutions pour « stocker » des données associées à un modèle :

- les définir dans le base workspace (par un .m ou un .mat)
- les définir dans le model workspace
- utiliser un data dictionary

Capability	Base Workspace	Model Workspace	Data Dictionary
Data-model linkage	implicit	implicit	✓
Unified interface for defining data	✓	✓	✓
Model-data dependency	✓	✓	✓
Data entry comparison	✓	✓	✓
Data entry persistence		✓	✓
Options to remedy a missing variable	✓	✓	Additional options
Shared data	✓		✓
Data grouping			✓
Change tracking for data entries			✓
Change tracking for configuration sets			✓
Data entry merging and reconciliation			✓
Storage and partitioning of auxiliary data			✓
Requirements linking			✓

A voir aussi (en 4 parties, par Guy Rouleau)

https://blogs.mathworks.com/simulink/2022/05/13/unifying-matlab-and-simulink-a-user-story-part-4/

6.2 DATA DICTIONARY

En v2014 les data dictionary files (.sldd) sont introduits :

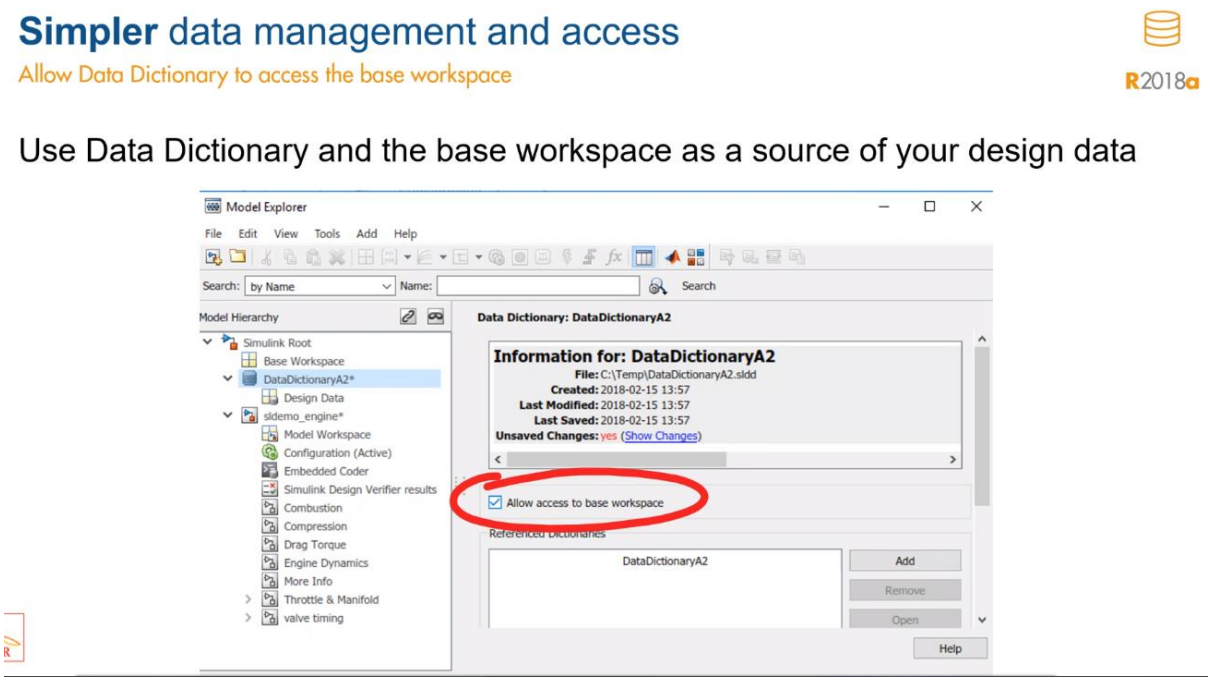
https://www.distek.com/blog/mbsd-data-dictionary/

Pemettent de faire du versioning

https://www.mathworks.com/help/simulink/ug/what-is-a-data-dictionary.html

https://www.mathworks.com/help/simulink/ug/store-data-in-dictionary-programmatically.html

Un .sldd peut également accéder au base workspace à condition de cocher « Allow access … »

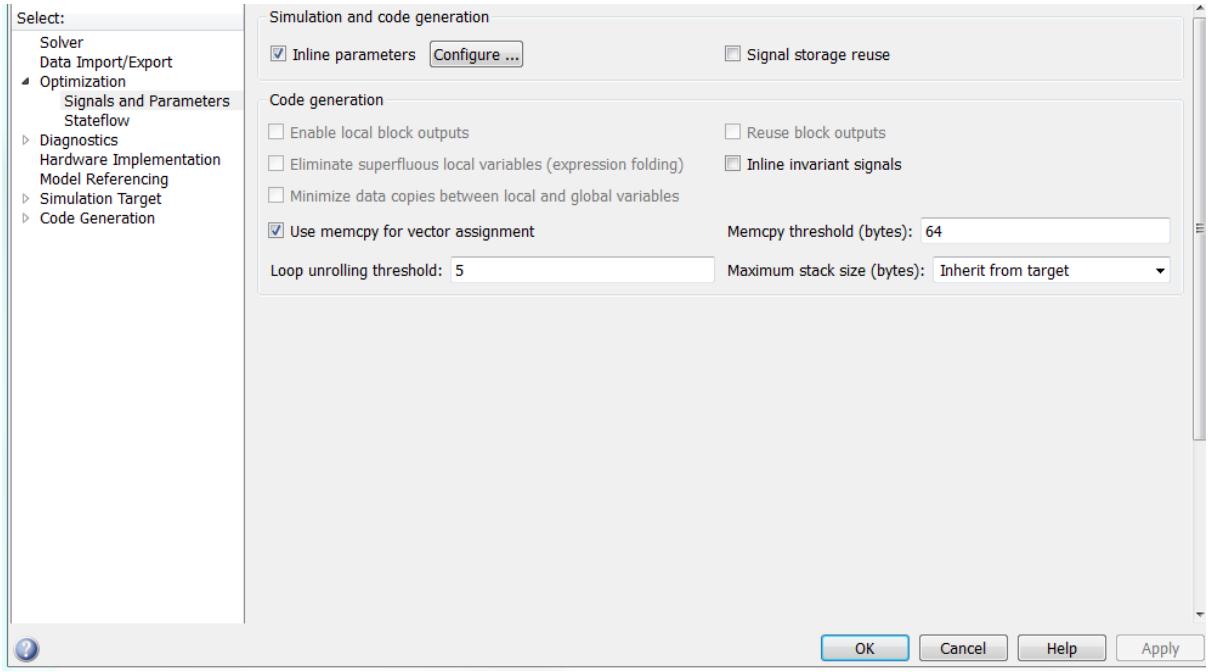


Si des données idetiques sont présentes dans les deux, un message d'erreur apparait.

6.3 TUNABLE PARAMETERS

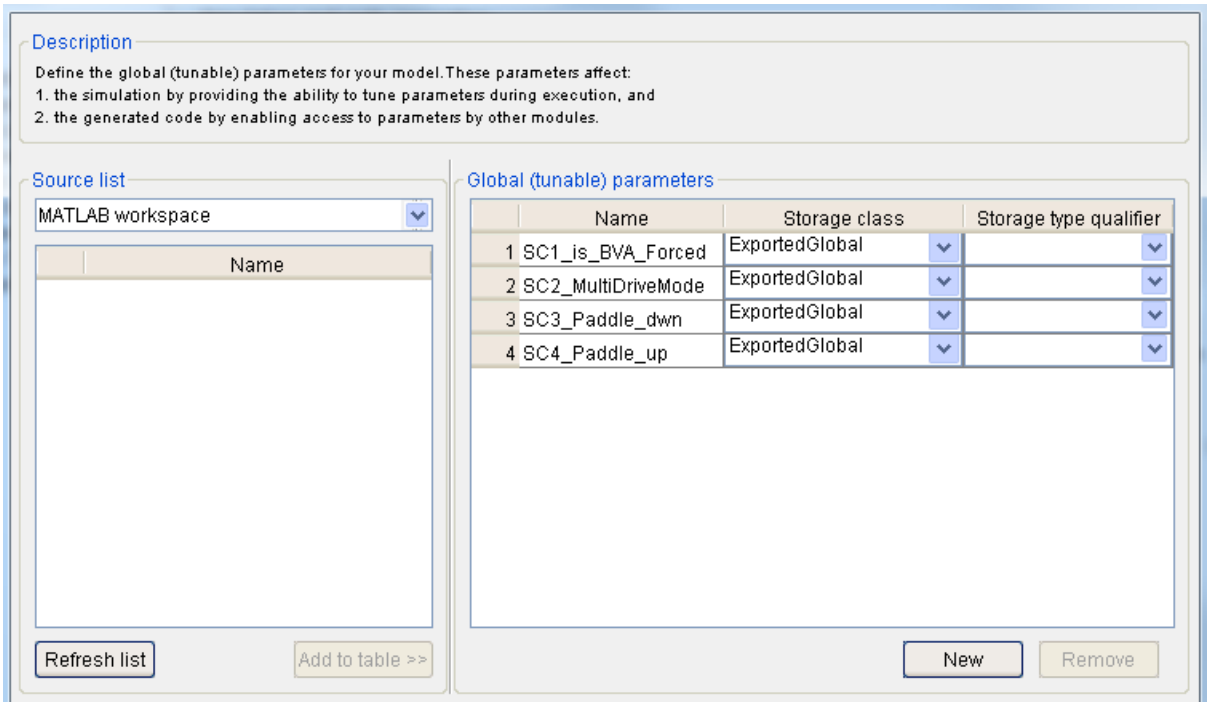
Pour en ajouter/modifier :

- Ouvrir le modèle
- Cliquer sur *Simulation/Configuration parameters…(ctrl+E)*
- Choisir *Optimization/Signal and Parameters* dans l'arbre de gauche



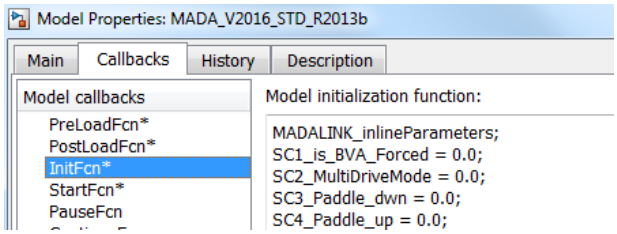
- Cocher la case *Inline parameters*
- Cliquer sur *Configure*

Nommer les paramètres :

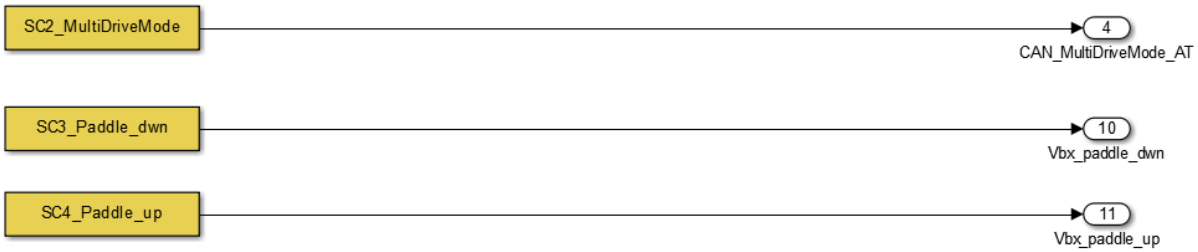


Penser à les définir dans l'InitFcn

File>Model Properties>Model Properties>Callbacks



Pour récupérer la variable dans le Simulink, créer une constante et indiquer le nom du paramètre dans sa valeur comme dans l'exemple ci-dessous

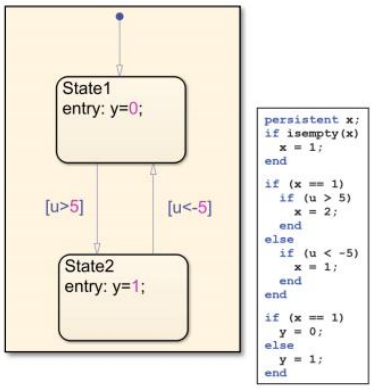


7 STATEFLOW

Introduction à Stateflow R2019b

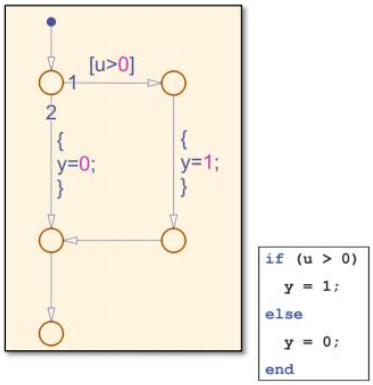
https://fr.mathworks.com/matlabcentral/fileexchange/73601-introduction-a-stateflow-r2019b?s_tid=prof_contriblnk

- Two types of Stateflow charts:



STATE DIAGRAMS

Chart that contains **State(s) with conditional logic** to **transition** from one state to another



FLOW CHARTS

Chart with only **transitions** and **conditional logic** between connective junction

8 INTERFACES

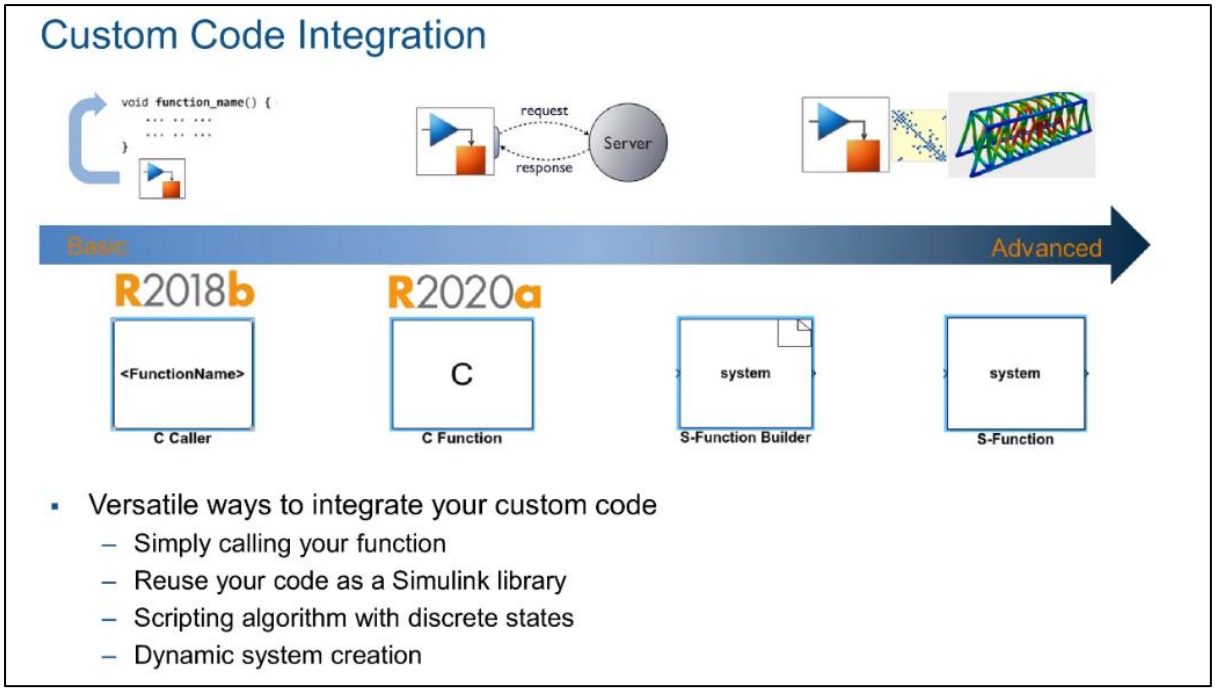
8.1 WITH MATLAB

On peut faire tourner un modèle Simulink depuis Matlab.

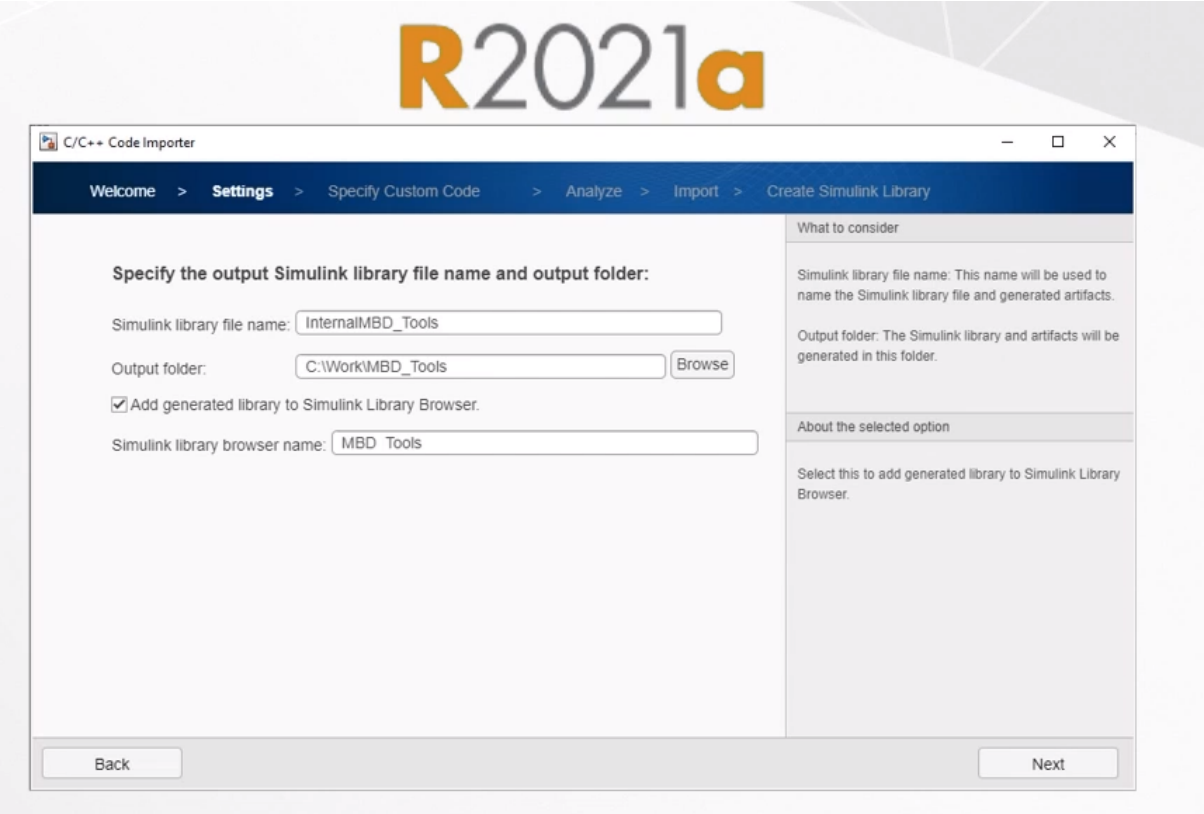
Commandes `sim` et `simset`

8.2 CODE INTEGRATION

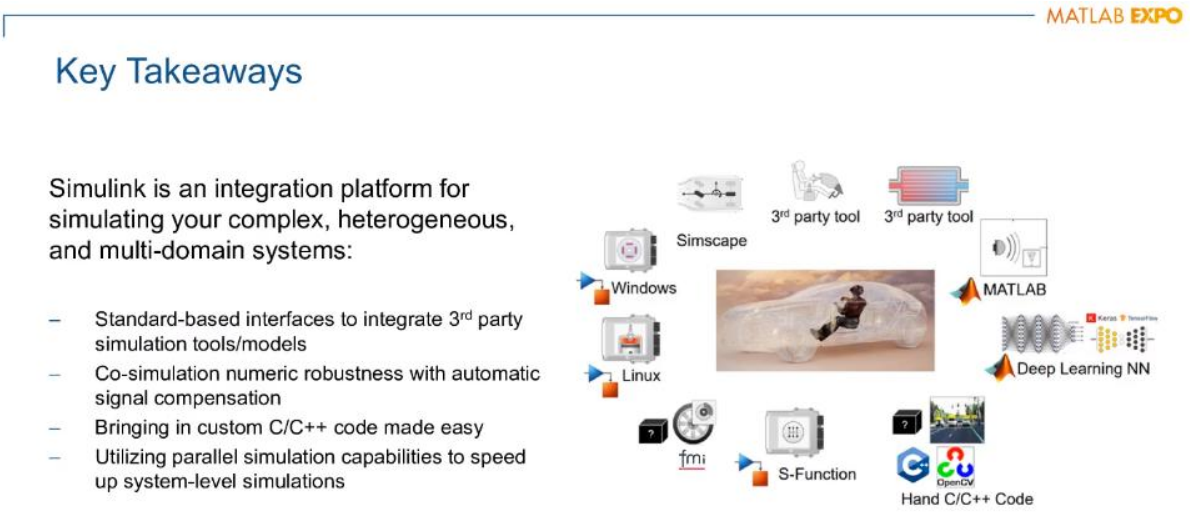
On peut intégrer du code C « nativement » depuis la 2020a



8.3 CODE IMPORTER (SINCE 2021A)



8.4 INTEGRATION OF THIRD-PARTY MODELS

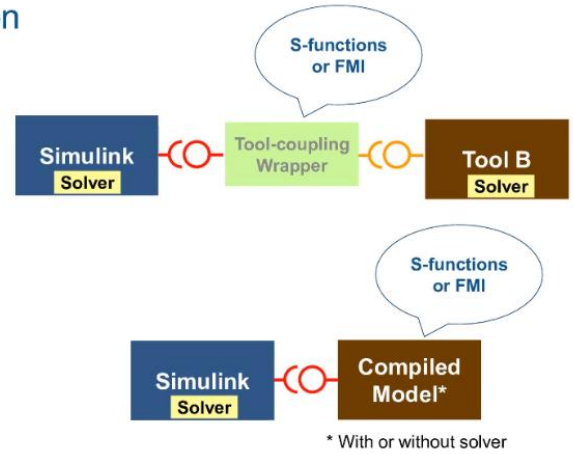


8.5 WHITE-BOX VS BLACK-BOX INTEGRATION

MATLAB EXPO

White-box and Black-box Integration

- White-box (tool-coupling)
  - Both Simulink and the external tool are running during simulation
- Black-box (compiled model)
  - Only Simulink is running during simulation
  - The 3rd party model is a component inside Simulink
- Standard-based interfaces used for both styles
  - S-functions
  - Functional Mockup Interface (FMI)



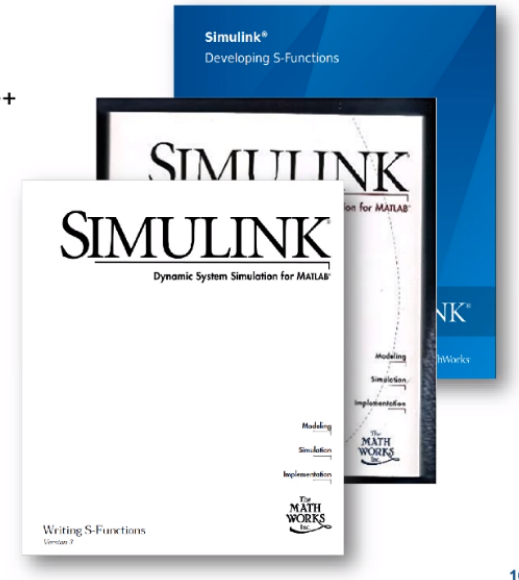
8.6 S-FUNCTIONS

Voir §4.6

MATLAB EXPO

S-functions Interface

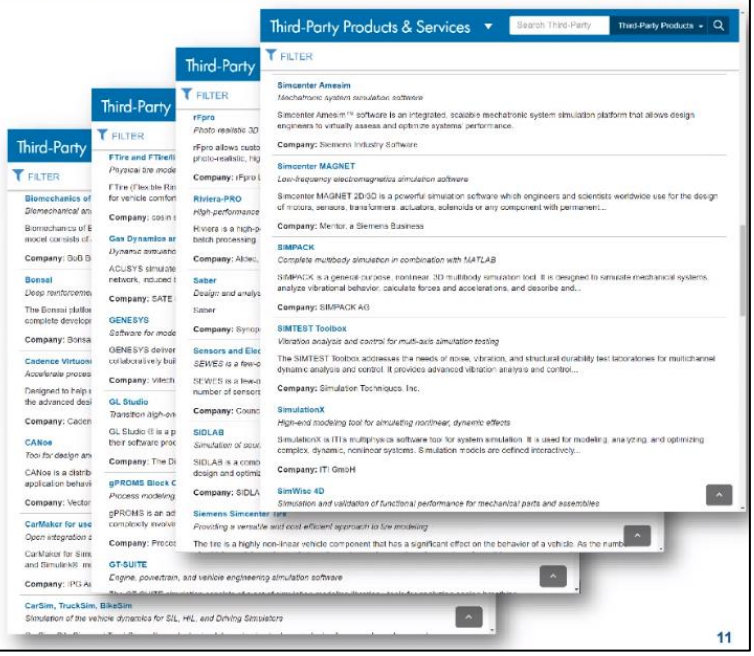
- Build custom dynamic systems in MATLAB, or C/C++
- Supports all Simulink semantics
- Well validated by industry for 20+ years
- The de facto standard to couple external tools with Simulink



10

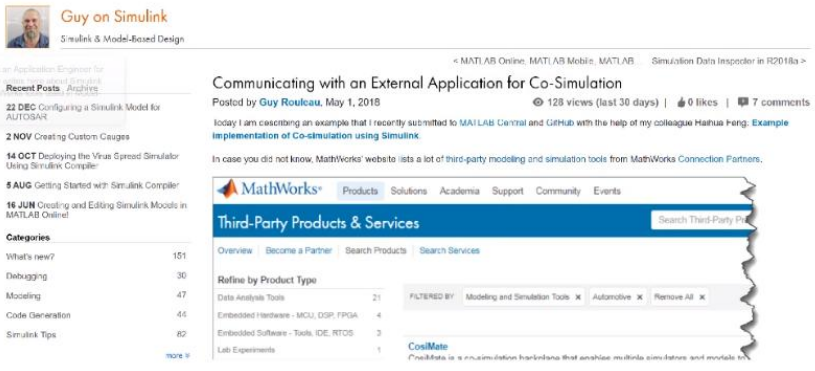
S-functions Interface

- Many of the MathWorks' Connection Partner simulation tools (150+) provide prebuilt co-simulation interfaces using S-functions
- S-functions based co-simulation interface is also available with some non-partner's tools

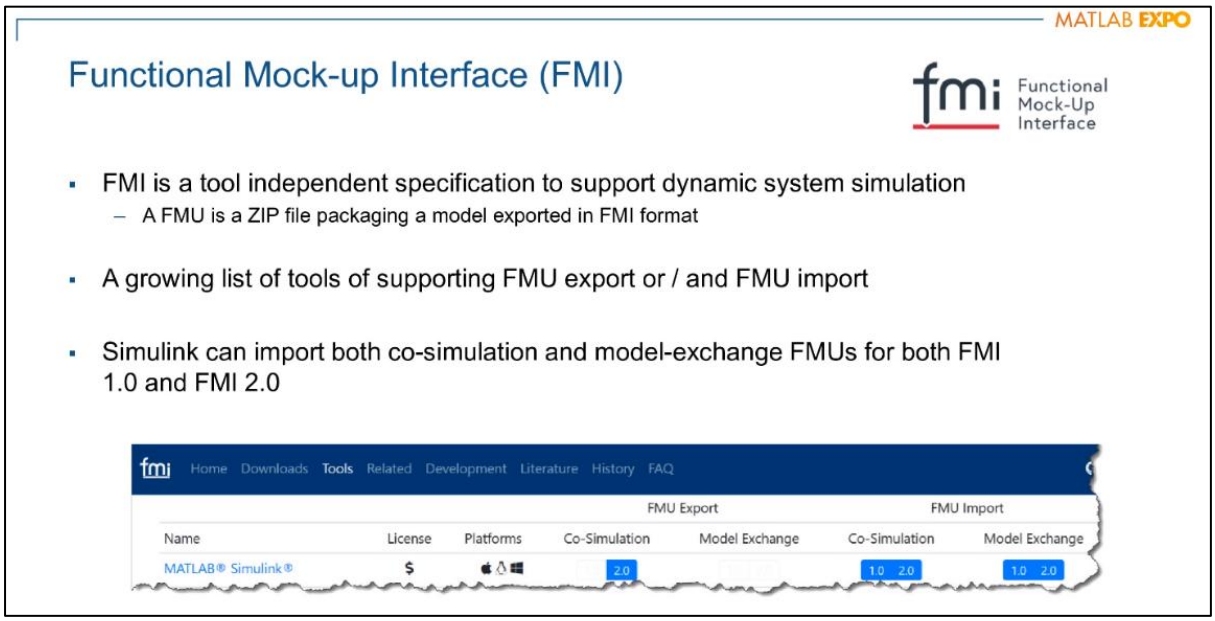


To learn more to use S-functions to communicate with an external application

- Example template to use S-functions as the tool-coupling interface
  - Available on "Guy on Simulink"



8.7 FMI/FMU



8.7.1 IMPORT

Le block FMU existe dans Simulink depuis la 2017b

https://www.mathworks.com/help/simulink/ref_extras/fmu.html

En model exchange :

https://www.mathworks.com/help/simulink/slref/importing-a-model-exchange-fmu-into-simulink.html

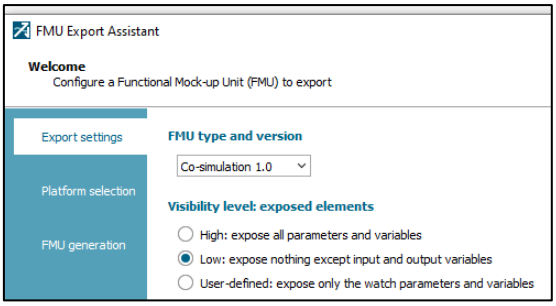
En cosim :

https://www.mathworks.com/help/simulink/slref/importing-a-co-simulation-fmu-into-simulink.html

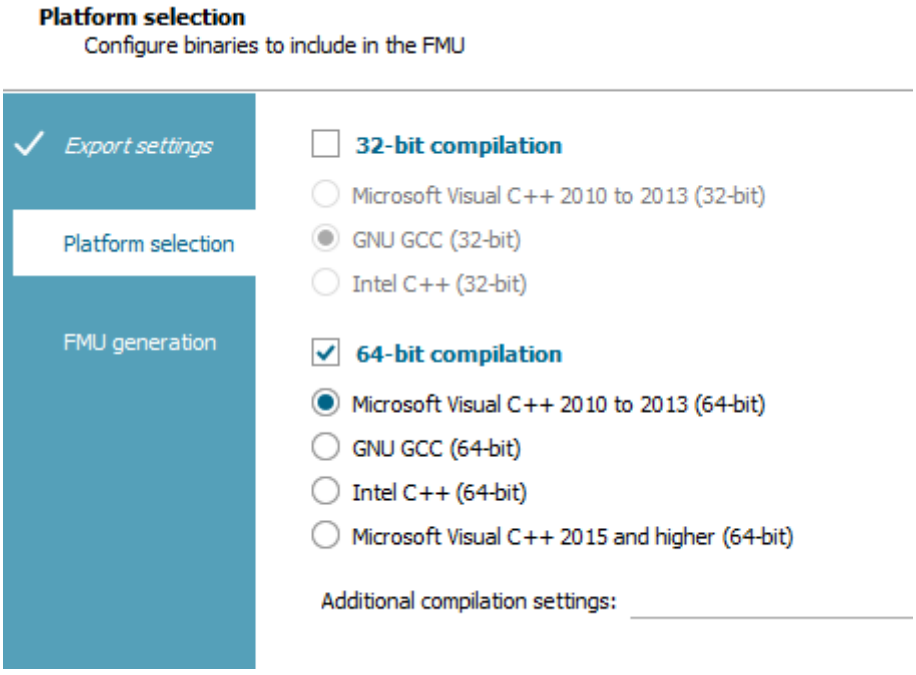
8.7.2 IMPORT A .FMU FROM AMESIM

Amesim (2021.1) steps

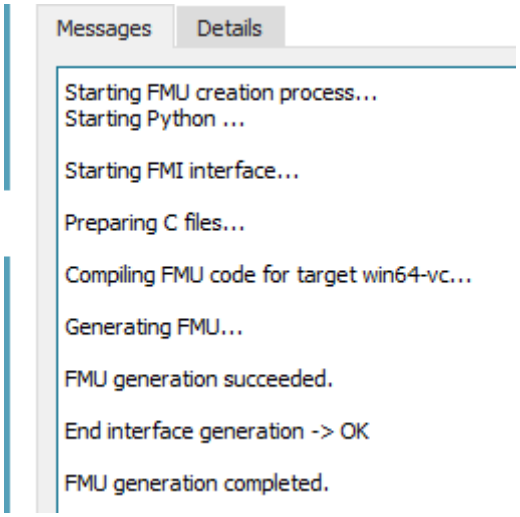
Export an .fmu with Cosimulation type (my example worked both with 1.0 and 2.0)



Select adequate compiler

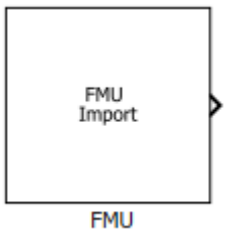


Then Generate :

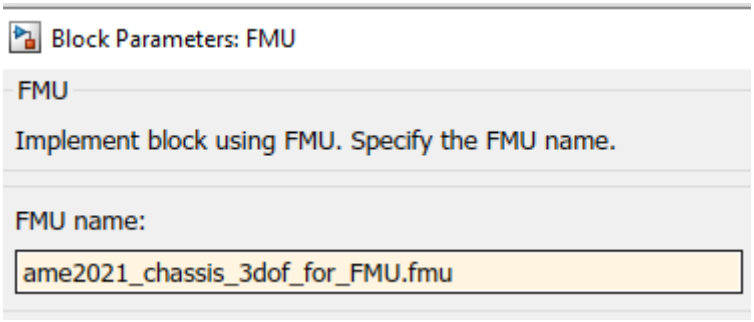


Simulink steps (tested with 2018b)

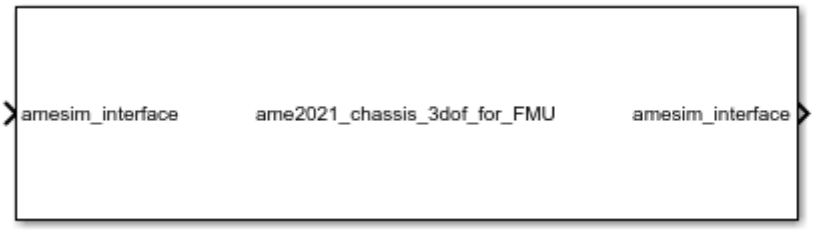
Insert an FMU import block from the library (did not exist in 2016b, anyway)



Double-click on the block and select the .fmu generated from Amesim



You should get the following result :



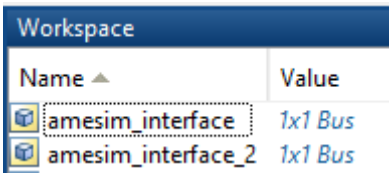
This FMU component expects non-virtual buses as inputs and outputs (which does not respect the FMI standard …)

Run the Matlab command

```
fmudialog.createBusType('myModel/FMU')
```

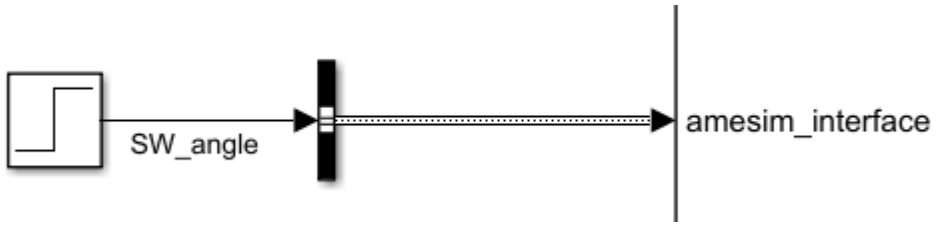
where *myModel* is the name of the .slx

This will create two Bus objects in the Base Workspace for the input and output :

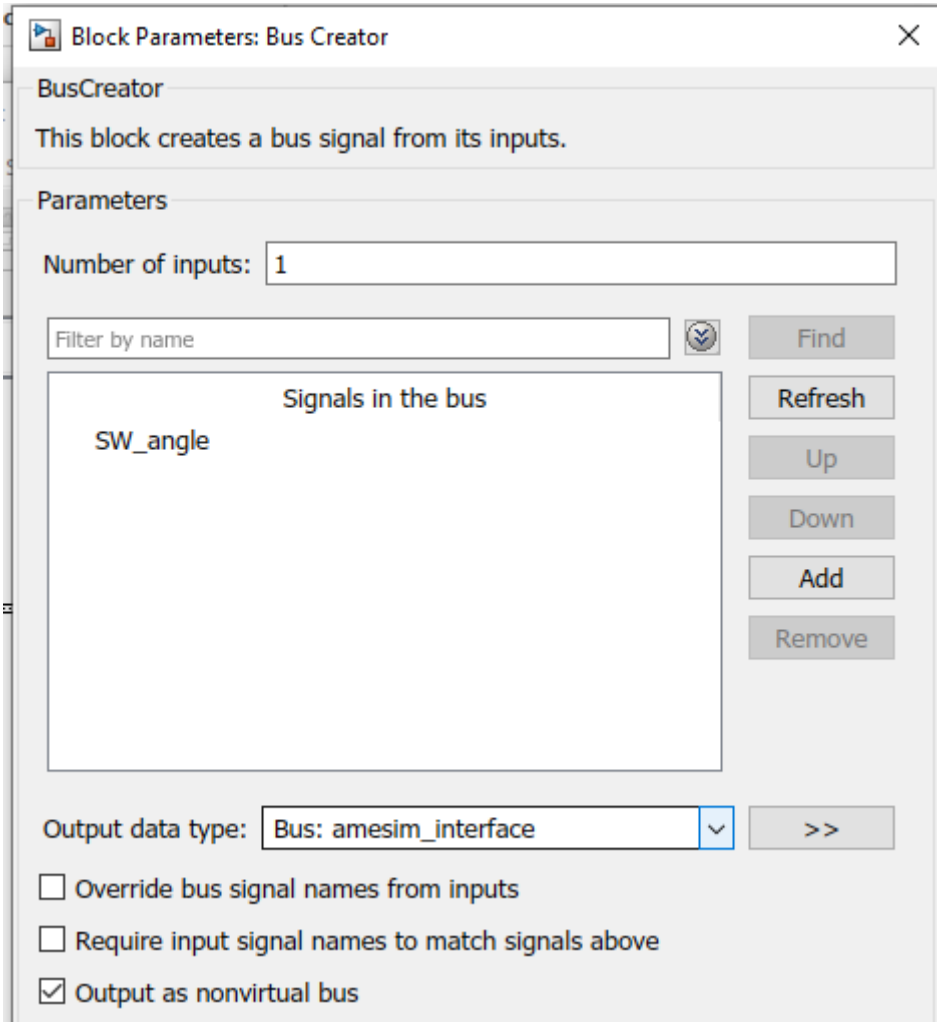


With my simple model there's only one input, but it must still be converted to a bus.

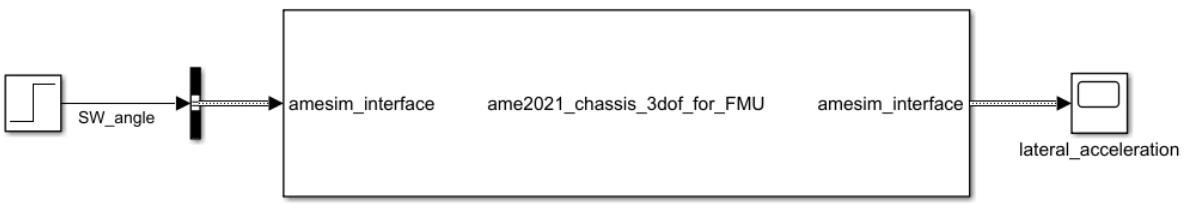
Insert a Bus creator after the input signal :



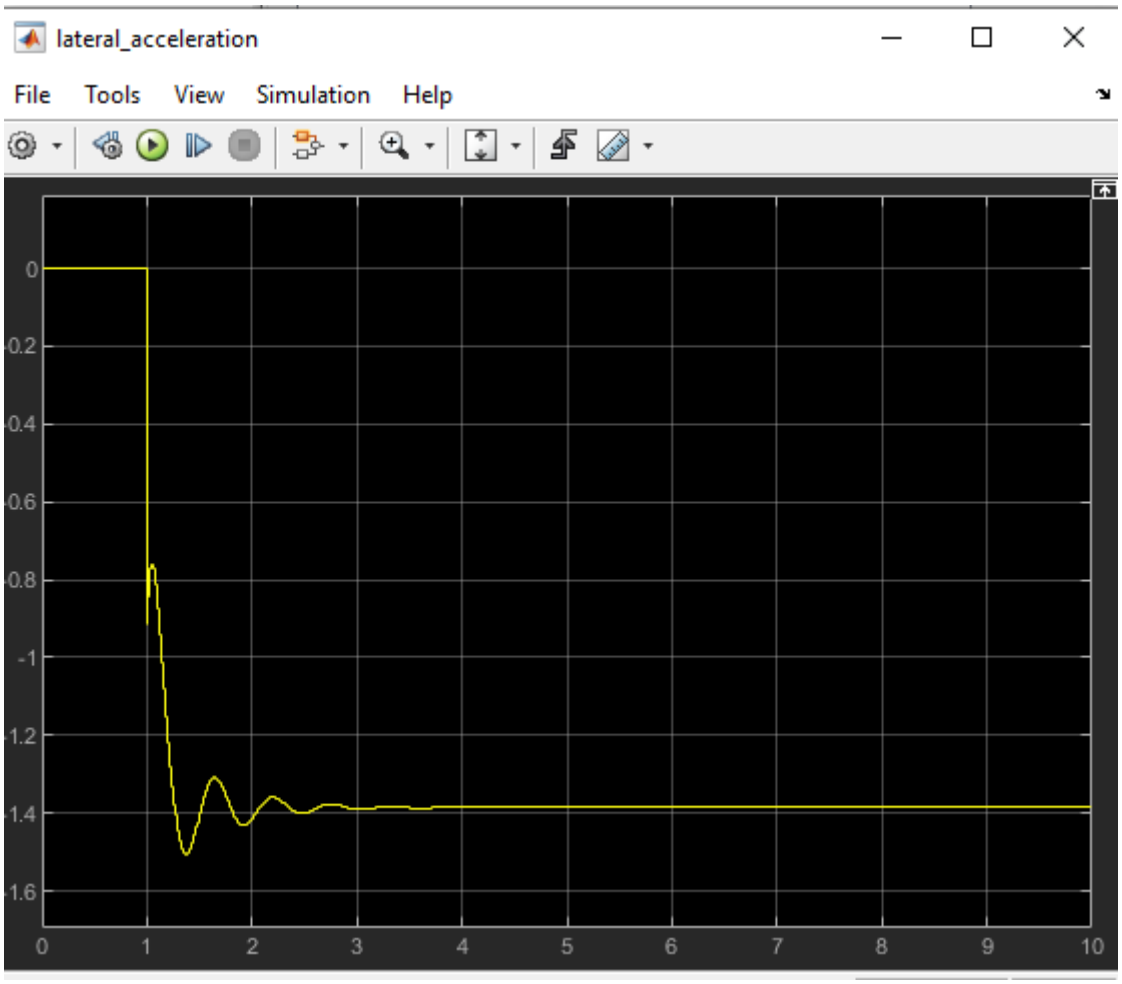
and set it as follows :



Select the name of the input bus object previously created as the output data type, and don't forget to tick "Output as nonvirtual bus" (otherwise you'll get an error message)



It works !



8.7.3 CAUTION : WHEN TRYING TO RUN AN ALREADY CREATED MODEL

Matlab 2018b + Amesim 2021.2

If you reopen a model (that worked) calling an FMU, you need to double-click on the FMU block to "reload" it, otherwise it won't run.

The error message says "FMU import block is not loaded. Specify a valid .fmu file, and double-click block icon or update diagram to load FMU.", but the update (Ctrl+D) doesn't work.

So this is probably an issue when running in command line …

8.7.4 EXPORT

Pour les versions jusqu'à la 2018a, il fallait recourir à des outils externes :

For R2018a and prior, there are third-party products which may provide this functionality. Please refer to the following links for more information:

- https://www.mathworks.com/products/connections/product_detail/modelon-fmi-toolbox.html
- https://www.mathworks.com/products/connections/product_detail/iti-fmi-target.html
- ><https://www.mathworks.com/products/connections/product_detail/claytex-fmi-blockset.html

Pour la 2018b, un add-on du nom de « Tool-Coupling Co-Simulation FMU Export for Simulink » était nécessaire :

https://fr.mathworks.com/matlabcentral/fileexchange/68814-tool-coupling-co-simulation-fmu-export-for-simulink

> To integrate Simulink® components into third-party software, export a Simulink model as a tool-coupling functional mockup unit (FMU). When a third-party tool runs the FMU, it checks out required licenses and starts a local installation of Simulink to start the model. This function requires the Tool-Coupling Co-Simulation FMU Export for Simulink support package. Use the Add-On Explorer to download and install the support package

Depuis la 2019a, il s'agit d'une fonctionnalité native de Simulink :

https://fr.mathworks.com/help/releases/R2019a/simulink/ug/export-model-as-tool-coupling-fmu.html

A noter que le FMU généré doit disposer sur la machine hote d'une installation locale de Simulink pour pouvoir fonctionner.

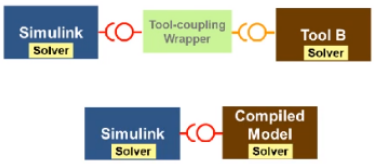
A noter, dans les autres outils disponibles :

- FMIKit (Dassault Systèmes)
- FMI Toolbox (Modelon)
- Simulix : A third-party Simulink tool to generate FMUs from models using the C-API (source)
- FMI Target (ESI), linked with Simulation X
- FMI Blockset (Claytex)

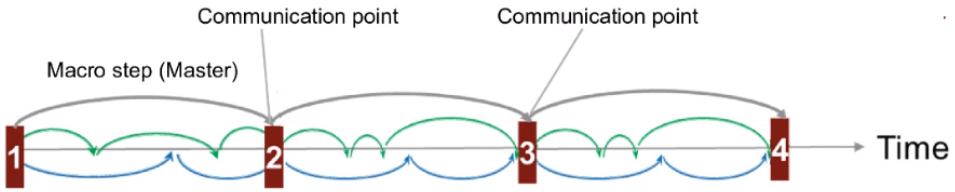
8.8 CO-SIMULATION

Co-simulation

- A frequently used method to bring models of external tools into Simulink
  - Each co-simulation component has its own solver
  - Can be implemented either white-box or black-box style

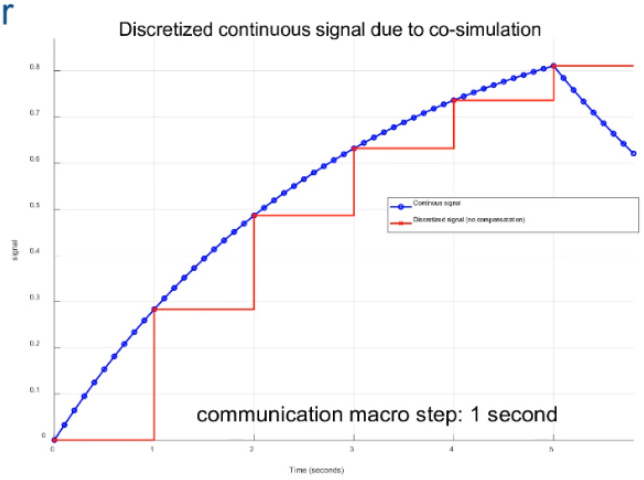


- Co-simulation components can run in parallel freely between communication macro steps



Co-simulation numeric behavior

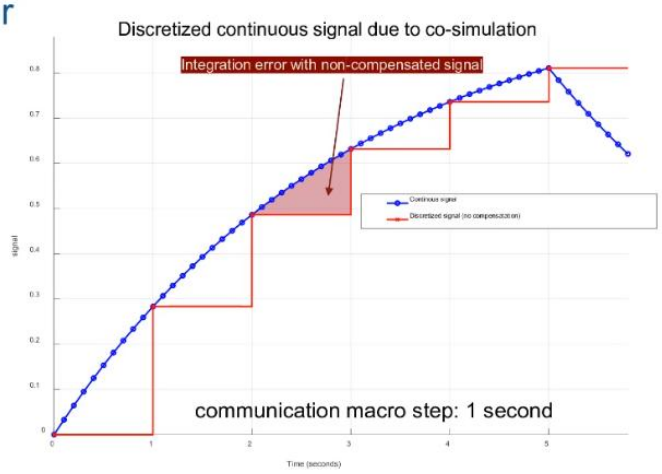
- Model integration is more than coupling the signals
- Potential error when coupling continuous signals
  - Discretized and delayed crossing co-simulation boundary
  - Non-compensated signals could lead to accuracy loss or even system instability



The un-compensated signal (red line) deviates from the ideal, continuous signal (blue line) due to discretization

Co-simulation numeric behavior

- Model integration is more than coupling the signals
- Potential error when coupling continuous signals
  - Discretized and delayed crossing co-simulation boundary
  - Non-compensated signals could lead to accuracy loss or even system instability

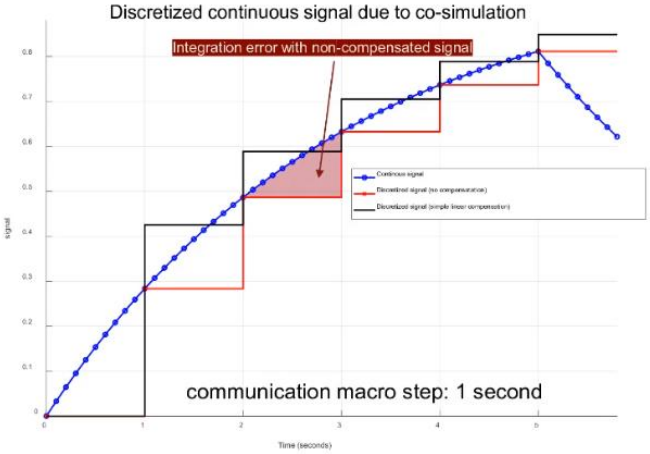


The un-compensated signal (red line) deviates from the ideal, continuous signal (blue line) due to discretization

Robust co-simulation

- Automatic and manual mechanism to compensate the discretized continuous signals
  - Choice of linear or high order extrapolation compensation methods
- More robust co-simulation results compared to un-compensated co-simulation



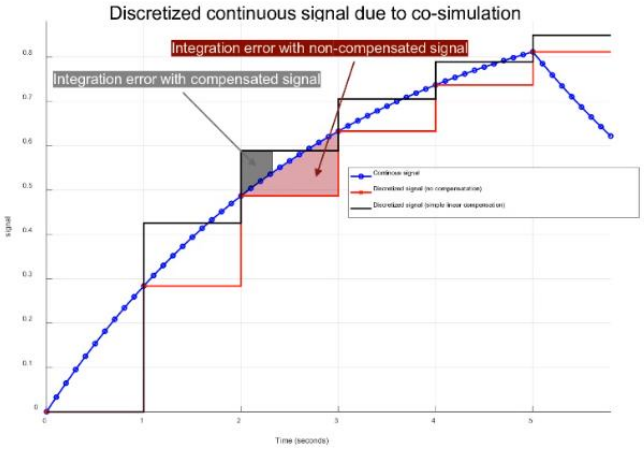


The compensated signal (black line) with simple linear extrapolation is closer to the ideal, continuous signal (blue line) than the uncompensated, discretized signal (red line)

Robust co-simulation

- Automatic and manual mechanism to compensate the discretized continuous signals
  - Choice of linear or high order extrapolation compensation methods
- More robust co-simulation results compared to un-compensated co-simulation





The compensated signal (black line) with simple linear extrapolation is closer to the ideal, continuous signal (blue line) than the uncompensated, discretized signal (red line)

9 SOLVEURS

- **Solveurs à pas fixe**
  - Le pas de calcul est fixé par l'utilisateur.
  - Les temps de calcul sont maîtrisés et dépendent du pas choisi.
  - Adapté aux problèmes non raides (pas de grandes variations d'états).
  - Les sorties ne sont calculées que pour le pas choisi.
- **Solveurs à pas variable**
  - Le pas est modifié par SIMULINK en cours de simulation ➔ les sorties en dépendent
  - Permet la détection de « zéro-crossing » ➔ augmente le temps de simulation.
  - L'utilisateur spécifie la tolérance sur la précision des résultats.
  - Permet de résoudre tout type de problème complexe.
  - nécessite une licence lorsque le modèle est compilé.

9.1 PROBLEMES NUMERIQUES POTENTIELS

9.1.1 ZERO-CROSSING

http://fr.mathworks.com/help/simulink/ug/zero-crossing-detection.html

A variable-step solver dynamically adjusts the time step size, causing it to increase when a variable is changing slowly and to decrease when the variable changes rapidly. This behavior causes the solver to take many small steps in the vicinity of a discontinuity because the variable is rapidly changing in this region. This improves accuracy but can lead to excessive simulation times.

The Simulink® software uses a technique known as zero-crossing detection to accurately locate a discontinuity without resorting to excessively small time steps. Usually this technique improves simulation run time, but it can cause some simulations to halt before the intended completion time.

9.1.2 BOUCLES ALGEBRIQUES

http://fr.mathworks.com/help/simulink/ug/algebraic-loops.html

9.2 LANCEMENT EN PARALLELE (PARSIM)

```
Cf_sweep = 2500*(0.05:0.1:0.95);

for i = 1:numel(Cf_sweep)
in(i) = Simulink.SimulationInput('sldemo_suspn_3dof');
in(i) = in(i).setBlockParameter(...
    'sldemo_suspn_3dof/Road-Suspension Interaction',...
    'Cf', num2str(Cf_sweep(i)))
end

out = parsim(in)
```

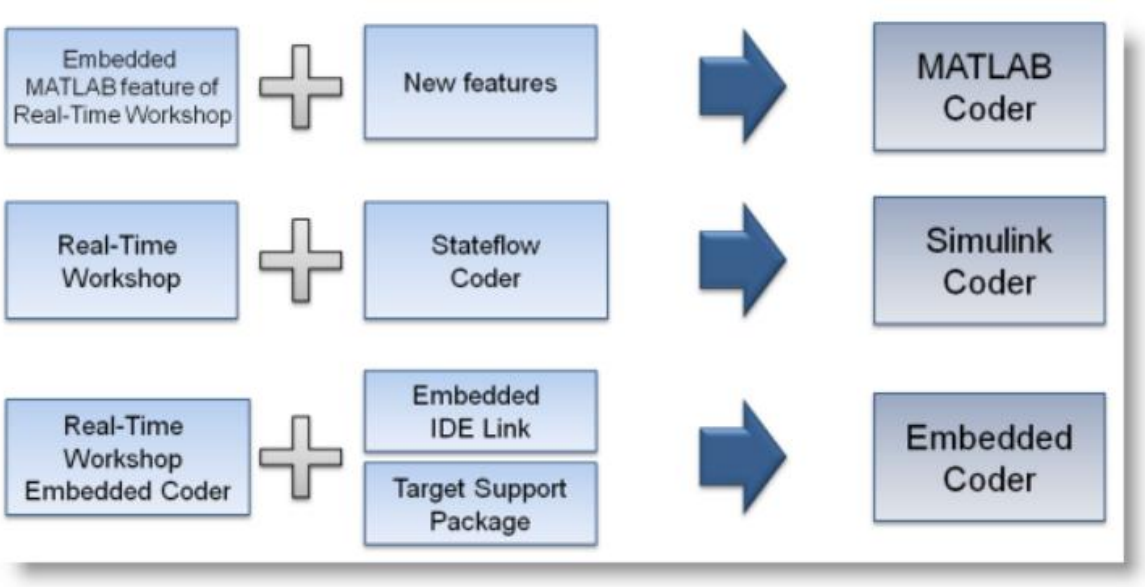
10 SIMULINK CODER – GENERATION DE CODE (SINCE 2011A)

10.1 INTRODUCTION

L'outil de compilation / génération de code s'appelait auparavant Real-Time Workshop. Depuis la R2011a il se nomme Simulink Coder :

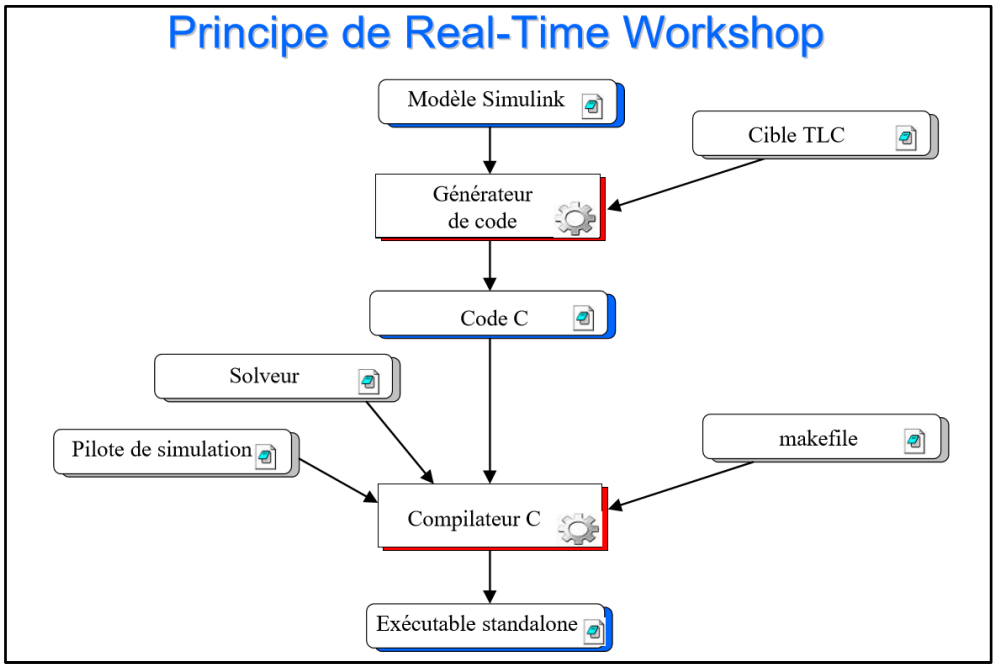
https://fr.mathworks.com/products/simulink-coder.html

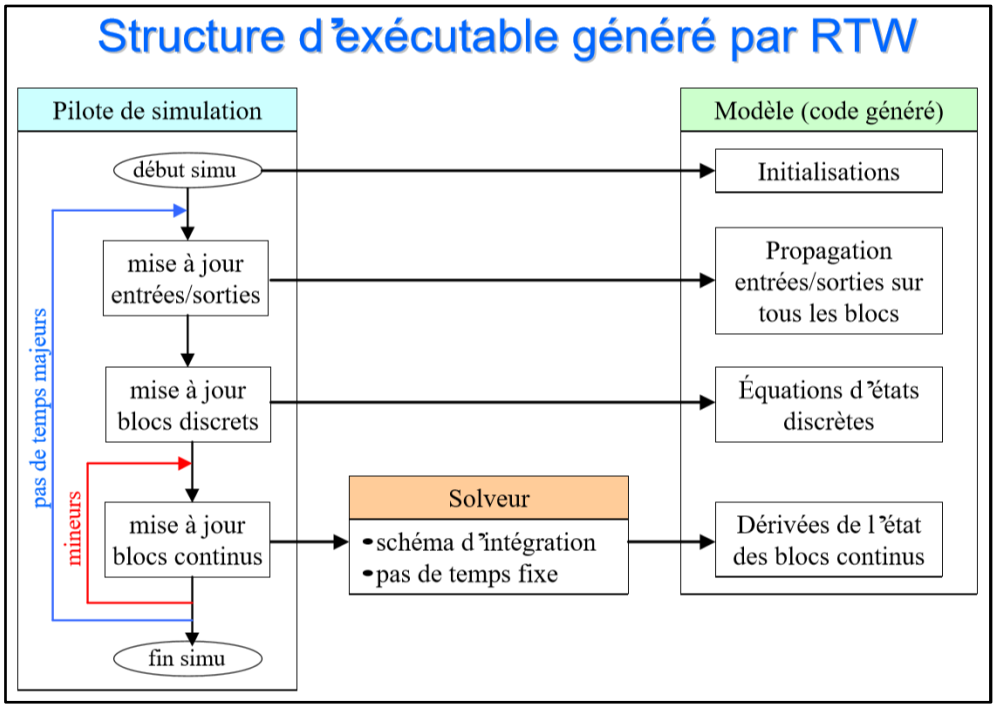
https://blogs.mathworks.com/simulink/2011/04/08/welcome-to-the-coders/



10.2 OLDIES : REAL-TIME WORKSHOP (RTW)

Slides issus d'un club matlab 2002 …





10.3 LES CONCEPTS

10.3.1 TARGET ET HOST

A *target* is an environment -- hardware or operating system -- on which your generated code will run. The process of specifying this environment is called *targeting*.

The process of generating target-specific code is controlled by a *system target file*, a *template makefile*, and a *make command*. To select a desired target, you can specify these items individually, or you can choose from a wide variety of ready-to-run configurations.

The *host* is the system you use to run MATLAB, Simulink, and the Real-Time Workshop. Using the build tools on the host, you create code and an executable that runs on your target system.

10.3.2 CODE FORMAT

A *code format* specifies a framework for code generation suited for specific applications.

When you choose a target configuration, you implicitly choose a code format. If you use the Real-Time Workshop Embedded Coder, for example, the code generated will be in *embedded C* format. The embedded C code format is a compact format designed for production code generation. Its small code size, memory usage, and simple call structure make it optimal for embedded applications.

Many other targets, such as the generic real-time (GRT) target, use the *real-time* code format. This format, less compact but more flexible, is optimal for rapid prototyping applications.

10.3.3 GENERIC REAL-TIME TARGET (GRT)

The Real-Time Workshop provides a generic real-time development target. The generic real-time (GRT) target provides an environment for simulating fixed-step models in single or multitasking mode. A program generated with the GRT target runs your model, in simulated time, as a stand-alone program on your workstation.

The GRT target allows you to perform code validation by logging system outputs, states, and simulation time to a data file. The data file can then be loaded into the MATLAB workspace for analysis or comparison with the output of the original model.

The GRT target also provides a starting point for targeting custom hardware. You can modify the GRT harness program, grt_main.c, to execute code generated from your model at interrupt level under control of a real-time clock.

10.3.4 TARGET LANGUAGE COMPILER FILES (.TLC)

The Real-Time Workshop uses *Target Language Compiler files* (or *TLC files)* to translate your Simulink model into code. The Target Language Compiler uses two types of TLC files during the code generation and build process. The *system target file*, which describes how to generate code for a chosen target, is the entry point for the TLC program that creates the executable. *Block target files* define how the code looks for each of the Simulink blocks in your model.

System and block target files have the extension .tlc. By convention, a system target file has a name corresponding to your target. For example, grt.tlc is the system target file for the generic real-time (GRT) target.

10.3.5 TEMPLATE MAKEFILES (.TMF)

The Real-Time Workshop uses template makefiles to build an executable from the generated code.

The Real-Time Workshop build process creates a makefile from the template makefile. Each line from the template makefile is copied into the makefile; tokens encountered during this process are expanded into the makefile.

The name of the makefile created by the build process is *model*.mk. The *model*.mk file is passed to a make utility. The make utility compiles and links an executable from a set of files.

By convention, a template makefile has an extension of .tmf and a name corresponding to your target and compiler. For example, grt_vc.tmf is the template makefile for building a generic real-time program under Visual C/C++.

10.3.6 BUILD PROCESS

A high-level M-file command controls the Real-Time Workshop build process. The default command, used with most targets, is make_rtw. When you initiate a build, the Real-Time Workshop invokes make_rtw. The make_rtw command, in turn, invokes the Target Language Compiler and other utilities such as make. The build process consists of the following stages:

1. First, make_rtw compiles the block diagram and generates a model description file, *model*.rtw.
2. Next, make_rtw invokes the Target Language Compiler to generate target-specific code, processing *model*.rtw as specified by the selected system target file.
3. Next, make_rtw creates a makefile, *model*.mk, from the selected template makefile.
4. Finally, make is invoked. make compiles and links a program from the generated code, as instructed in the generated makefile.

10.4 CREER UNE S-FUNCTION A PARTIR D'UN SUBSYSTEM

https://www.mathworks.com/help/rtw/ug/generated-s-function-block-deployment.html#f4918

To deploy your generated S-Function block for inclusion in other models *for simulation*, you need only provide the binary MEX-file object that was generated in the current working folder when the S-Function block was created. The required file is:

- *subsys*_sf.*mexext*

where *subsys* is the subsystem name and *mexext* is a platform-dependent MEX-file extension (see mexext). For example, SourceSubsys_sf.mexw64.

To deploy your generated S-Function block for inclusion in other models *for code generation*, provide the files that were generated in the current working folder when the S-Function block was created. The required files are:

- *subsys*_sf.c or .cpp, where *subsys* is the subsystem name (for example, SourceSubsys_sf.c)
- *subsys*_sf.h
- *subsys*_sf.*mexext*, where *mexext* is a platform-dependent MEX-file extension (see mexext)
- Subfolder *subsys*_sfcn_rtw and its contents

10.5 CODE C

On peut choisir différentes « cibles », cette possibilité étant généralement plutôt utilisée pour générer du code embarqué. Mais on peut également générer du code … réutilisable sous forme de S-function.

Ca tourne un peu en rond, mais ça permet aussi de protéger ses modèles lors d'échange avec des partenaires : voir Accelerate Simulation, Reuse Code, or Protect Intellectual Property by Using S-Function Target

Si l'on veut utiliser cette approche :

- pour de la simulation, seul le fichier .mex est nécessaire (platform-dependent, ex .mexw64)
- pour de la génération de code, sont requis le fichier .mex ainsi que le .c(pp) et le .h

10.6 CODER.EXTRINSIC

https://fr.mathworks.com/help/simulink/slref/coder.extrinsic.html

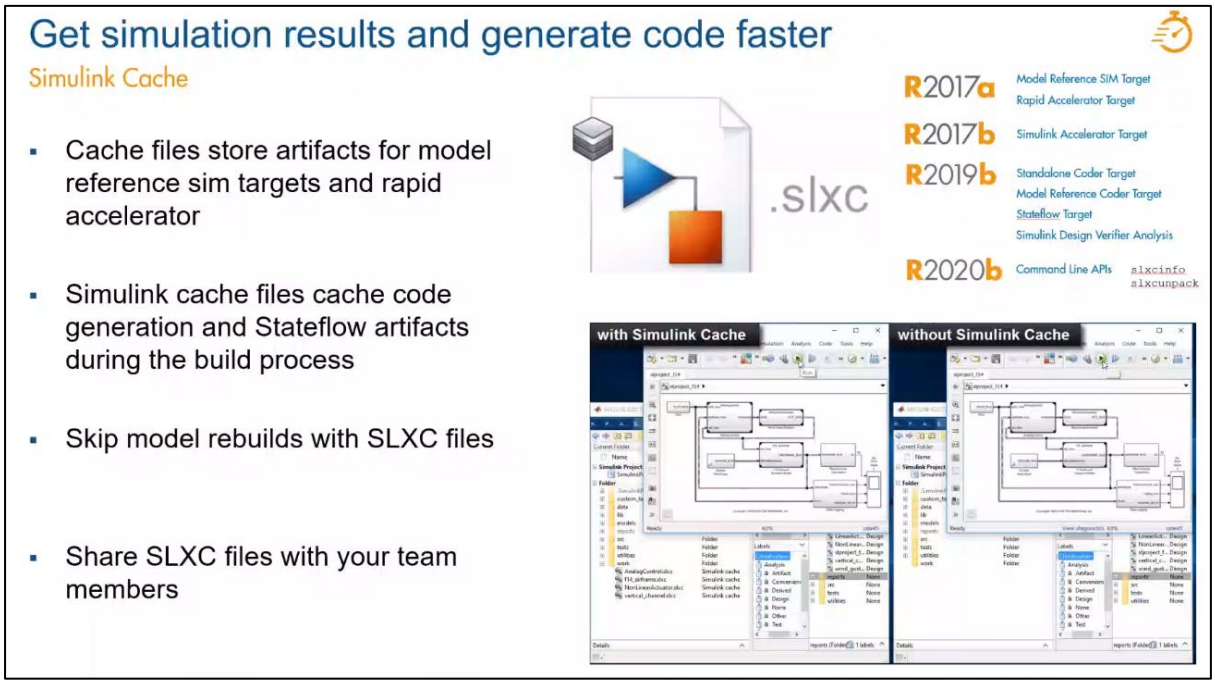
> During simulation, the code generator produces code for the call to an extrinsic function, but does not produce the function's internal code. Therefore, **simulation can run only on platforms where MATLAB software is installed**. During standalone code generation, MATLAB attempts to determine whether the extrinsic function affects the output of the function in which it is called — for example by returning mxArrays to an output variable. Provided that there is no change to the output, MATLAB proceeds with code generation, but excludes the extrinsic function from the generated code. Otherwise, compilation errors occur.

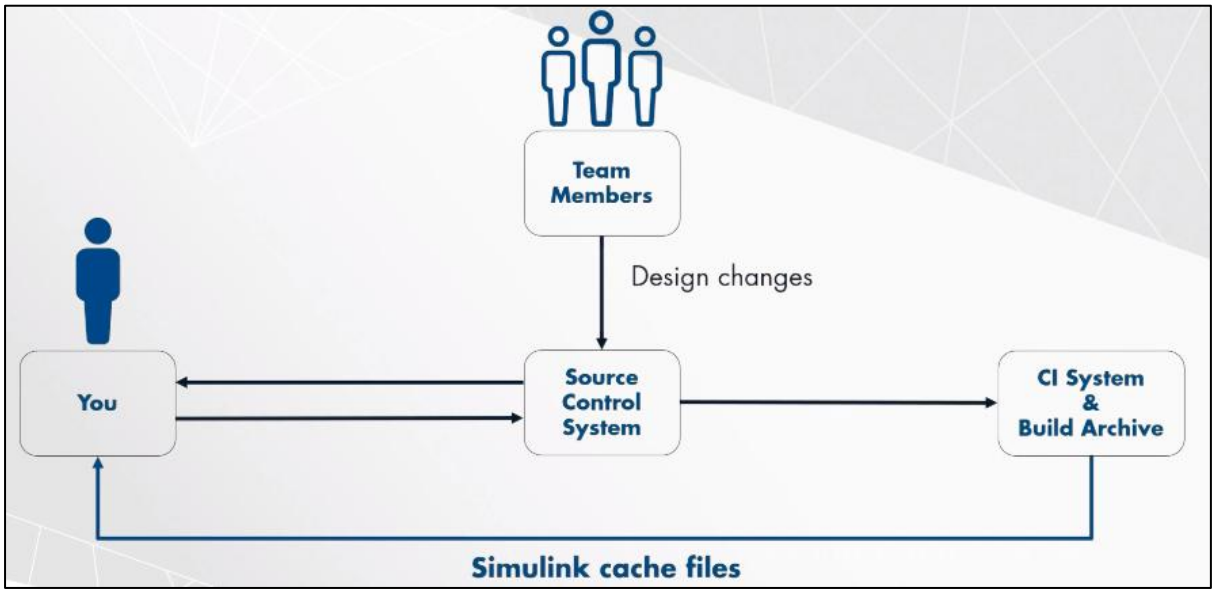
Limitations :

- The code generator does not support the use of coder.extrinsic to call functions that are located in a private folder.
- The code generator does not support the use of coder.extrinsic to call local functions.

10.7 SIMULINK CACHE

Conserver (voire échanger !) le cache permet d'éviter de faire un rebuild.





11 EMBEDDED CODER (SINCE 2011A)

Embedded Coder allows to optimize/customize the code generated by MATLAB Coder and Simulink Coder for production use

https://www.mathworks.com/products/embedded-coder.html

> Embedded Coder® generates readable, compact, and fast C and C++ code for embedded processors used in mass production. It extends MATLAB Coder™ and Simulink Coder™ with advanced optimizations for precise control of the generated functions, files, and data. These optimizations improve code efficiency and facilitate integration with legacy code, data types, and calibration parameters. You can incorporate a third-party development tool to build an executable for turnkey deployment on your embedded system or rapid prototyping board.
>
> Embedded Coder offers built-in support for AUTOSAR, MISRA C®, and ASAP2 software standards. It also provides traceability reports, code documentation, and automated software verification to support DO-178, IEC 61508, and ISO 26262 software development. Embedded Coder code is portable, and can be compiled and executed on any processor.

12 SIMULINK COMPILER (SINCE 2020A)

Simulink Compiler est apparu en R2020a

12.1 DIFFÉRENCE AVEC SIMULINK CODER

Compiler permet de générer des applications standalone exécutables

Coder génère du code C.

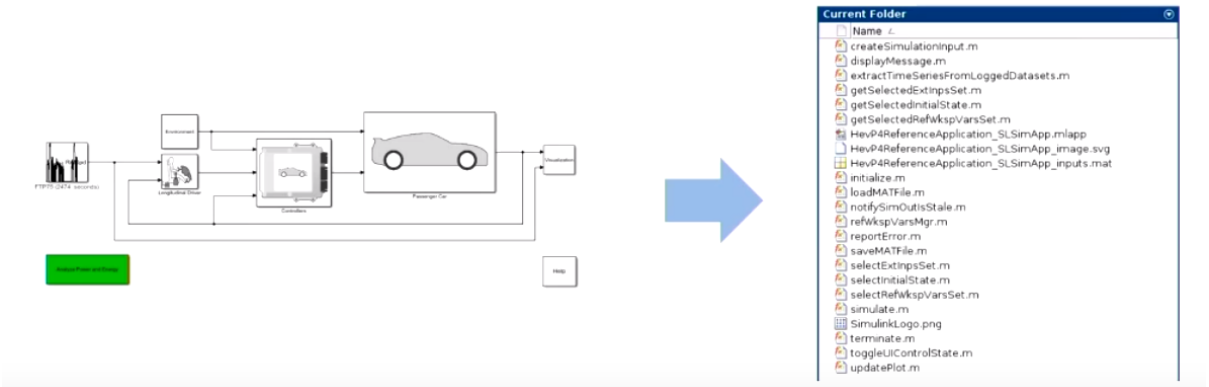
> Simulink® Compiler™ enables you to share Simulink simulations as standalone executables. You can build the executables by packaging the compiled Simulink model and the MATLAB® code to set up, run, and analyze a simulation. Standalone executables can be complete simulation apps that use MATLAB graphics and UIs designed with MATLAB App Designer. To cosimulate with an external simulation environment, you can generate standalone Functional Mockup Unit (FMU) binaries that adhere to the Functional Mockup Interface (FMI) standard.
>
> Simulink Coder™ generates and executes C and C++ code from Simulink models, Stateflow® charts, and MATLAB functions. The generated source code can be used for real-time and non-real-time applications, rapid prototyping, and hardware-in-the-loop testing. You can tune and monitor the generated code using Simulink or run and interact with the code outside MATLAB and Simulink.

Outputs and Support	Simulink Compiler	Simulink Coder
Main Use Case	Deploy simulations as standalone executables on desktop or production servers	Generate portable C/C++ code for Simulink model that can be deployed on embedded platforms or desktop
Output	Executable or software component or shared library	Portable and readable C/C++ source code
Simulink Block Support	All the blocks supported in Rapid Accelerator mode in Simulink	A subset of Simulink blocks
Supported Blocksets	All the blocksets supported by Rapid Accelerator mode in Simulink	A subset of Simulink blocks
Production	MATLAB Production Server	Embedded Coder
Simulink Graphics Support	Supports MATLAB Graphics.	None
Library Dependencies	MATLAB Runtime	None

Common Questions	Simulink Compiler	Simulink Coder
What files are produced?	Shared executables or libraries, along with the required MATLAB Runtime components.	Source code (`*.c` & `*.h`) that can be compiled into shared libraries and executables
Which platforms can these files be deployed to?	All platforms supported by MATLAB (Windows, Mac, and Linux)	Any platform that supports ANSI/ISO C/C++ code
Does it generate readable code?	No, only non-readable shared libraries	Yes, readable source code
Is it faster than Simulink?	Runs at the same speed as Rapid Accelerator mode in Simulink.	Has the potential to be faster, depending on the algorithm. The generated code is not faster for optimized MATLAB functions (such as FFT, or Image Processing, and Computer Vision functions) Use GPU Coder GPU Coder™ to generate CUDA source code that potentially runs faster on NVIDIA GPUs.
Does it take advantage of hardware accelerators?	Supports the same hardware as MATLAB, including GPUs and AVX. Multicore and clusters are supported via Parallel Computing Toolbox	C code running on local multicore machines can be supported using the OpenMP API. Use GPU Coder to generate CUDA source code that runs on NVIDIA GPUs. Use HDL Coder™ to generate Verilog or VHDL that runs on FPGAs.
What is the difference in memory use on a desktop?	Highly dependent on the executables. MATLAB Runtime itself uses more memory than the Simulink Coder.	Highly dependent on the MATLAB code.
What file I/O formats does each software support?	Supports all formats supported in MATLAB	Limited file support: text files, audio, and video formats. Does not support image formats.

12.2 EN LIGNE DE COMMANDE

```
>> simulink.compiler.genapp(modelName, varargin)
```



13 SIMULINK PROJECT

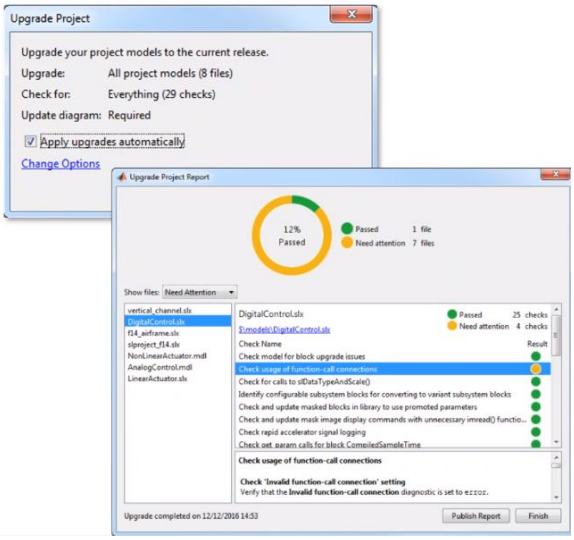
Share, reuse, and upgrade your project files

Project Upgrade

R2017a

Easily update all the models in your Simulink Project to the latest release

- Avoid the manual process of upgrading one model at a time
- Simulink Project upgrade is an easy to use UI to automate the upgrade process of all the models in a Simulink project
- Fixes are automatically applied and a report gets generated



13.1 EXPORTER UN PROJET COMPLET DANS UNE VERSION PRECEDENTE

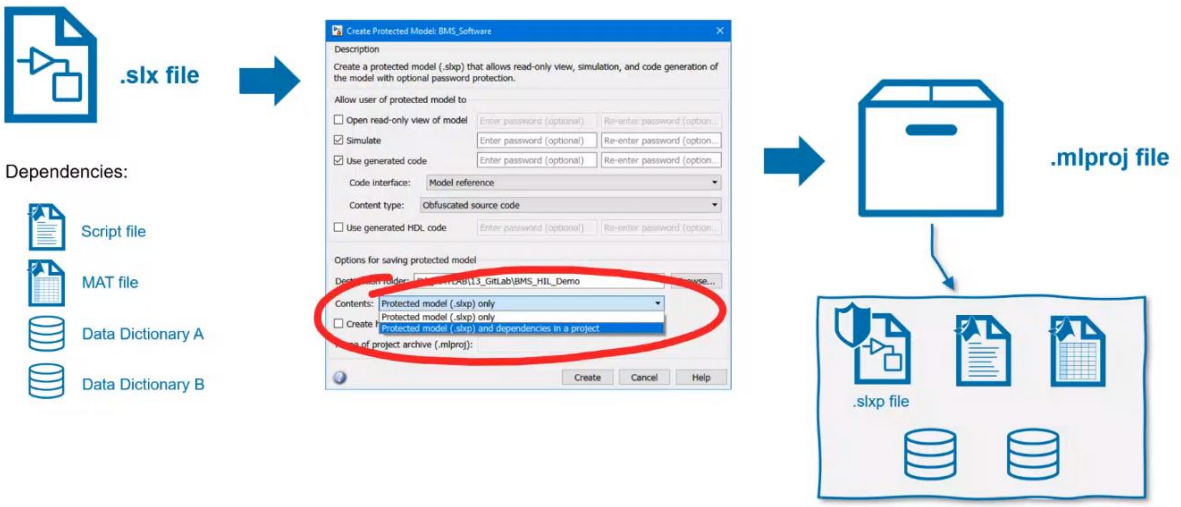
```
Simulink.exportToVersion(currentProject,'R2019b_AirframeProject','R2019b')
```

13.2 PARTAGE DE MODELES

Sharing protected models in a Project

Protected Model Contents

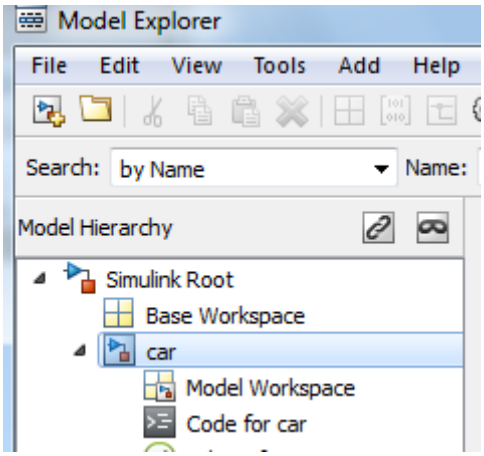
R2020b



14 ANALYSE D'UN MODELE

14.1 MODEL EXPLORER / WORKSPACE

NB : le modèle a son propre workspace



Pour y définir une variable :

- Sélectionner le Model Workspace
- Add / Matlab variable
- La renommer et lui donner une valeur

Par ordre de priorité, Simulink cherche les variables :

- Dans le Model Workspace
- Dans le Base Workspace

Pour récupérer une variable du model workspace dans le base workspace :

- créer un objet Workspace :

```
mdlWks=get_param(gcs,'ModelWorkspace')
```

- utiliser la méthode getVariable

```
a=getVariable(mdlWks,'a')
```

14.2 LINEARISATION DE SYSTEMES

14.2.1 IMPORTANT

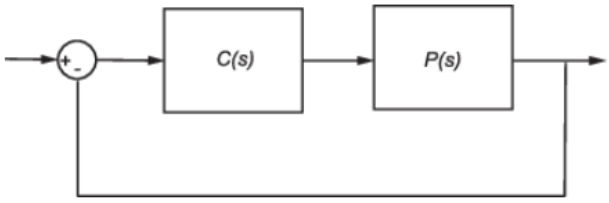
A lire attentivement

https://fr.mathworks.com/help/slcontrol/ug/open-loop-response-of-control-system-for-stability-margin-analysis.html

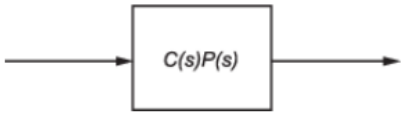
et

https://fr.mathworks.com/help/slcontrol/ug/specify-model-portion-to-linearize.html

The open-loop response of a control system is the combined response of the plant and the controller, excluding the effect of the feedback loop. For example, the following block diagram shows a single-loop control system.

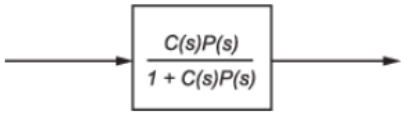


If the controller, $C(s)$, and plant, $P(s)$, are linear, the corresponding open-loop transfer function is $C(s)P(s)$.

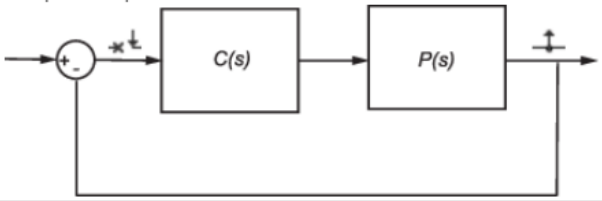
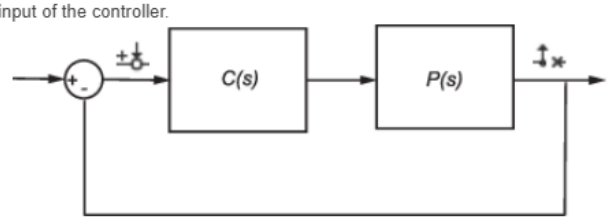


To remove the effects of the feedback loop, insert a loop opening analysis point without manually breaking the signal line. Manually removing the feedback signal from a nonlinear model changes the model operating point and produces a different linearized model. For more information, see How the Software Treats Loop Openings.

If you do not insert a loop opening, the resulting linear model includes the effects of the feedback loop.



A noter que les deux solutions suivantes sont équivalentes :

Analysis Point	Description	To compute $C(s)P(s)$
Open-loop input	Specifies a loop opening followed by an input perturbation.	Specify an open-loop input at the input to the controller and an output measurement at the output of the plant. 
Open-loop output	Specifies an output measurement followed by a loop break.	Specify an open-loop output at the output of the plant and an input perturbation at the input of the controller. 

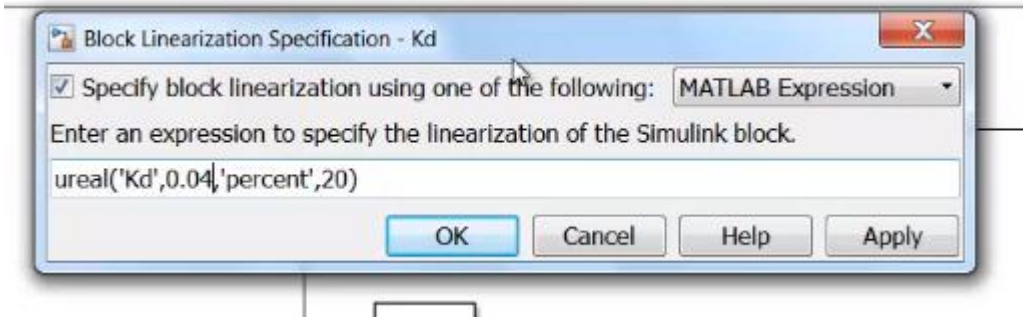
14.2.2 VISUALISATION DIAGRAMME DE BODE PENDANT LA SIMULATION

https://fr.mathworks.com/help/slcontrol/ug/visualize-bode-response-of-simulink-model-during-simulation.html

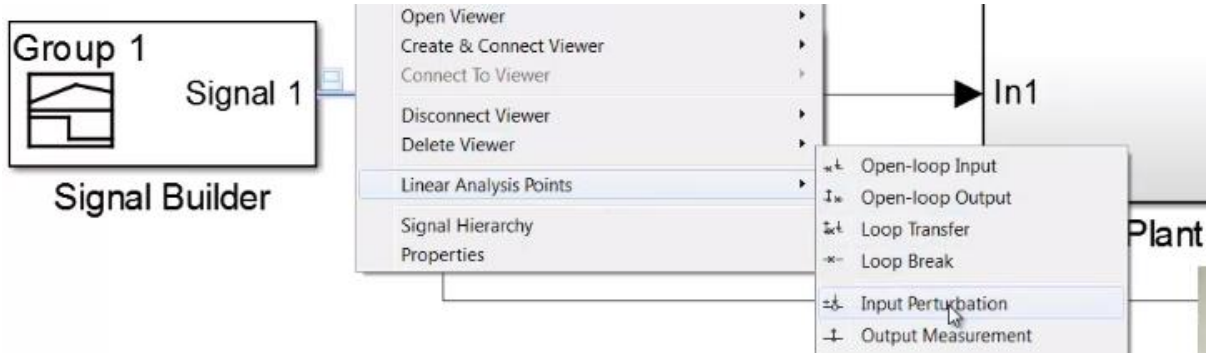
14.2.3 LINEARISATION AVEC INCERTITUDES

(A vérifier : tout ceci nécessite la Robust Control Toolbox)

Sur un bloc constante, on peut utiliser la méthode suivante pour inclure les incertitudes dans un modèle :

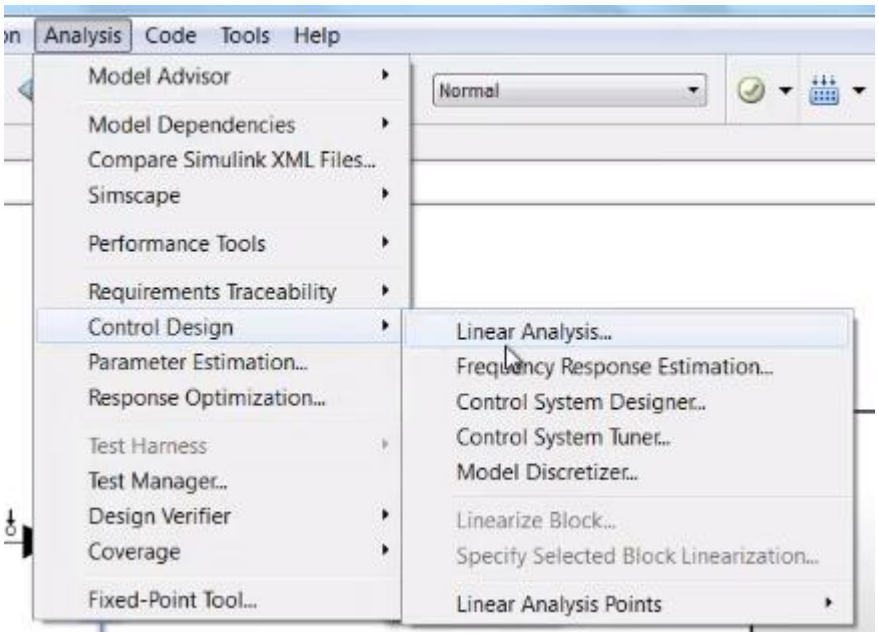


Cliquer sur le signal d'entrée du plant model et le définir comme input perturbation :

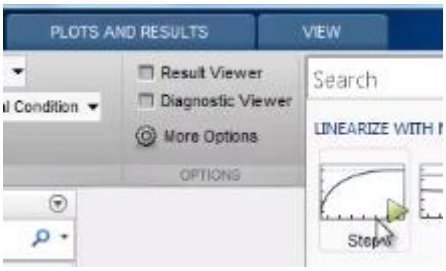


Faire de même sur le signal de sortie, et sélectionner Output Measurement.

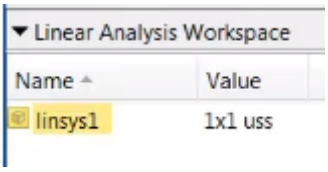
Passer ensuite dans le menu Analysis / Control Design / Linear Analysis



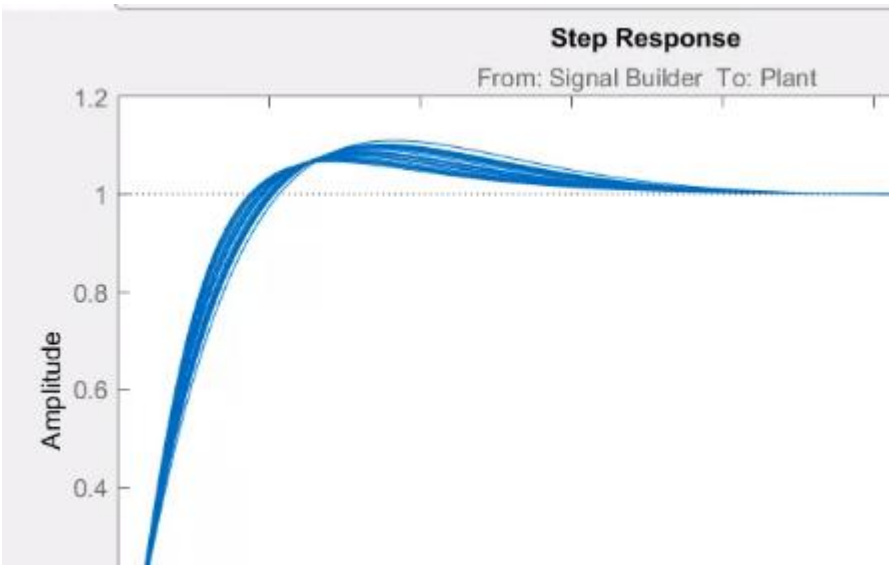
Choisir une analyse, par exemple Step Response pour commencer :



Un système linéaire de type `uss` (uncertain state space model) est automatiquement créé :

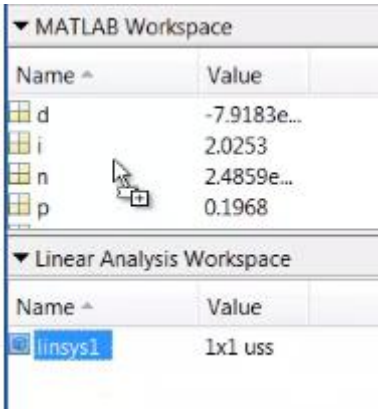


La réponse est tracée avec la dispersion :



Reste à savoir quelle est la « moins bonne » solution : on ne sait pas ici à quelle valeur du gain (issu du bloc sur lequel on a introduit l'incertitude) correspond la courbe avec le plus fort overshoot, par exemple.

Mieux vaut alors passer dans Matlab, en faisant tout simplement glisser le linsys depuis le linear workspace jusqu'au workspace Matlab :



On utilise la fonction `wcgain` :

```
[mgain,wcvalue]=wcgain(linsys1)
```

Les deux sorties sont le maximum gain, et la worst-case value :

```
mgain =

          LowerBound: 1.1133
          UpperBound: 1.1134
    CriticalFrequency: 9.5955


wcvalue =

    Kd: 0.0480
```

On peut ensuite calculer le système correspondant, avec la fonction `usubs`

```
sys_worst=usubs(linsys1,wcvalue)
```

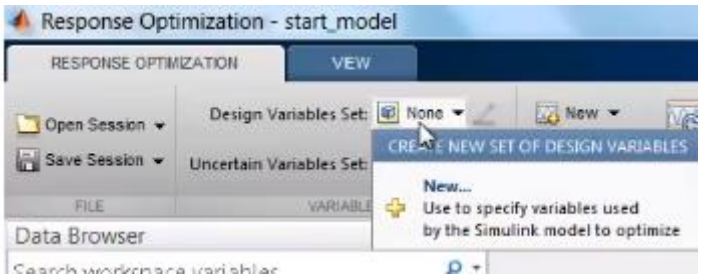
Puis tracer sa réponse :

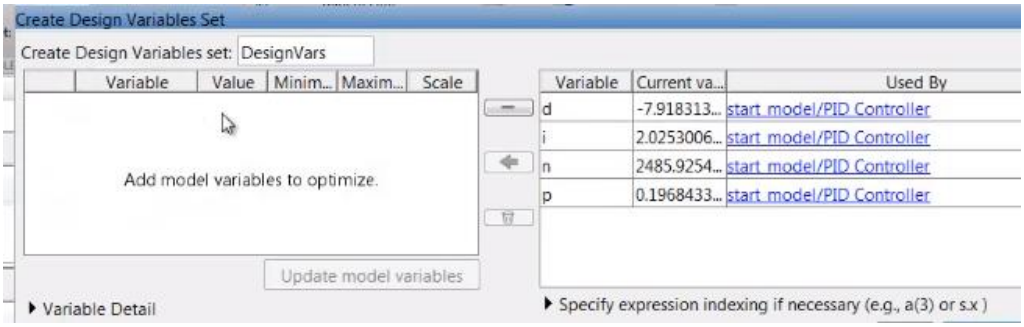
```
step(linsys1,sys_worst)
```

Jusque là on s'est contenté d'étudier l'influence de la dispersion d'un paramètre sur la réponse d'un système, sur la base d'un seul critère qui est le worst-case gain.

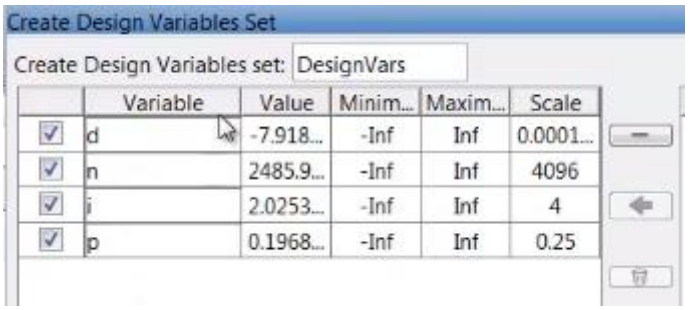
On peut aller plus loin avec l'outil Analysis / Response Optimization

On définit les paramètres de conception :

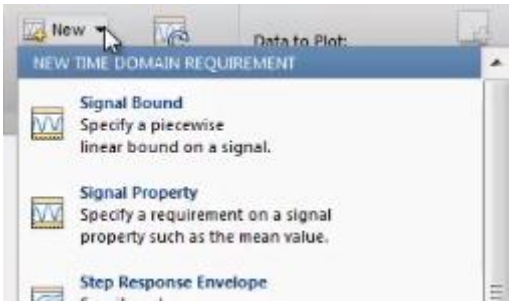




On passe tous les paramètres de notre PID dans le bloc de gauche :



On définit ensuite les requirements, qui peuvent être définis dans le domaine temporel ou fréquentiel



Et voilà … ou presque !

14.2.4 TRIMMING

Le trimming consiste à trouver un ensemble de commandes qui permette de se placer en équilibre à un (vecteur d')état donné.

Voir https://www.youtube.com/watch?v=5gEattuH3tI

Trimming and Linearization, Part 1: What is Linearization?

et

https://www.youtube.com/watch?v=K1oMF4684OQ

Trimming and Linearization, Part 2: The Practical Side of Linearization

14.3 REACH-COREACH TOOL

The tool highlights data/control flow dependencies in Simulink models and performs model slicing.

https://fr.mathworks.com/matlabcentral/fileexchange/51180-reach-coreach-tool

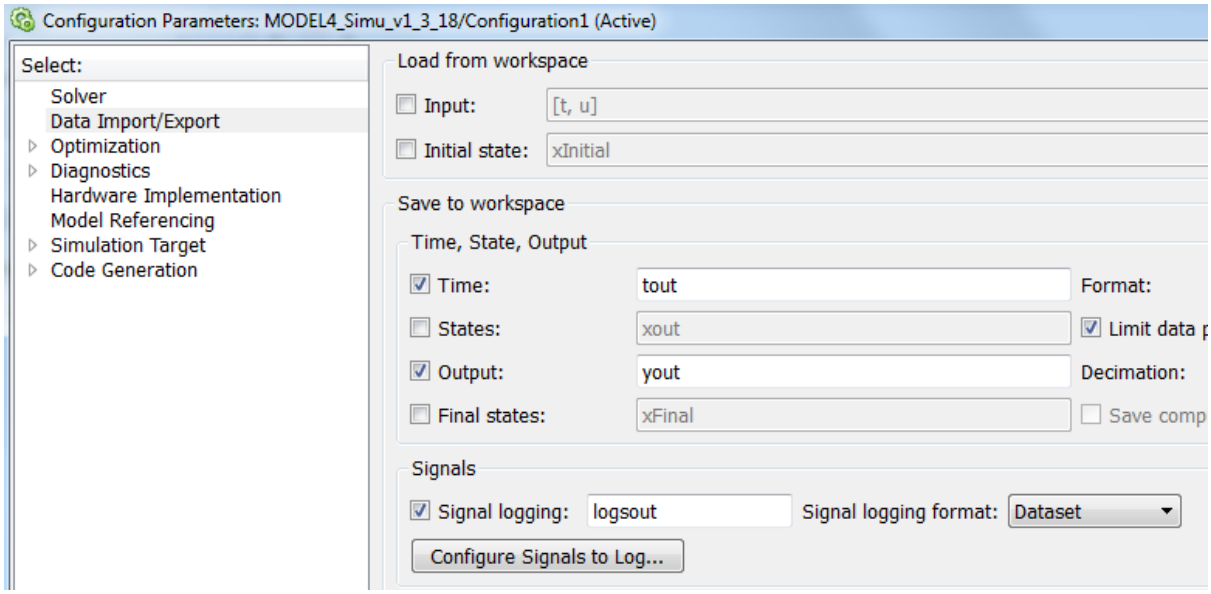
14.4 LOG DE SORTIES

14.4.1 DATASET

Pour enregistrer un signal il vaut mieux qu'il ait déjà un nom, ensuite clic droit / Propriétés et cocher Log signal data



Cela crée un `dataset` dont le nom peut être spécifié dans les Configuration Parameters, dans le champ « Signal Logging » de la rubrique Data Import/Export :



Après la simulation il apparait dans le Workspace, mais on ne peut l'ouvrir par un simple double-clic.



On disposer des méthodes suivantes pour y accéder :

`logsout.getElementNames` permet d'accéder aux noms des variables qu'il contient

```
>> logsout.getElementNames

ans =

    'ACC_M_BRK'
    'ACC_M_PWT'
    'AEB_M_BRK'
    'M_BRK_computed'
    'P_BRK_eff'
    'P_BRK'
```

`logsout.getElement('varName').Values` permet de voir la nature de la variable :

```
>> logsout.getElement('ACC_M_BRK').Values
  timeseries

  Common Properties:
            Name: 'ACC_M_BRK'
            Time: [716x1 double]
        TimeInfo: [1x1 tsdata.timemetadata]
            Data: [716x1 double]
        DataInfo: [1x1 tsdata.datametadata]
```

On peut alors en afficher les valeurs :

```
logsout.getElement('ACC_M_BRK').Values.Data
```

.. ou, puisqu'il s'agit d'une TimeSeries, la tracer en fonction du temps :

```
plot(logsout.getElement('ACC_M_BRK').Values.Time,logsout.getElement('ACC_M_BRK').Values.Data)
```

14.5 PERFORMANCE

14.5.1 PROFILER REPORT

Pour déterminer quels sont les blocs les plus consommateurs :

Analysis / Performance Tools / Show Profiler Report

On obtient un tableau contenant la liste des fonctions et le temps associé :

Name	Time		Calls	Time/call	Self time	
simulate(AEB_v1c_MADA17_Matlab13b)	799.08392230	100.0%	1	799.0839223000003	0.00000000	0.0%
simulationPhase	745.46637860	93.3%	1	745.4663786000001	66.19122430	8.3%
AEB_v1c_MADA17_Matlab13b.Outputs.Major	672.44231050	84.2%	6001	0.1120550425762	236.79391790	29.6%

The outputs.major and outputs.minor are related to major and minor time steps respectively.

For more information on these time steps, please check the following documentation on types of sample time: https://www.mathworks.com/help/simulink/ug/types-of-sample-time.html

Also, have a look at the following MATLAB Answers link, you have a very nice explanation:

https://in.mathworks.com/matlabcentral/answers/334668-what-determines-how-many-times-a-function-is-called-in-simulink?s_tid=answers_rc1-1_p1_MLT

14.5.2 CAUSES POSSIBLES DE LENTEUR

https://www.researchgate.net/post/How_can_I_solve_the_low_speed_of_my_simulation_in_simulink

There are several possibilities that could be causing your simulation to run slowly:

1. You have a **MATLAB Function Block** - When a MATLAB Fcn block is present, the MATLAB interpreter is called at each time step. This drastically reduces the simulation speed. So, you should use the built-in Fcn block whenever possible.
2. **S-functions written as MATLAB files** - S-functions are evaluated at every time step. By converting the MATLAB file to a MEX-file, performance can be greatly improved. Also, whenever possible, use the built-in blocks to model the S-functions.
3. **Small step sizes or sample times** (or a mixture of sample times that are not multiples of each other). It is important to keep the step size small enough to capture important events during the simulation. Conversely, a step size that is too small will produce more output points than necessary and slow down the simulation.
4. The **maximum step size** is too small. If you changed the maximum step size, try running the simulation with the default value (set to auto).
5. You may have asked for too much accuracy. The **default relative tolerance** (0.1% accuracy) is usually sufficient. For models with states that go to zero, if the absolute tolerance parameter is too small, the simulation may take too many steps around the near-zero state values. For more information on Error Tolerances, see page 5-13 of the Using Simulink Manual. Please remember that all tolerance values to be entered are absolute values. So the default relative tolerance value when it is set to 1e-3 , it means the relative tolerance is 0.001 or in terms of percentage it is 0.1%.
6. The **time scale** may be too long. Reduce the time interval.
7. Your model includes a Memory block. Using a **Memory block causes the variable-order solvers** (ode15s and ode113) to be reset back to order 1 at each time step.
8. Any masked blocks in the Extras library that incorporate one of the three above (i.e., graph scopes, autoscaling scopes, spectrum analyzers, etc). Unmask the blocks to see if they call an S-function.
9. Built in-scope blocks, although their effect is most likely very small.
10. **Algebraic loops** - The solutions to algebraic loops are iterative and performed at every time step. Therefore, they severely degrade speed. For more information on Algebraic loops, see page 3-18 of the Using Simulink Manual.
11. **Do not feed a White Noise block into an Integrator**. For continuous systems use the band-limited white noised block in the Extras/Sources library.
12. **The problem may be stiff but you are using a non-stiff solver**. Try using ode15s.
13. You may be hitting a **zero crossing** continuously and the simulation is getting 'stuck' causing it to take a very long (and possibly infinite time to complete). To fix this, you can disable the zero crossing detection. This can be done by selecting the Disable zero crossing detection option on the Advanced pane of the Simulation Parameters dialog box. In R11, this can be done by going under the Simulation -> Parameters -> Diagnostics section of the model.
14. The simulation speed could also be affected if the model is very complex (involves a lot of model referencing and subsystems) and involves a lot of **data logging**. Disabling the data logging feature also improves simulation speed

14.5.3 AMELIORER LA PERFORMANCE

https://fr.mathworks.com/company/newsletters/articles/improving-simulation-performance-in-simulink.html

Normal/Accelerator/Rapid Accelerator modes

- In Normal mode, Simulink interprets your model during each simulation run. If you change your model frequently, this is generally the preferred mode to use because it requires no separate compilation step.
- In Accelerator mode, Simulink compiles a model into a binary shared library or DLL where possible, eliminating the block-to-block overhead of an interpreted simulation in Normal Mode. Accelerator mode supports the debugger and profiler, but not runtime diagnostics.
- In Rapid Accelerator mode, Simulink compiles a standalone executable for the model, which can run on a separate processing core. You can use Rapid Accelerator mode only when the full model is capable of generating code, and this mode restricts interaction with the model during simulations. For example, Rapid Accelerator mode does not support debugging. As with Accelerator mode, it's best to use Rapid Accelerator mode when your simulations take much more time than the one-time cost of compilation

14.6 COMPLEXITÉ CYCLOMATIQUE

En gros le nombre de « chemins » que peut prendre un programme à cause des if/else et autres blocs switch par exemple.

A réduire autant que possible, donc, d'autant que ça impacte le nombre de tests à réaliser.

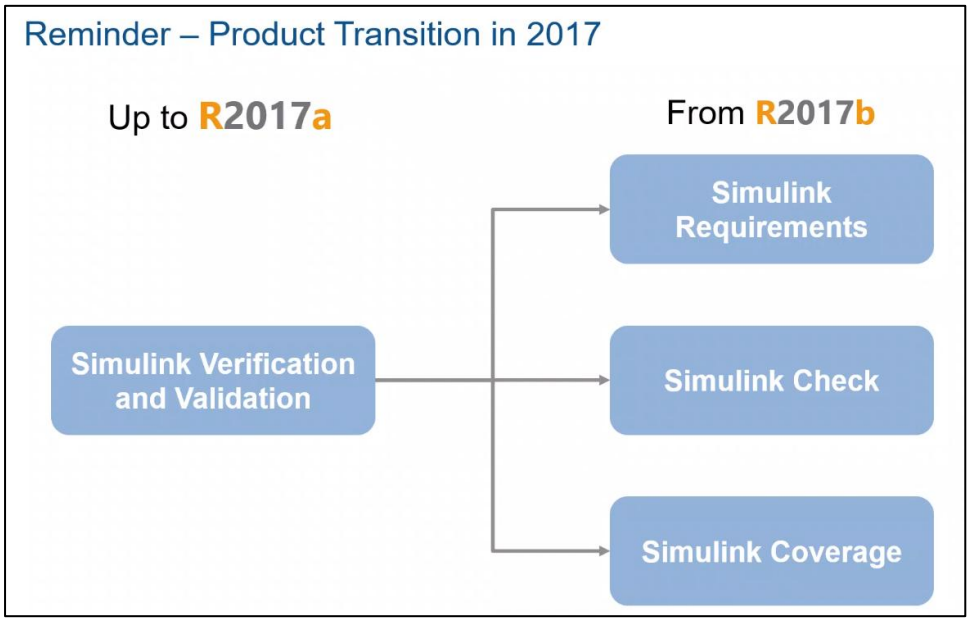
14.7 COUVERTURE

Analysis / Coverage

14.8 MESSAGES D'ERREUR / WARNINGS

Message	Solution
Matching "Goto" for "From"	`Ctrl+E / Diagnostics / Connectivity` Décocher `Unconnected line`

15 VERIFICATION ET VALIDATION



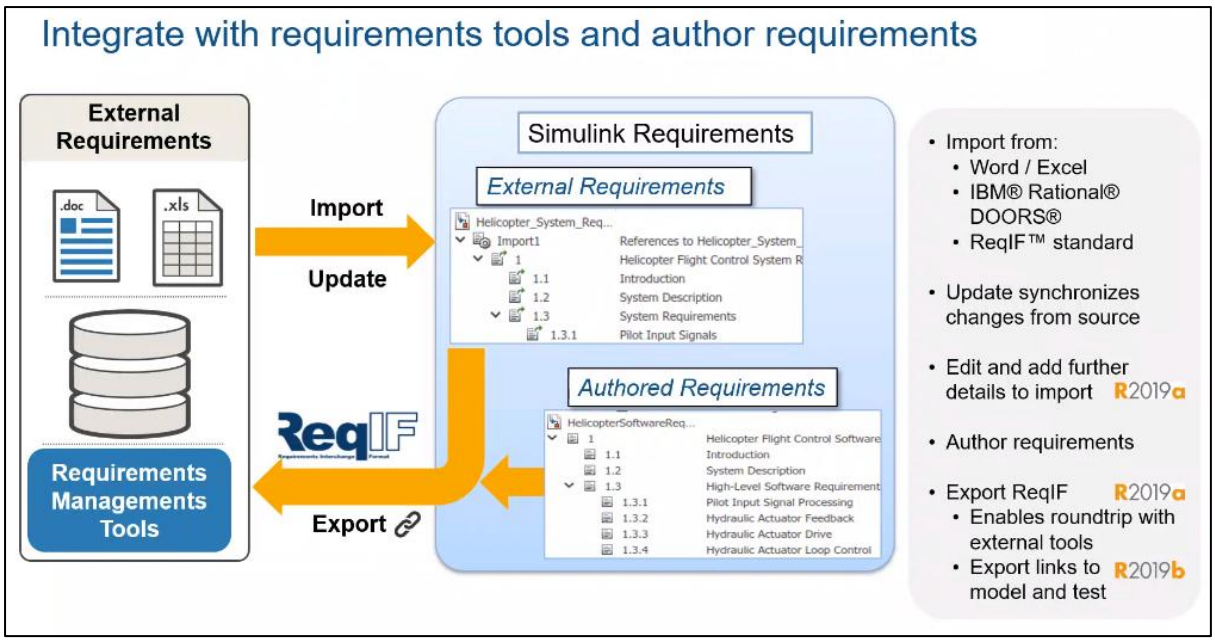
https://fr.mathworks.com/campaigns/offers/automating-verification-and-validation.confirmation.html?elqsid=1555659703472&potential_use=Commercial

15.1 EXIGENCES - REQUIREMENTS

15.1.1 INTRO

On peut créer des liens bijectifs entre le modèle et un document contenant des specifications, sous des formats divers (doc, pdf, doors …)

Simulink Requirements est dispo depuis la release 2017b.



15.1.2 OBJECTIF

Exploiter les fonctionnalités de gestion d'exigences de Matlab/Simulink, qui permettent d'améliorer la qualité des modèles.

Attention : cela ne permet que de formaliser les exigences, et non de vérifier qu'elles sont satisfaites.

15.1.3 SOLUTIONS POSSIBLES

Les exigences peuvent être "intégrées" (embedded) dans le modèle, ou externalisées dans un fichier externe.

Avantage de la solution intégrée : plus facile à transmettre, seul le fichier slx est nécessaire

Inconvénient : le fichier .slx doit être modifié à chaque modification des exigences

NB : jusqu'à la release 2017a, les fichiers d'exigences externes ont pour extension .req. A partir de la 2017b, ça devient .slmx

15.1.4 PREREQUIS : RMI SETUP

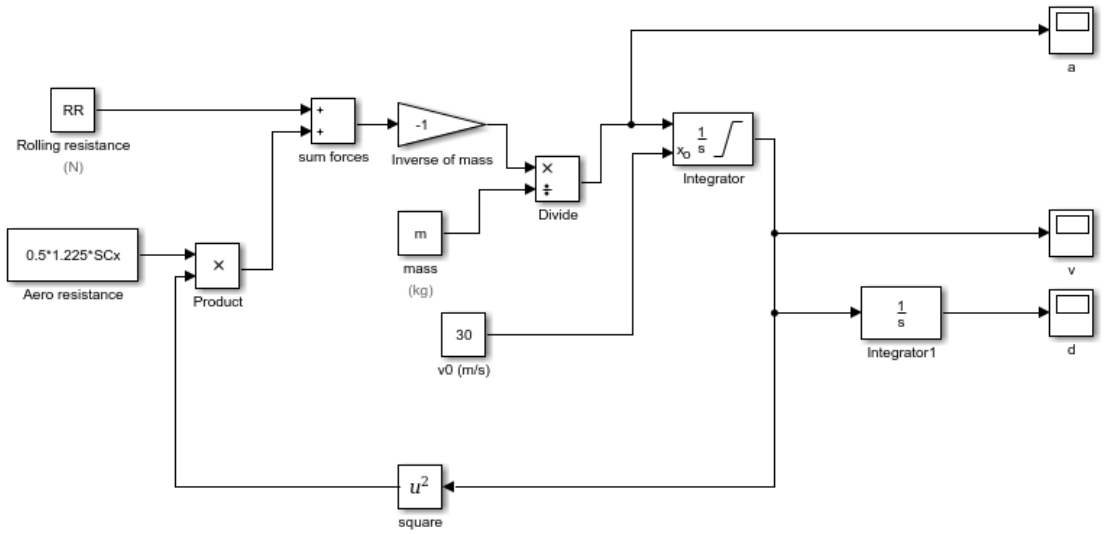
RMI = Requirements Management Interface

A ne lancer qu'à chaque nouvelle version de Matlab (tbc ?)

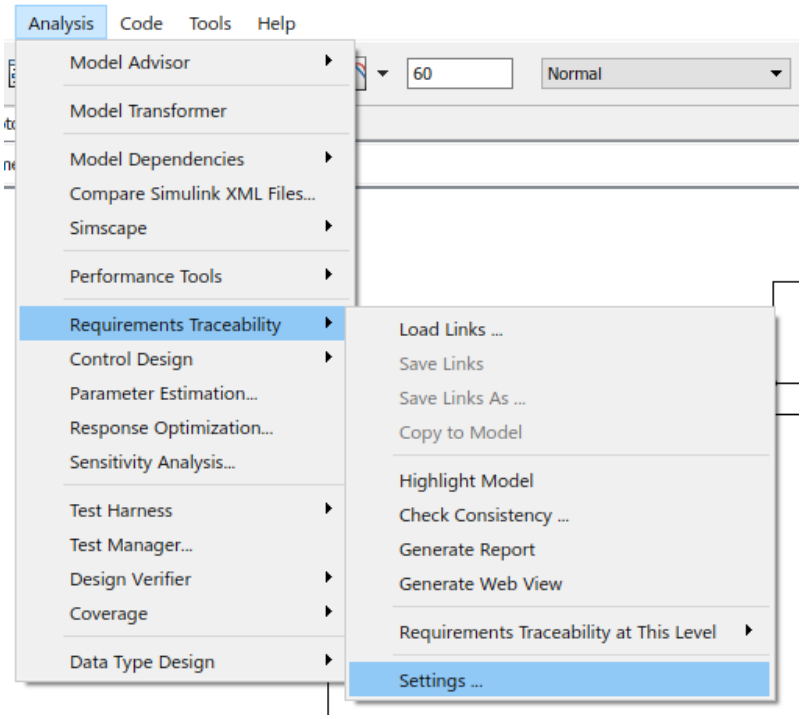
```
>> rmi setup
Ensuring required Active-X controls ..
Verifying MATLAB automation server path ..
Registering this MATLAB executable as an automation server ..
>>
```

15.1.5 EXIGENCES INTERNES

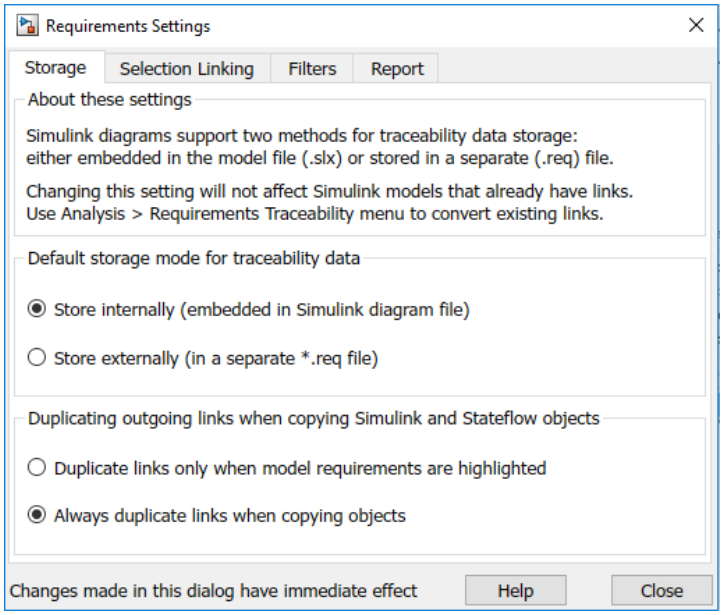
On part d'un modèle qui permet de calculer la perte de vitesse d'un engin motorisé soumis aux résistances aéro et de roulement



On passe pour commencer dans le menu Analysis / Requirements Traceability / Settings :



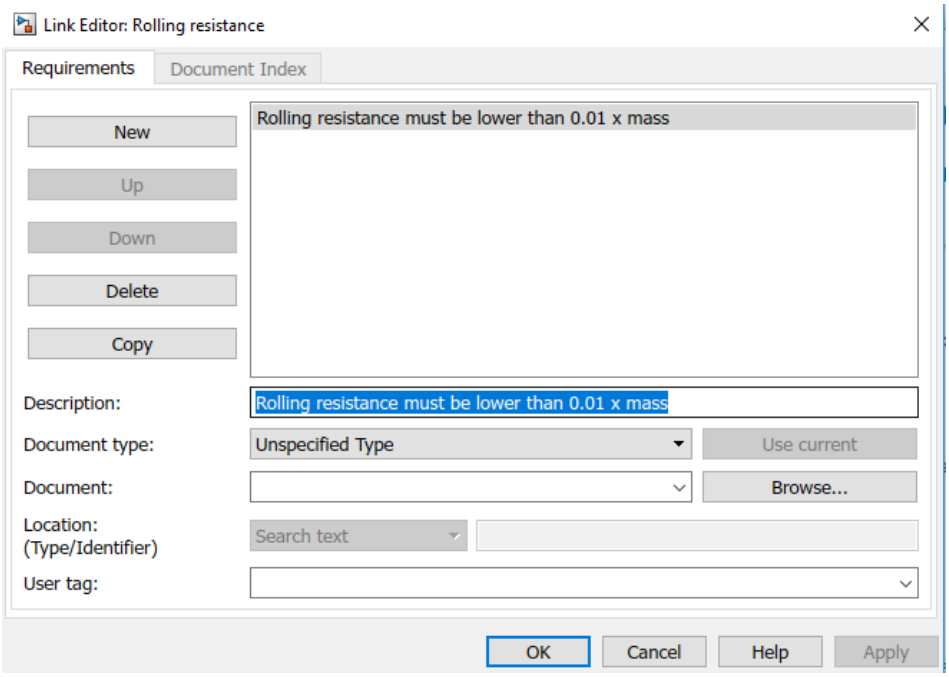
"**Store internally**"



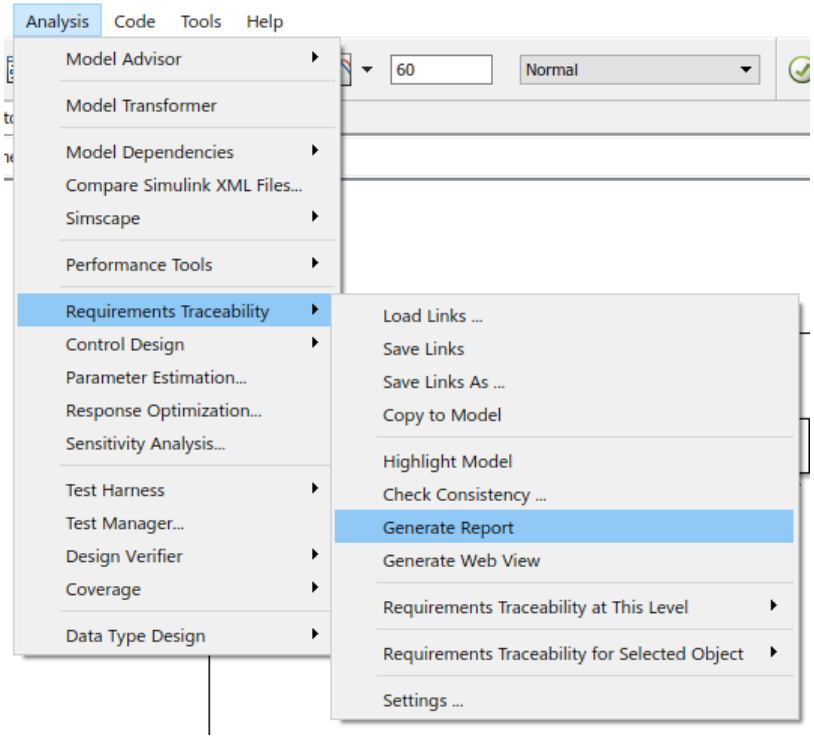
Clic-droit sur un bloc du modèle / Requirements / Open Link Editor :



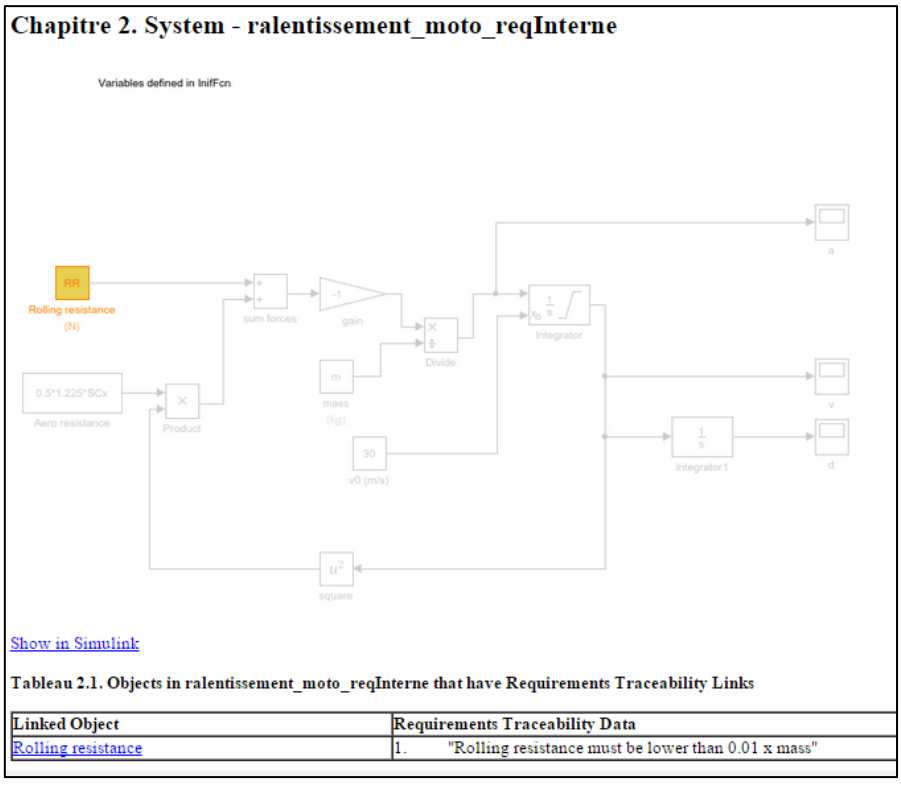
Dans la fenêtre qui s'ouvre, "New" puis entrer une description explicite :



Après validation, l'exigence est définie. On peut alors générer un rapport via le menu Analysis / Requirements / Generate report



dont voici un extrait du résultat



Chapitre 2. System - ralentissement_moto_reqInterne

Variables defined in InitFcn

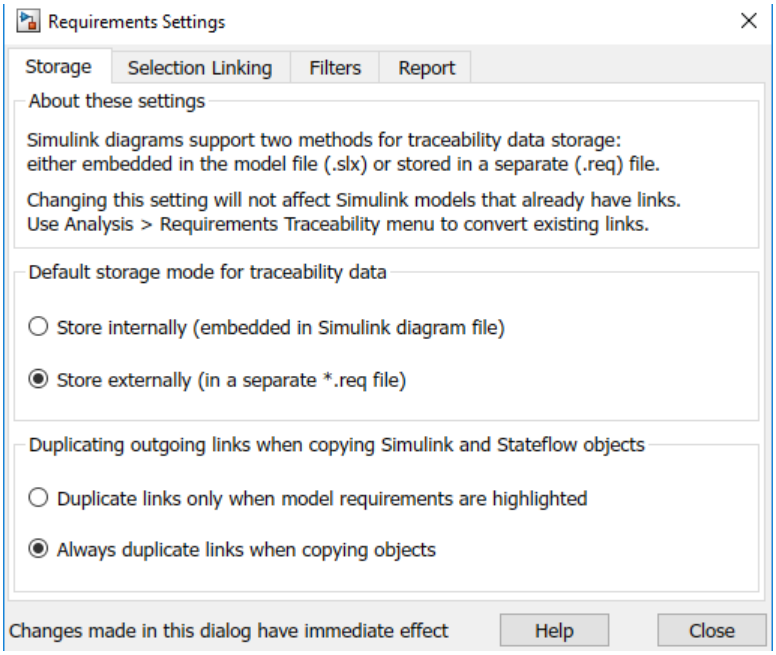
Show in Simulink

Tableau 2.1. Objects in ralentissement_moto_reqInterne that have Requirements Traceability Links

Linked Object	Requirements Traceability Data
Rolling resistance	1. "Rolling resistance must be lower than 0.01 x mass"

15.1.6 EXIGENCES EXTERNES

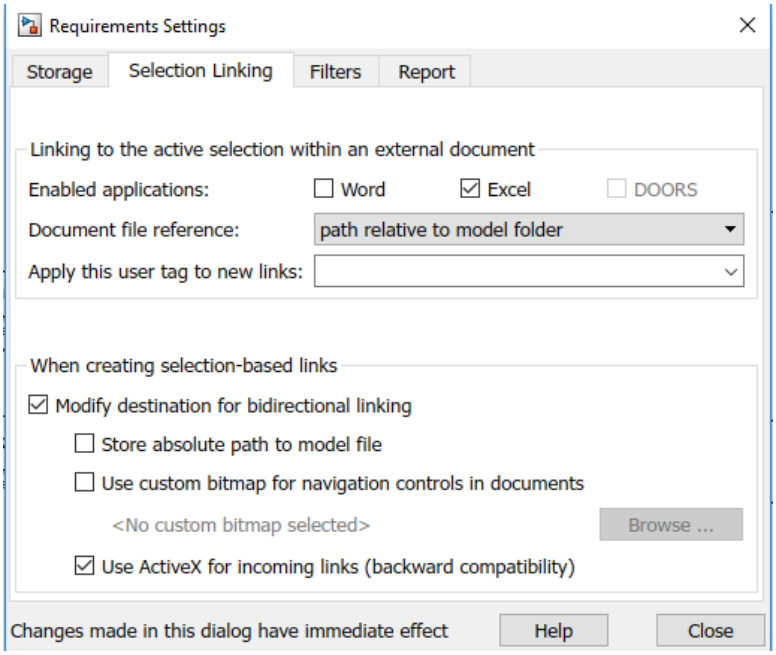
"**Store externally**" dans les settings (ici .req car test réalisé en 2016b)



Dans l'onglet Selection Linking :

- on privilégie « **path relative to model folder** » pour spécifier la façon dont le lien doit être fait entre le modèle et le doc
- on coche “**modify destination**” pour avoir la bidirectionnalité entre le doc d’exigences et le modèle.
- on coche "**Use ActiveX**" de manière à pouvoir retourner vers le modèle Simulink lorsqu'on clique sur les liens créés dans le document d'exigences

15.1.6.1 EXEMPLE AVEC FICHIER EXCEL



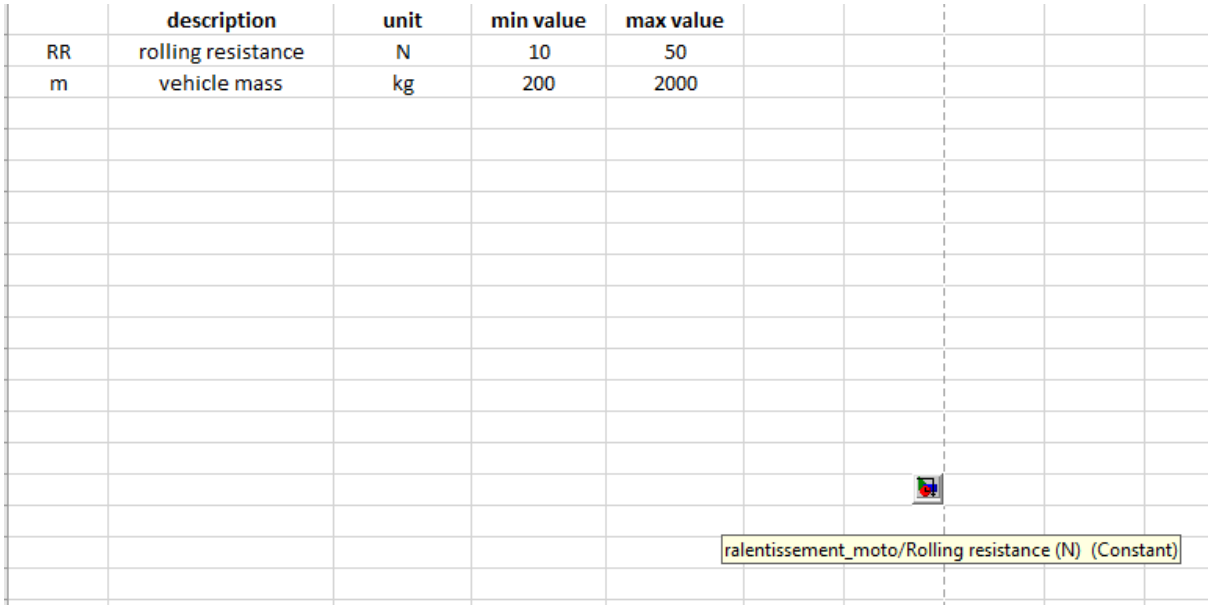
On ouvre ensuite un fichier Excel que l'on renseigne par exemple de la manière suivante :

	description	unit	min value	max value
RR	rolling resistance	N	10	50
m	vehicle mass	kg	200	2000

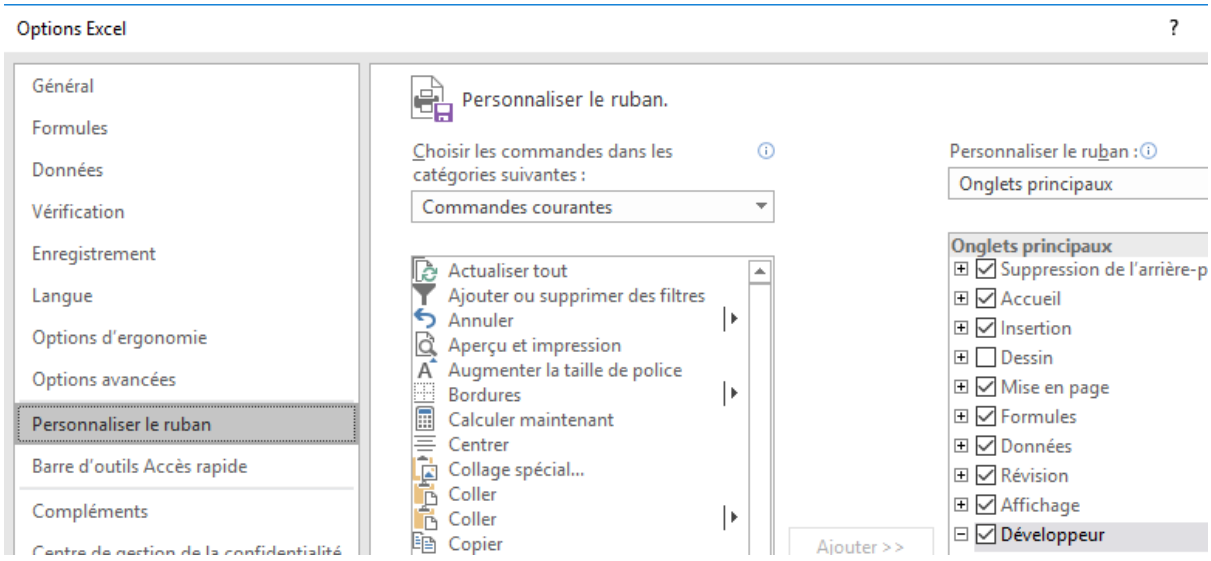
Dans le modèle on va ensuite lier le bloc résistance au roulement à son exigence => clic droit sur le bloc, puis :



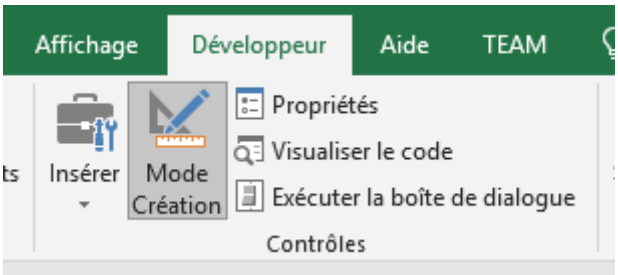
On sélectionne ensuite dans le fichier Excel les cellules concernées et … un contrôle Active-X est collé un peu en vrac :



Pour pouvoir déplacer ces contrôles, il faut ajouter le menu Développeur (qui n'est pas activé par défaut sauf chez les adeptes du VBA) dans Options Excel / Personnaliser le Ruban / cocher Développeur



… puis activer le Mode Création dans le menu qui est apparu :



(c'est un simple switch, la case doit être grisée pour être activée)

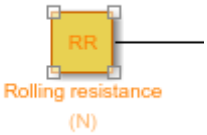
Après avoir créé un autre contrôle pour la masse et réagencé le tout on obtient donc :

	description	unit	min value	max value	
RR	rolling resistance	N	10	50	
m	vehicle mass	kg	200	2000	

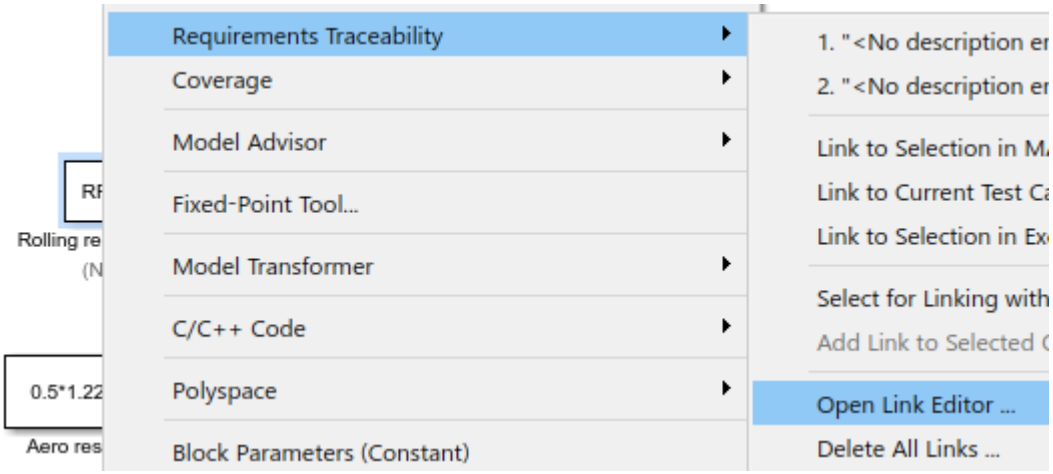
Si on double-clique sur l'un de ces contrôles on peut alors voir la fenêtre Matlab s'activer



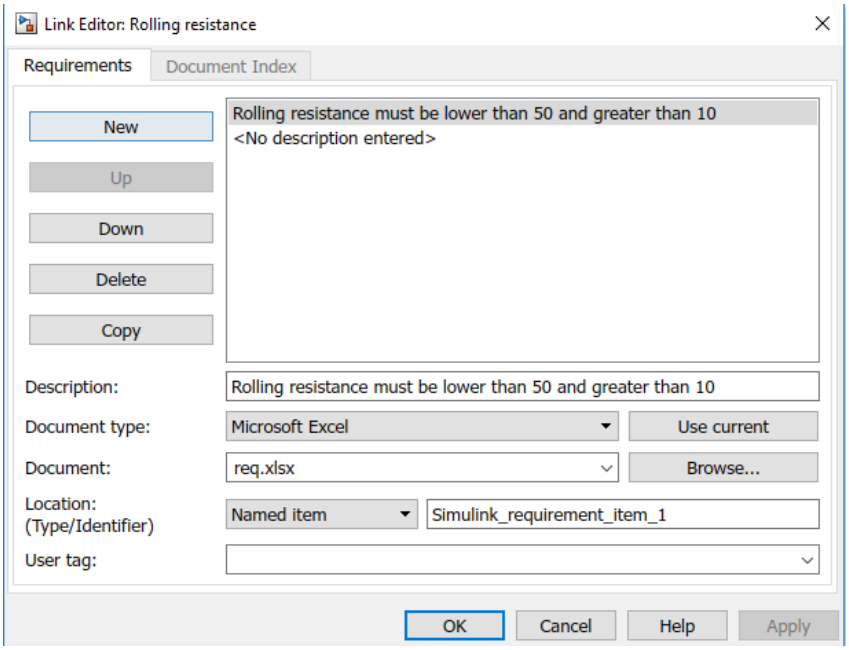
et le bloc correspondant y est mis en surbrillance



Dans le modèle un clic-droit sur un des blocs utilisés permet d'ouvrir le Link Editor :



On peut y modifier la description associée à chaque exigence :



15.1.6.2 EXEMPLE AVEC FICHIER WORD

Pour lier un bloc à un fichier "Word" contenant les spécifications du modèle, on procède de la manière suivante :

1) Ouvrir un document word

2) Selectionner dans ce document les mots désignant la fonction du bloc à spécifier

3) Retourner sur le modèle.

4) Cliquer droit sur le bloc à spécifier et aller dans "Requirements", puis "add link to Word selection"

Au final :

Un ActiveX est ajouté au document word.

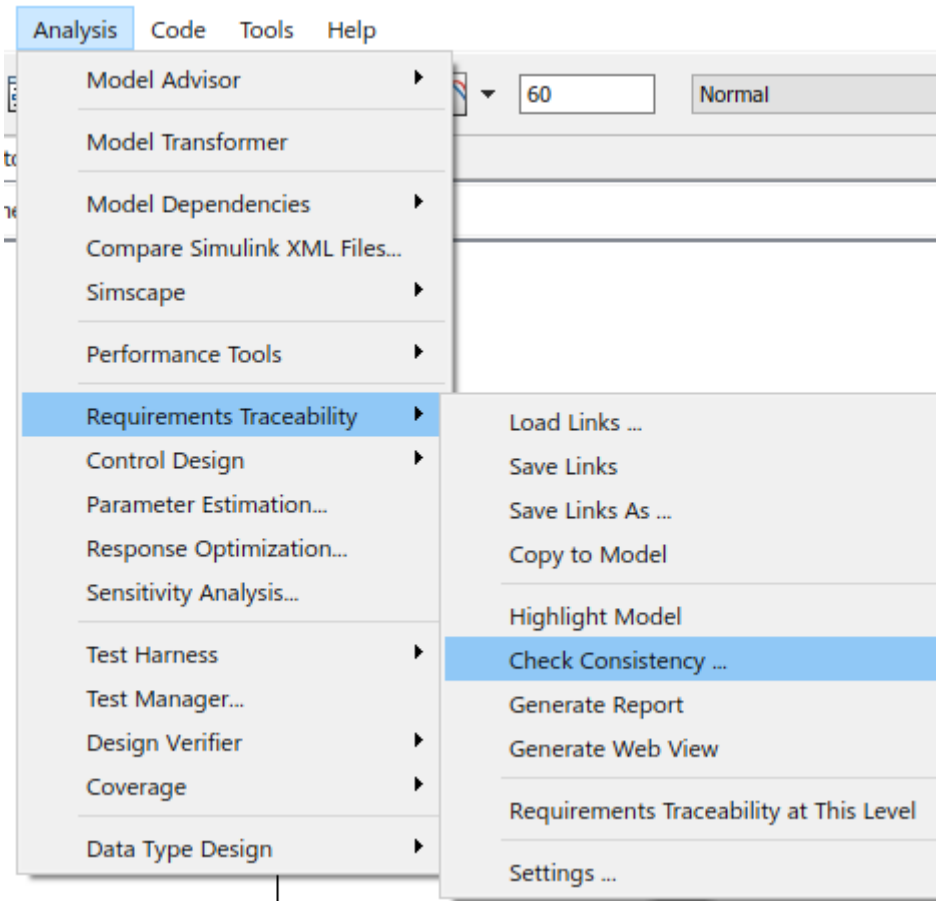
En double cliquant dessus, cela provoque la mise en surbrillance du bloc dans Simulink

De plus, à partir du bloc simulink en retournant dans "Requirements", puis "edit/Add links...",

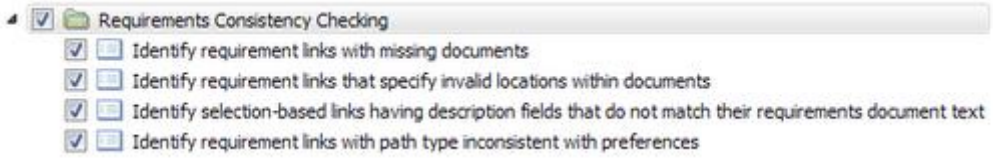
on retrouve les info sur le document Word contenant les "spec" et les mots correspondant à la "spec" énoncée.

On peut donc retrouver la "spec" à partir du bloc

15.1.7 CHECK CONSISTENCY



Cela permet de s'assurer de la cohérence entre le modèle et les exigences définies dans le document:



(Ne fonctionne que pour des exigences externes donc ?)

15.1.8 LIEN AVEC DES CAS-TESTS

https://fr.mathworks.com/help/slrequirements/ug/link-test-case-to-requirements-documents.html

15.1.9 EVOLUTION DES EXIGENCES

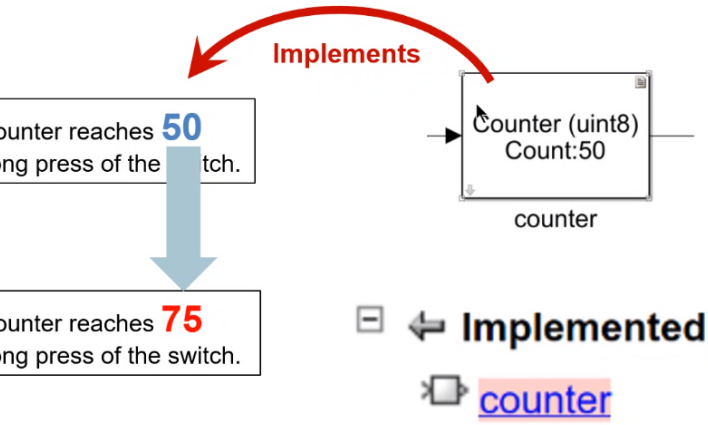
Respond to Change

Original Requirement

If the switch is pressed and the counter reaches **50** then it shall be recognized as a long press of the switch.

Updated Requirement

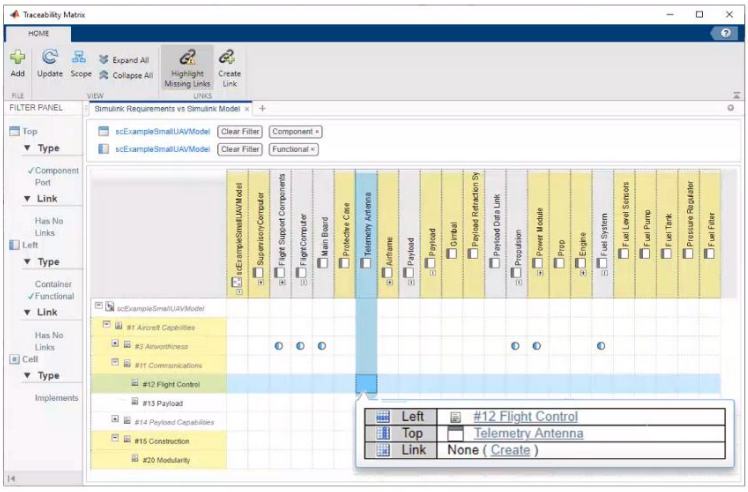
If the switch is pressed and the counter reaches **75** then it shall be recognized as a long press of the switch.



15.2 MATRICE DE TRAÇABILITE

Review and Analyze Traceability with Traceability Matrix R2020a

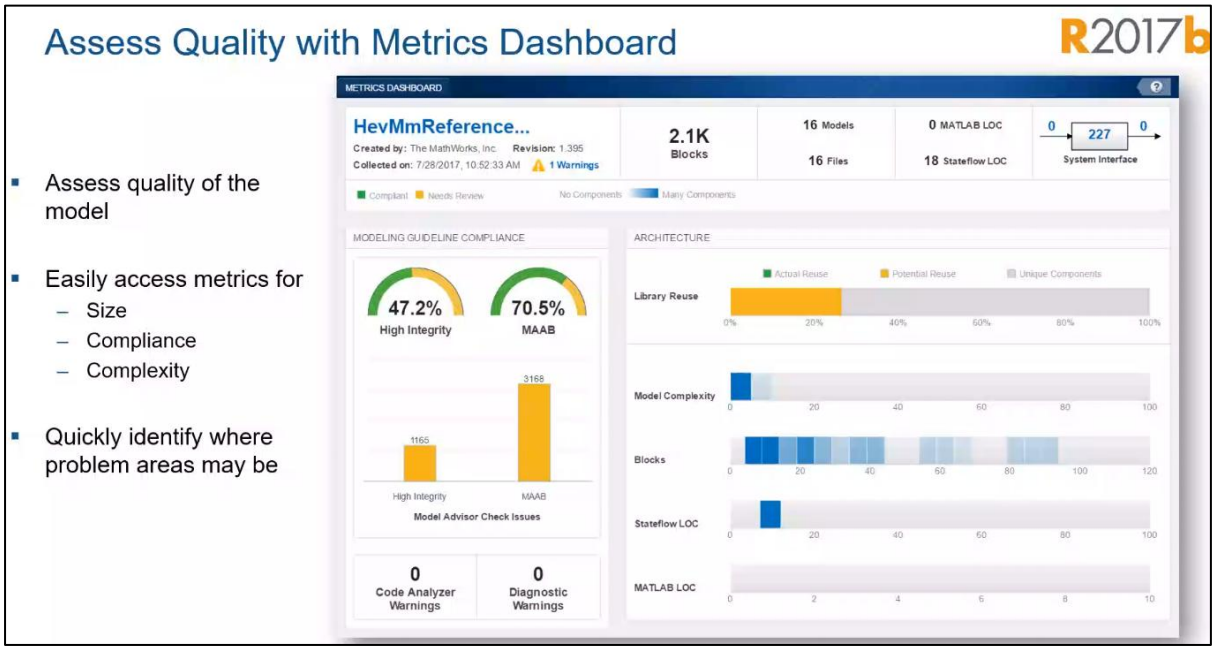
- Review links between different requirements, model, test
- Filter view to manage large sets of artifacts
- Highlight missing links
- Directly add links to address gaps



15.3 SIMULINK CHECK

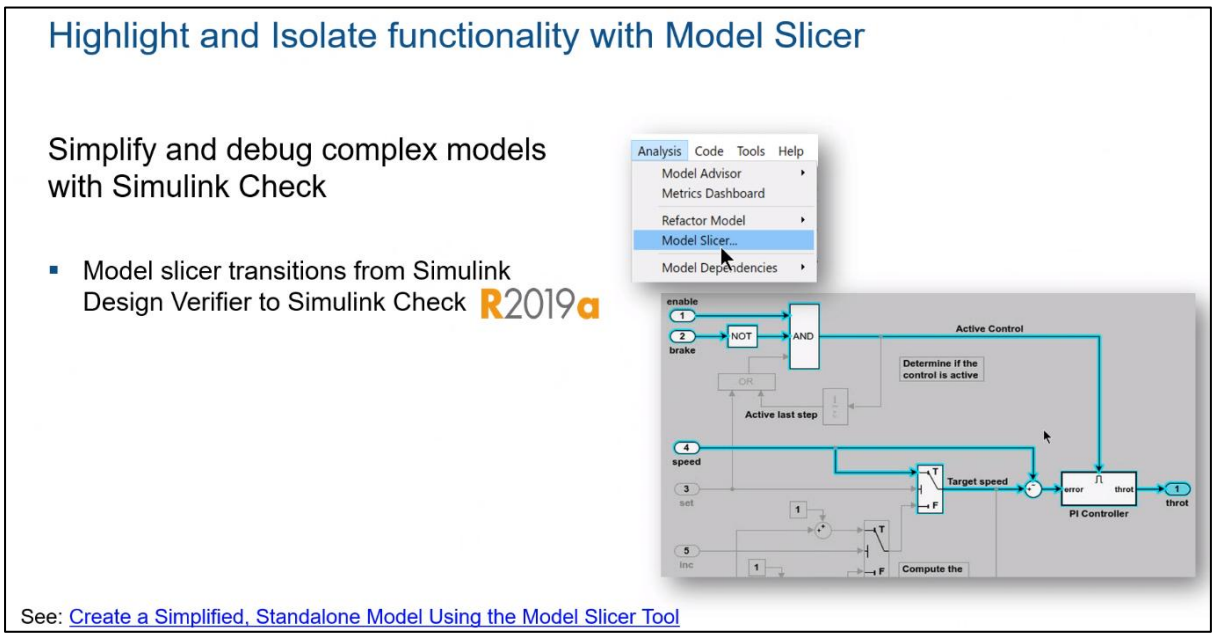
Simulink est une toolbox, plus complète et plus puissante que ce que propose le Model Advisor qui est lui disponible en « pur Simulink ».

15.3.1 METRICS

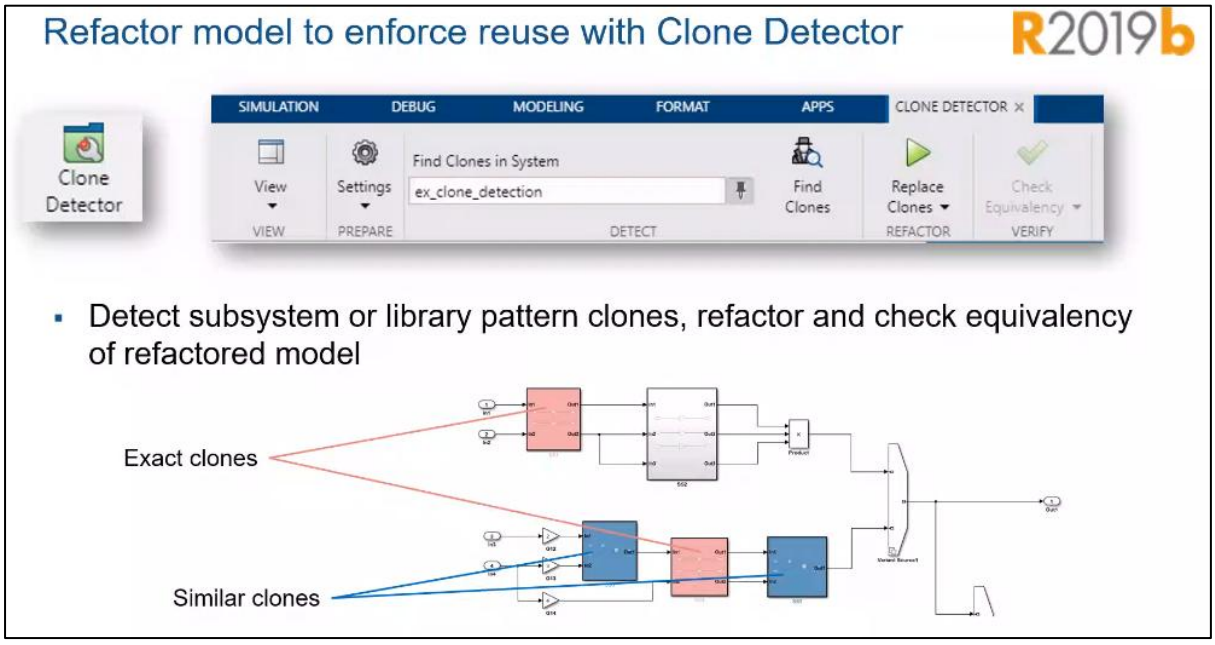


15.3.2 MODEL SLICER

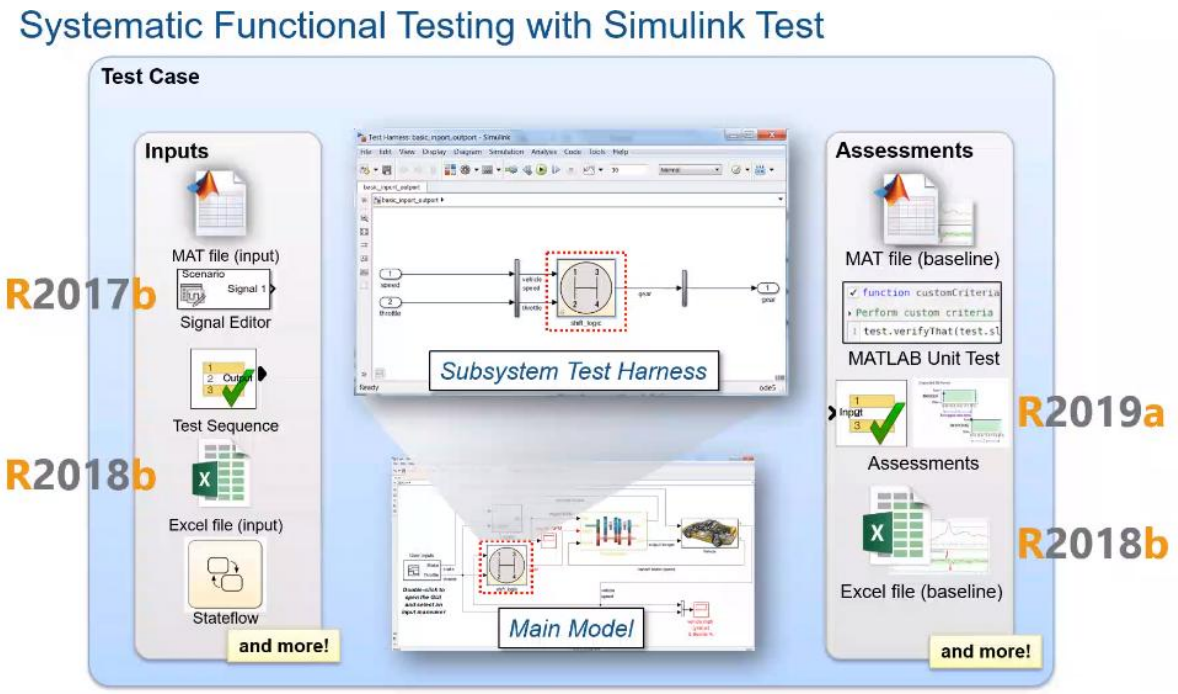
Sélectionner l'output qui nous intéresse, puis Analysis / Model Slicer



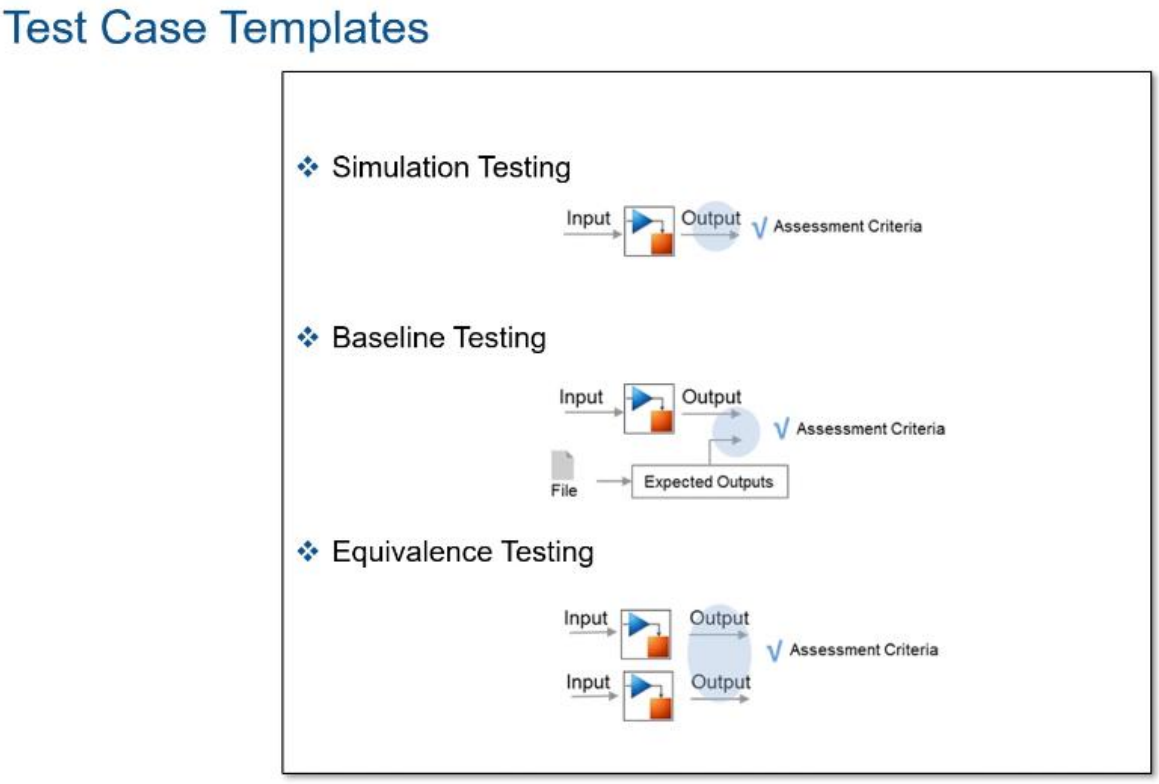
15.3.3 CLONE DETECTOR

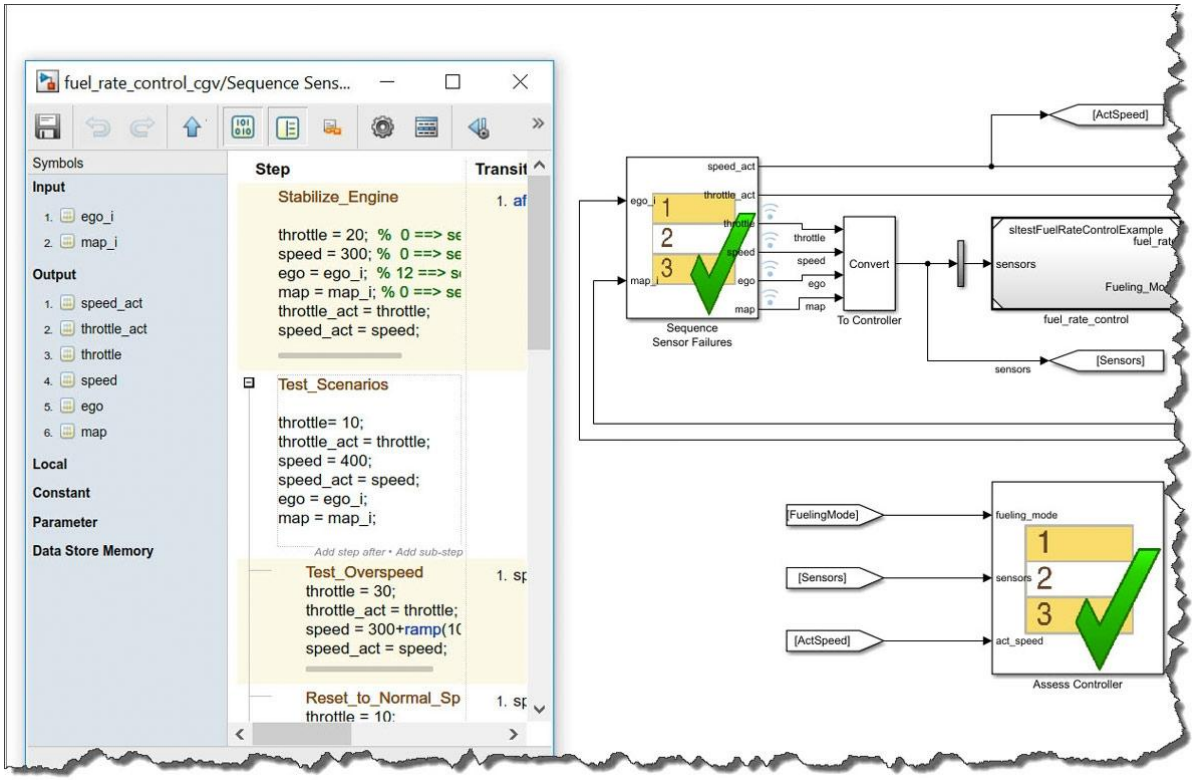


15.4 SIMULINK TEST

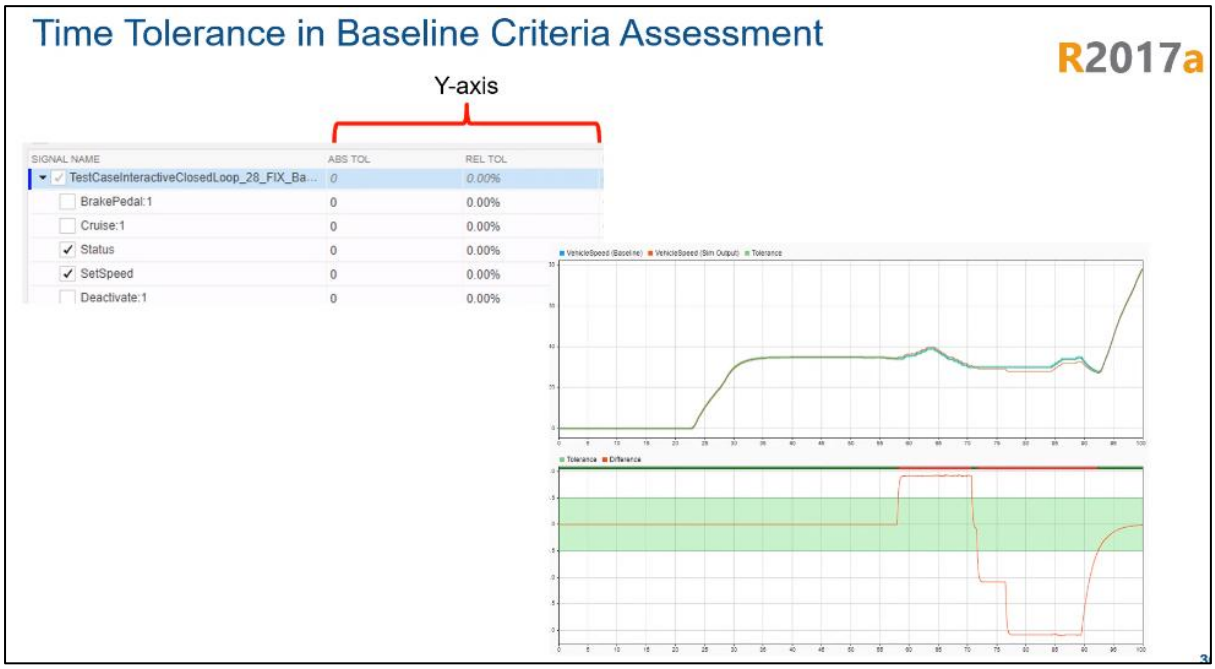


15.4.1 SIMULATION / BASELINE / EQUIVALENCE TESTING

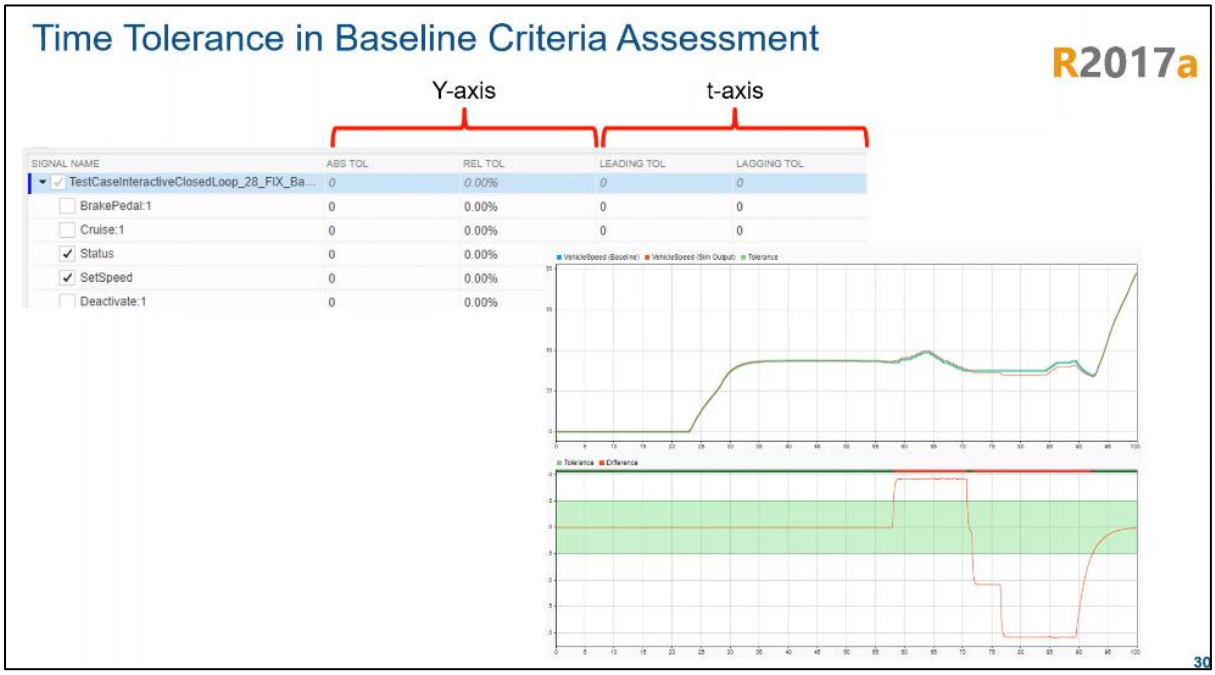




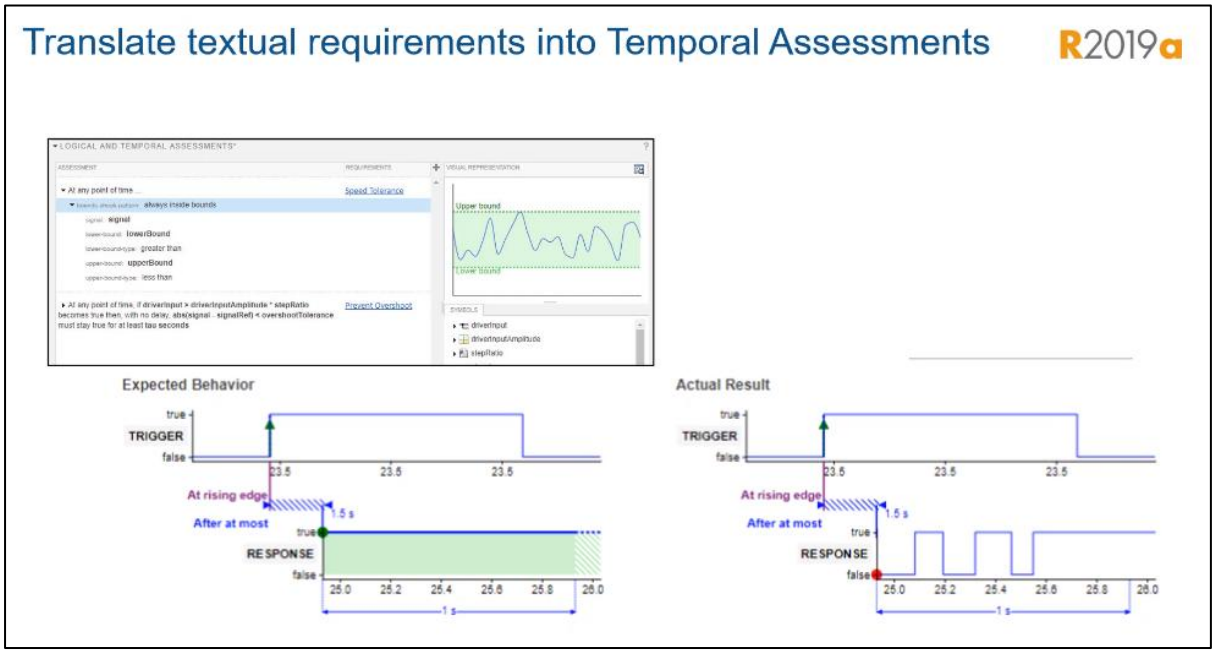
Jusqu'en 2017 on pouvait définir une tolérance sur la réponse



Ensuite a été implémentée la possibilité de définir des critères sur la réponse temporelle (retard, etc …)

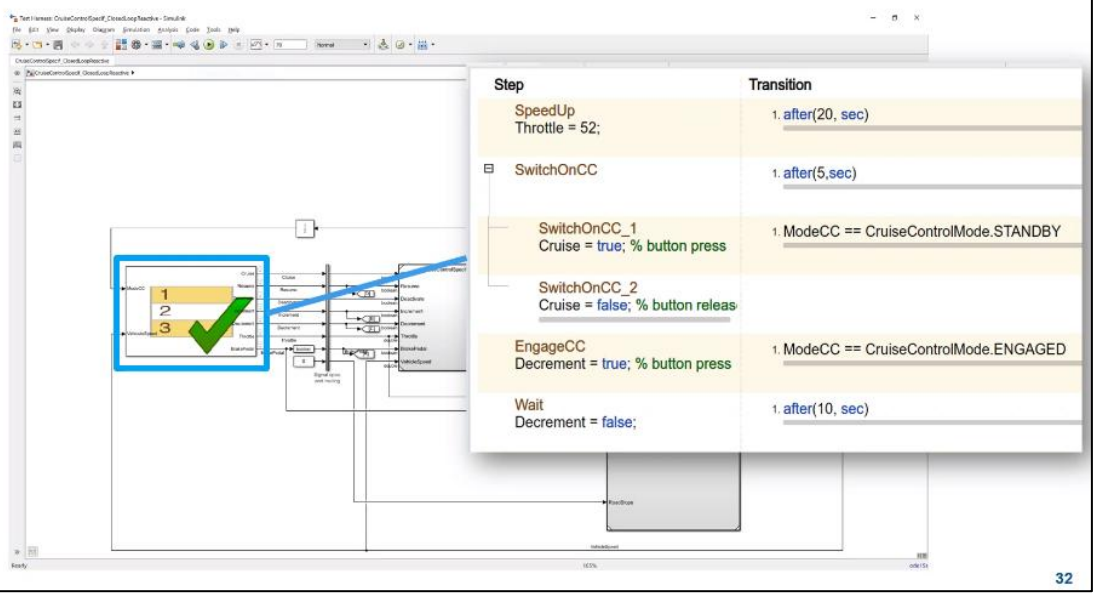


Possibilité de définir des exigences sous forme textuelle



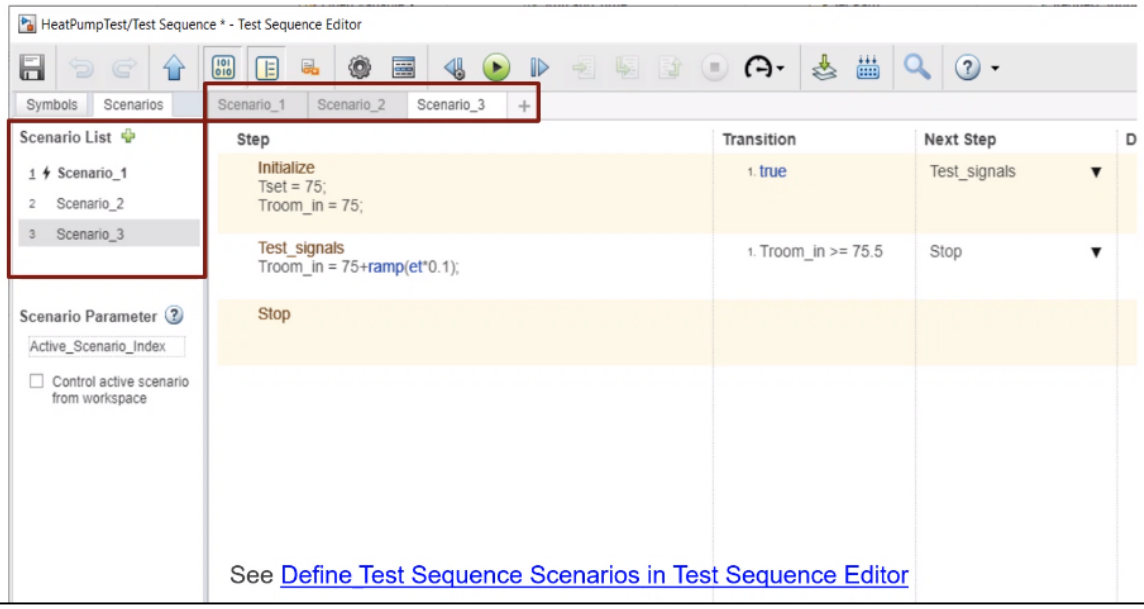
Langage dédié (inspiré de Stateflow)

Test Sequence Block for Easy Test Definition



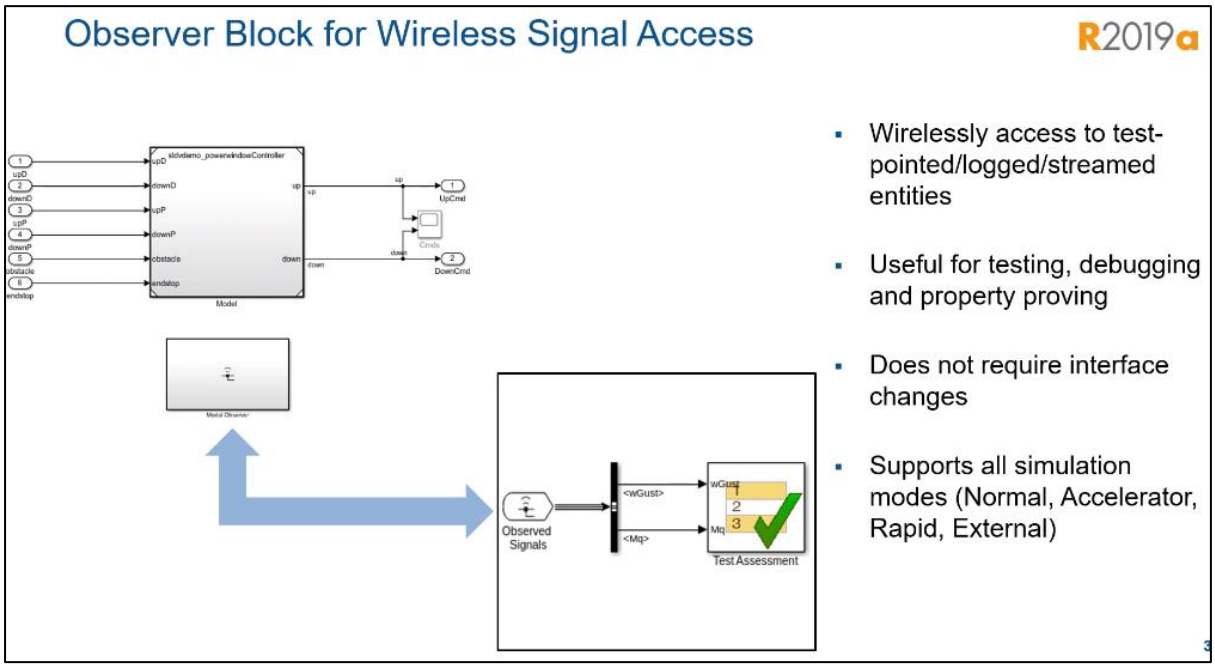
32

Test Sequence block now supports multiple scenarios R2020b

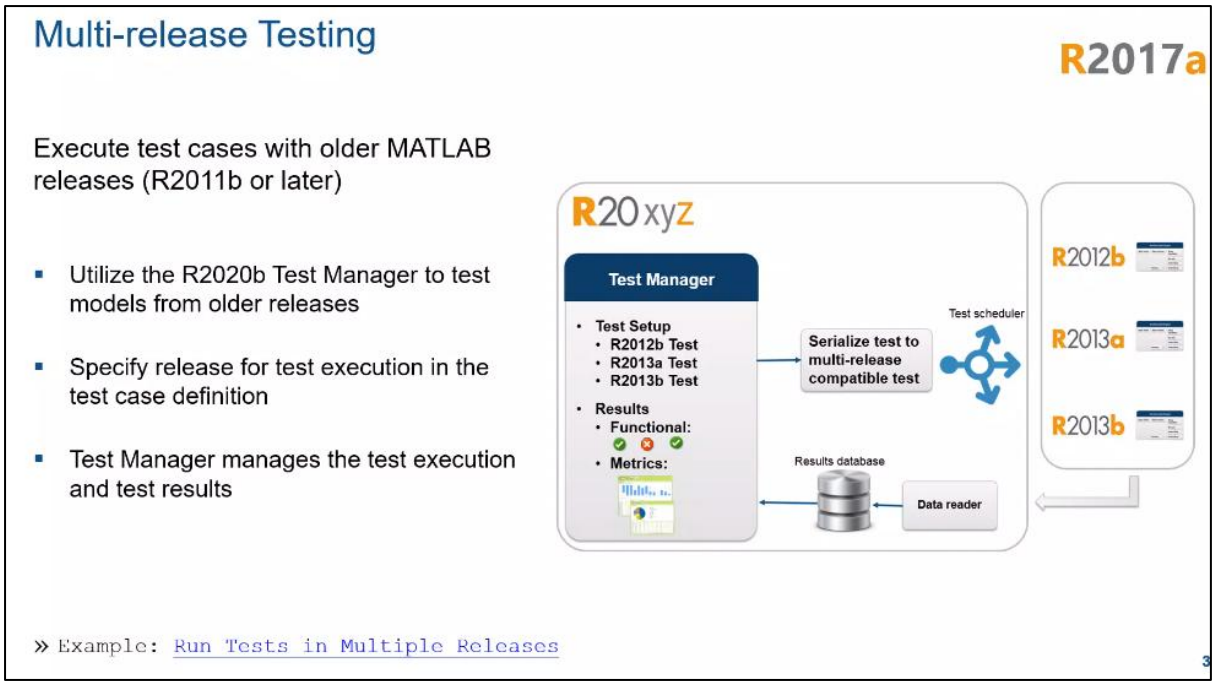


See Define Test Sequence Scenarios in Test Sequence Editor

Le bloc Model Observer permet de définir des « observateurs » sur un sous-système sans cablage additionel



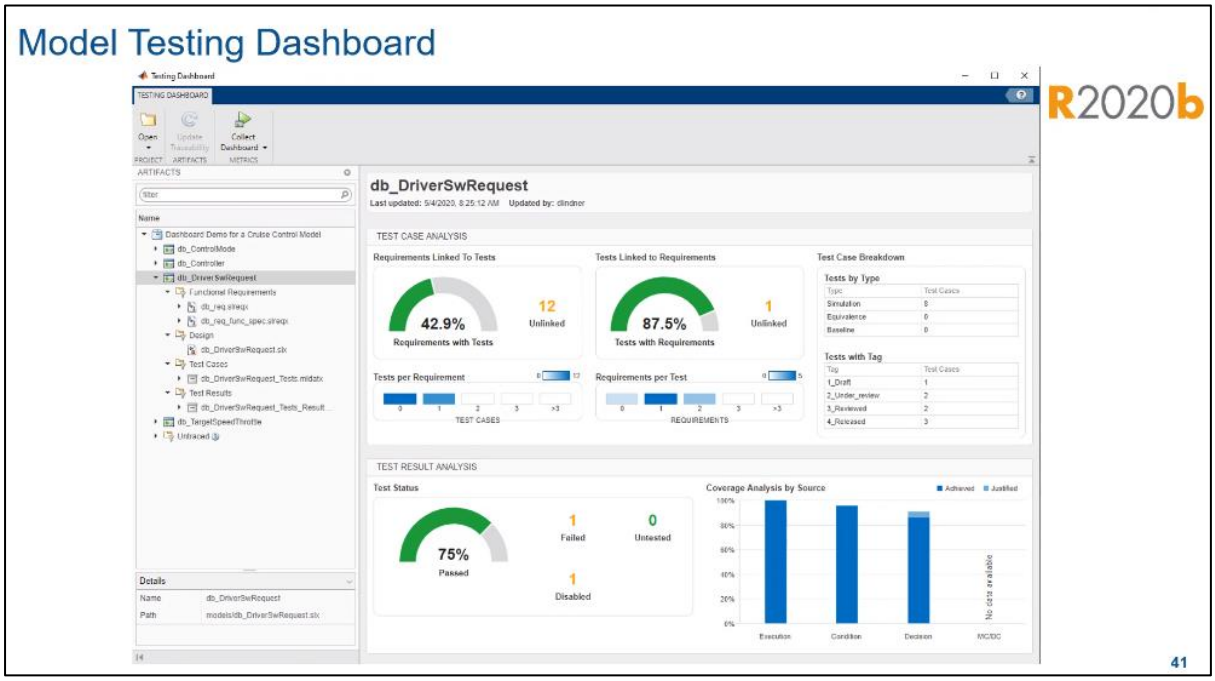
Test multi-versions



Simulink Projects simplifie l'exécution des tests

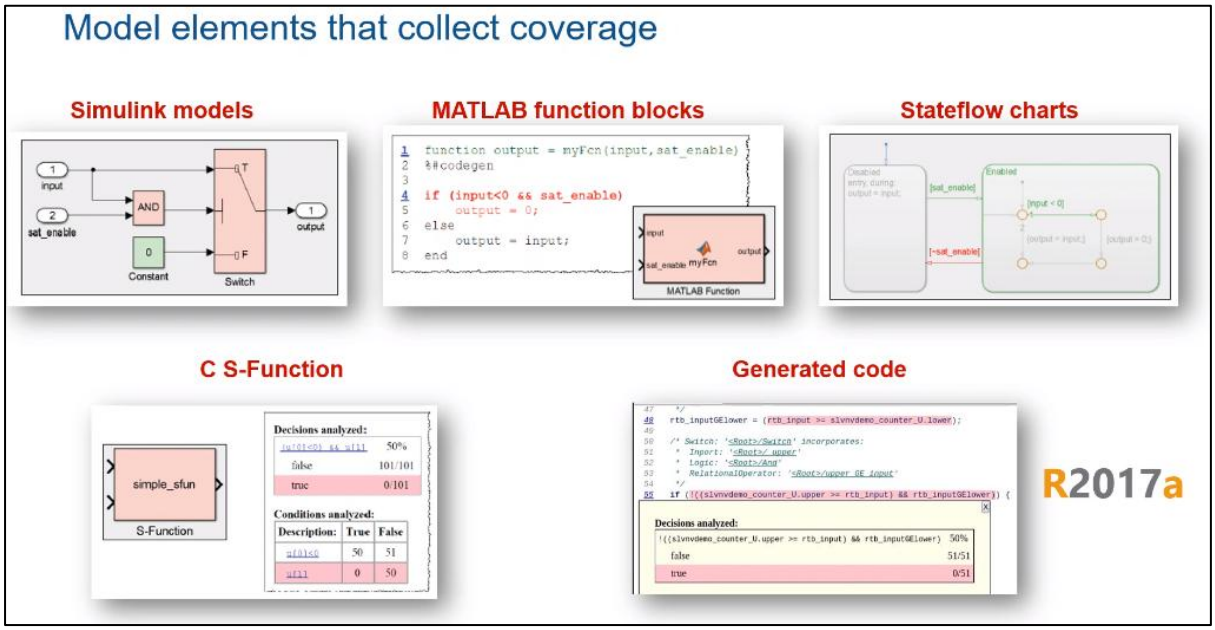


Testing dashboard

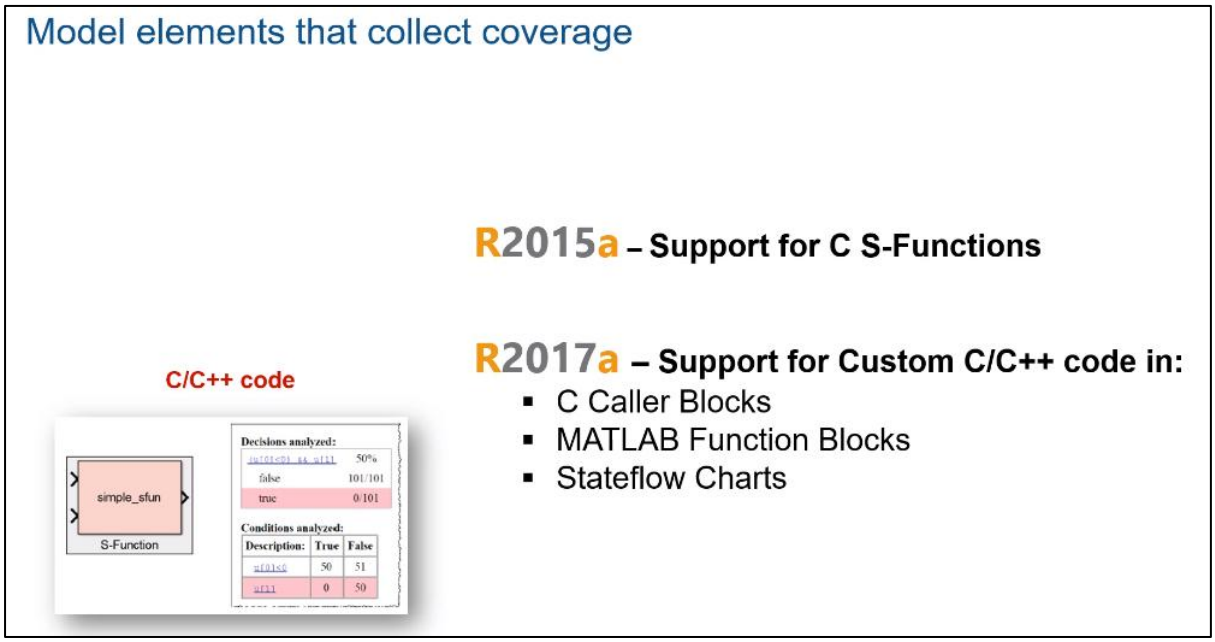


15.5 COVERAGE

Depuis la 2017a, on peut « collecter » la couverture avec le generated code. Il faut tourner en SIL mode

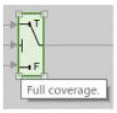


Egalement possible avec du code C/C++

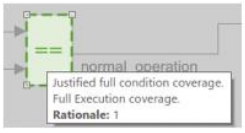


Enhanced coverage highlighting

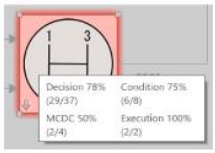
R2018b



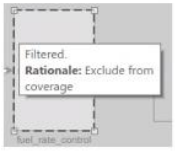
Green
Full Coverage



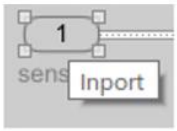
Green with dashed border
Justified Coverage



Red
Partial Coverage



Gray with dashed border
Filtered Coverage

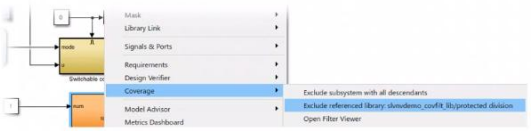


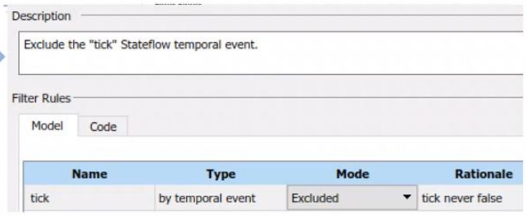
No coloring
Coverage not recorded

Combine Multiple Coverage Filters

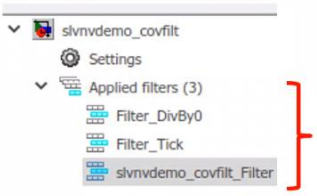
R2020a

- Create filter to exclude some blocks from coverage collection





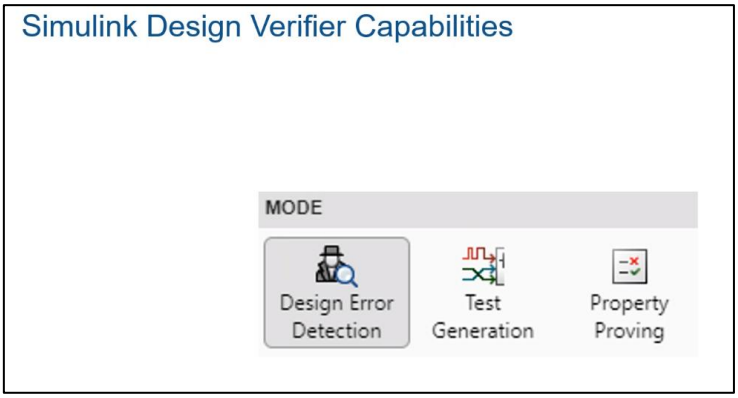
- New : Ability to combine multiple filters

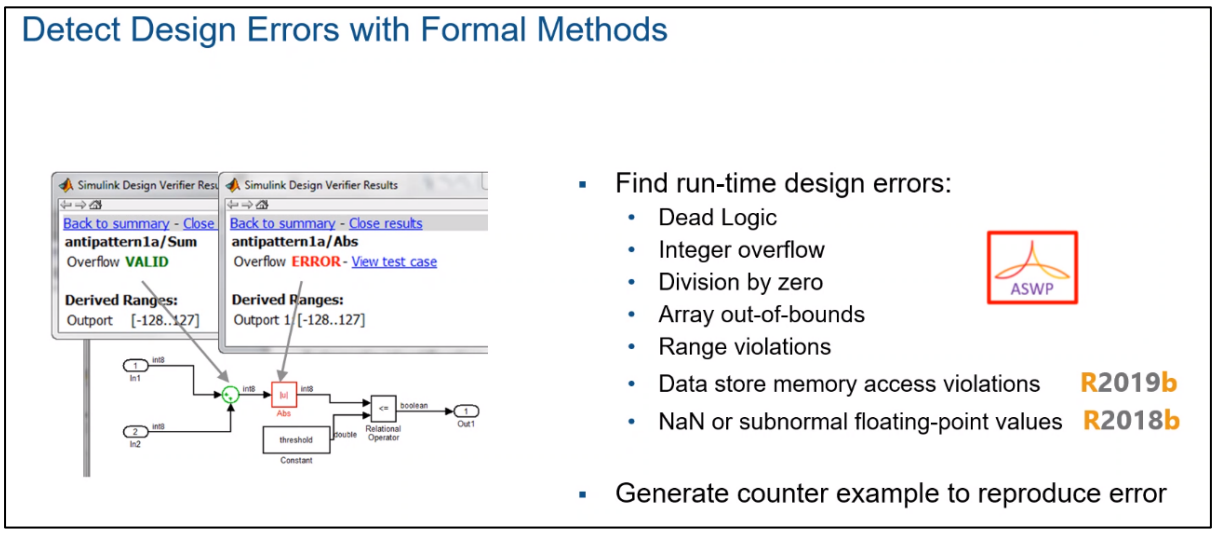


46

15.6 DESIGN VERIFIER

https://fr.mathworks.com/company/newsletters/articles/verifying-models-and-code-for-high-integrity-systems.html



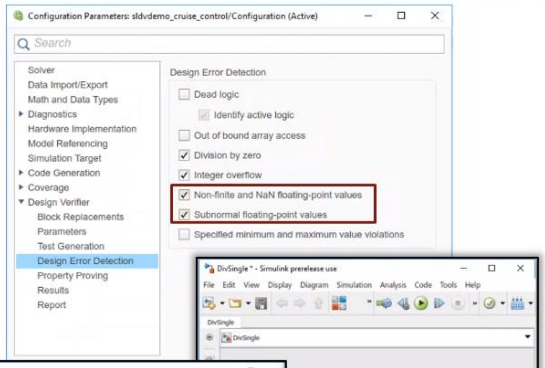


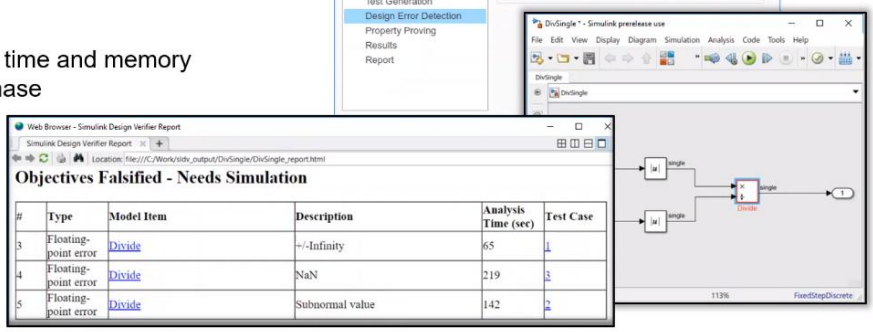
Floating point Design Errors

R2018b

Detect computation of +/-Infinity, NaN and subnormal values

- New design error checks for floating point computations
- Analysis may take more time and memory due to the refinement phase

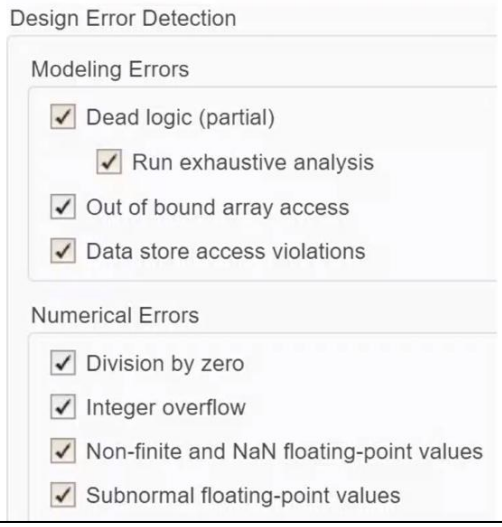




subnormal = close to 0

Run Dead Logic Detection in Conjuntion with Other Checks

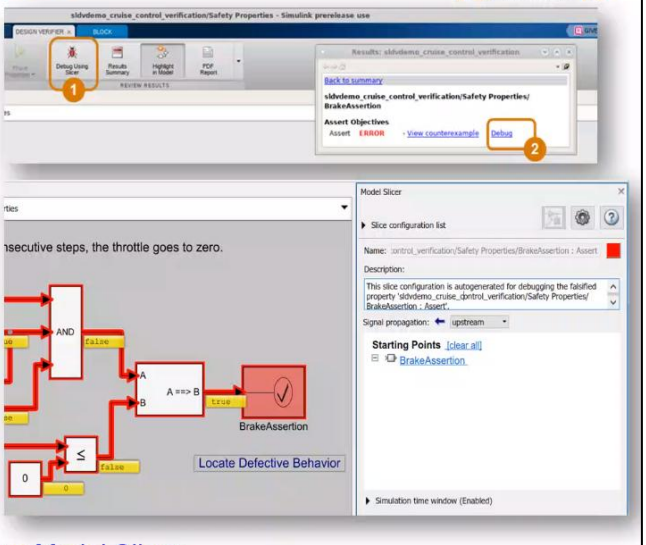
R2019b



Debugging Property Proving Violations

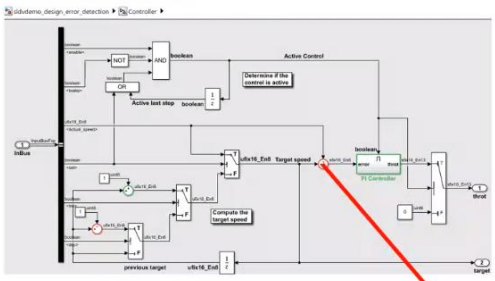
R2019b

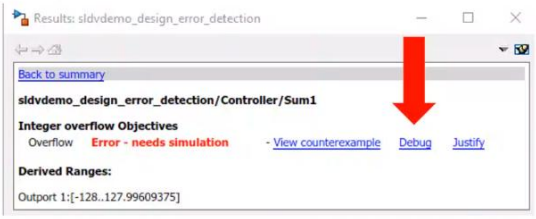
Launch Model Slicer to debug property proving violations from Simulink Design Verifier

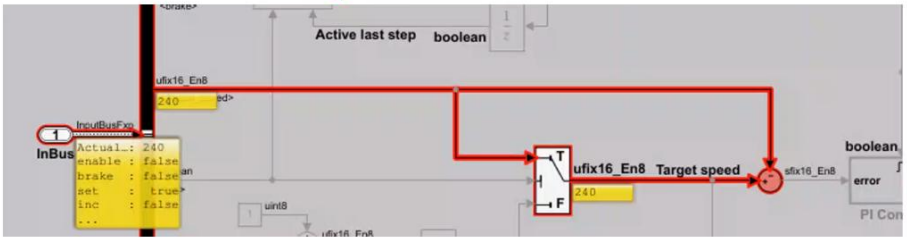


See : [Debug Property Proving Violations by Using Model Slicer]

Debug counterexample from design error detection with Model Slicer

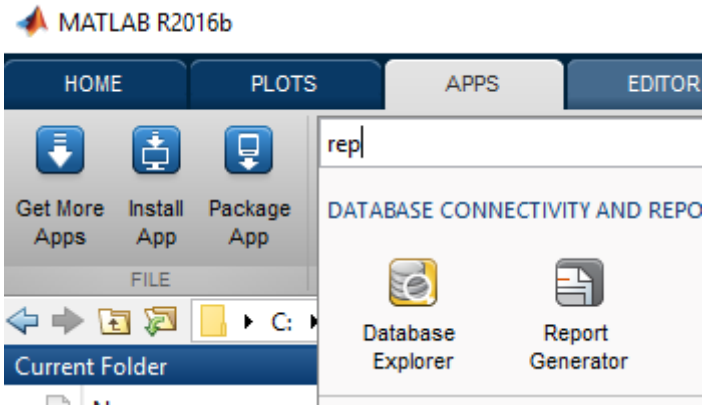




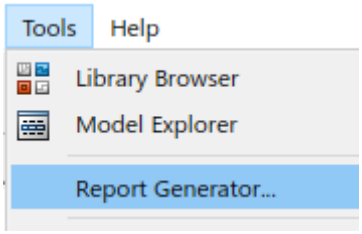


16 REPORT GENERATOR

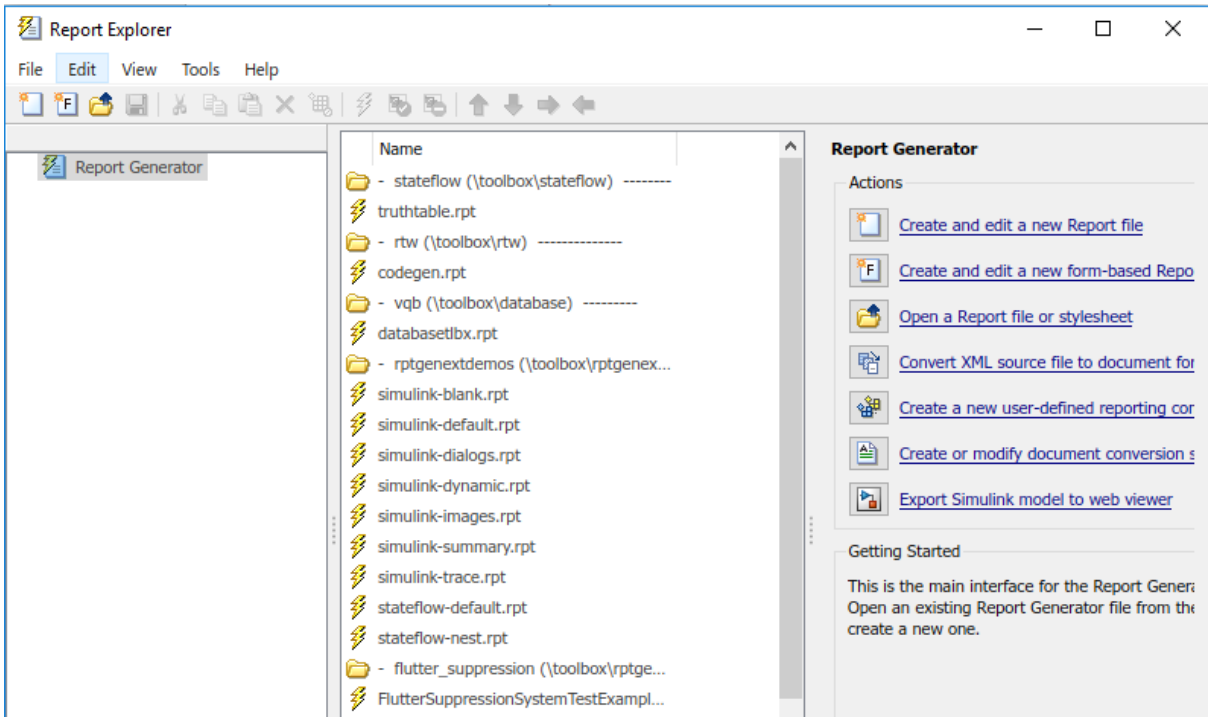
Accessible soit par le menu Apps



Soit une fois le modèle ouvert dans le menu Tools

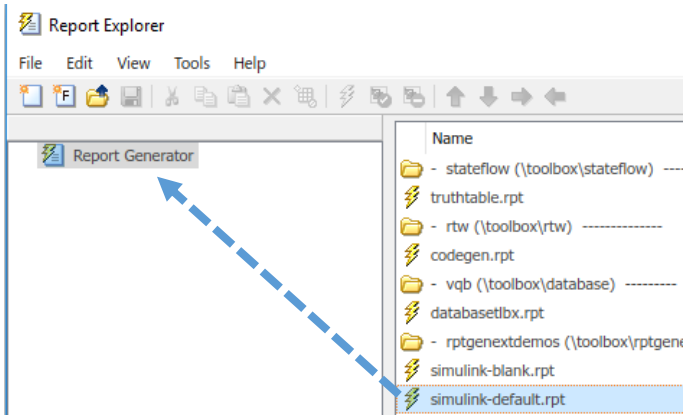


L'interface propose un espace "de travail" sur la gauche, et une bibliothèque de rapports pré-existants dans la partie centrale

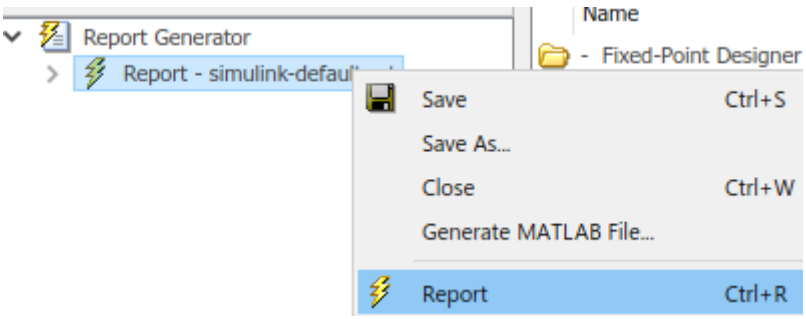


16.1 UTILISATION D'UN TEMPLATE EXISTANT

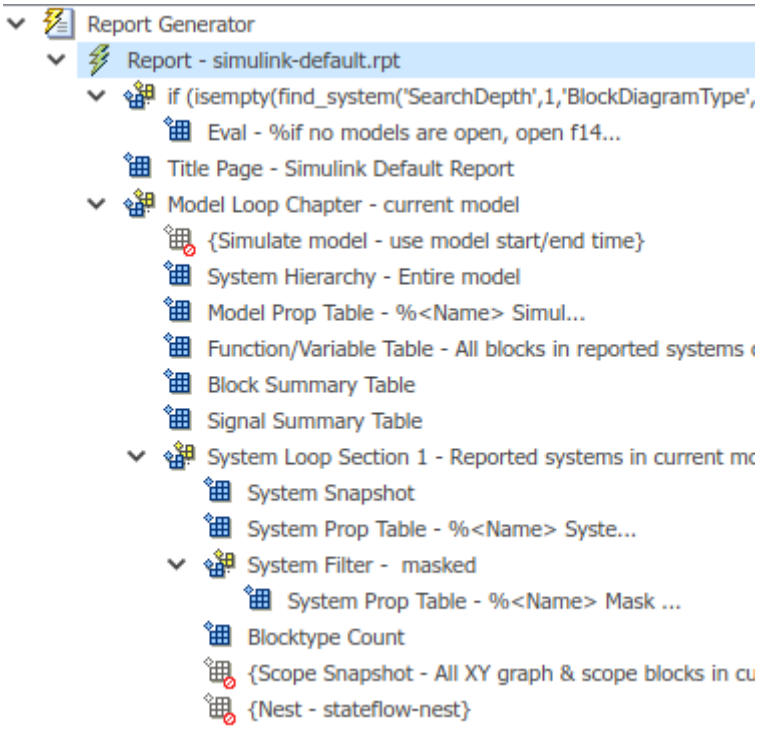
On peut faire glisser un .rpt (template) existant dans la colonne gauche (ça marche aussi par un double-clic si on est pressé)



puis Ctrl-R pour générer le rapport correspondant et visualiser rapidement un exemple

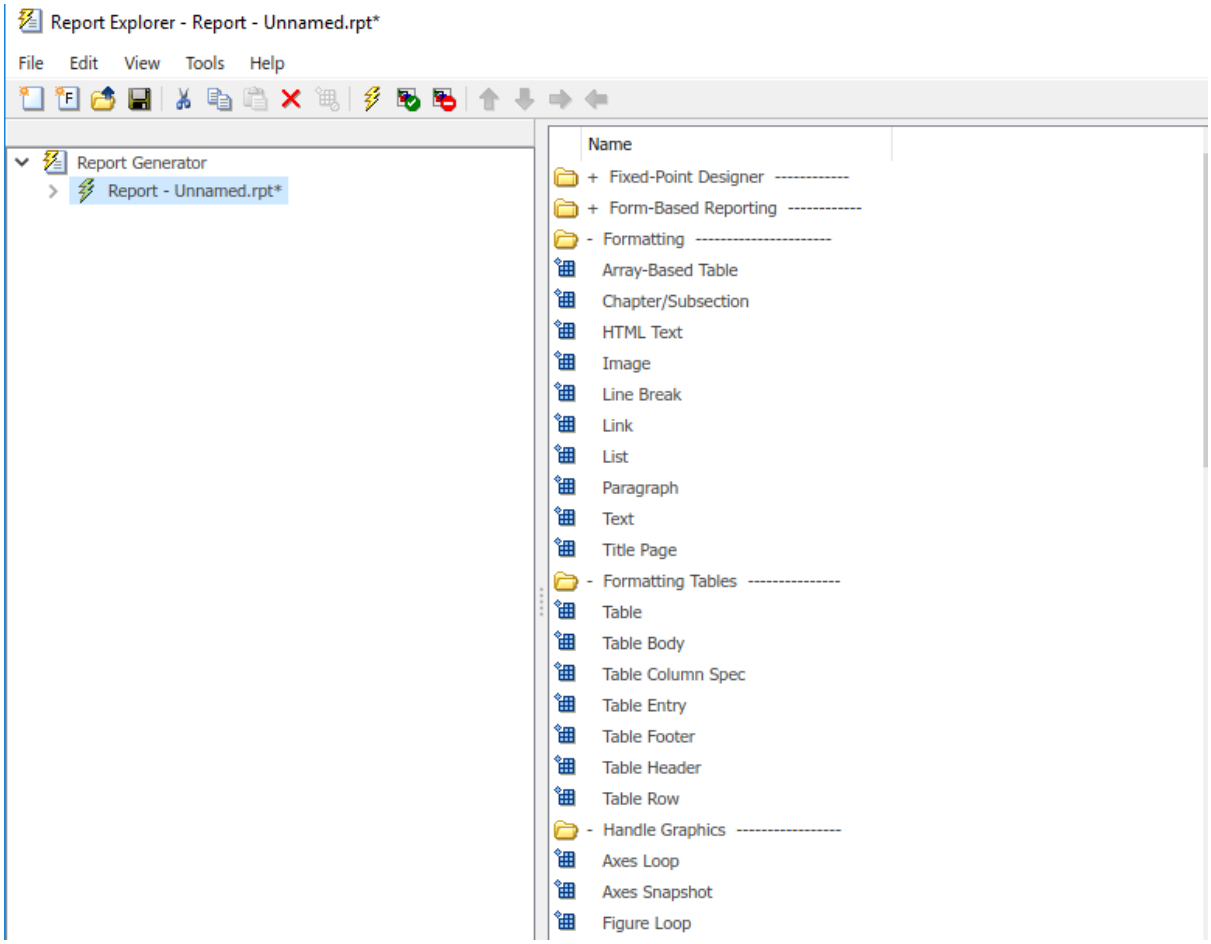


En déroulant on peut en voir le contenu

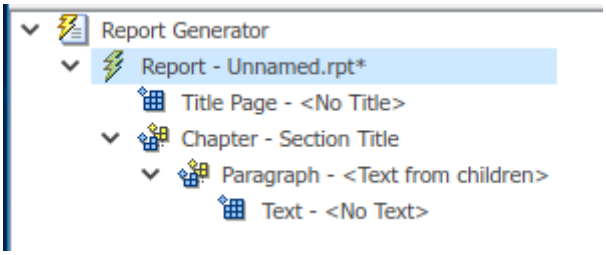


16.2 CREATION D'UN TEMPLATE FROM SCRATCH

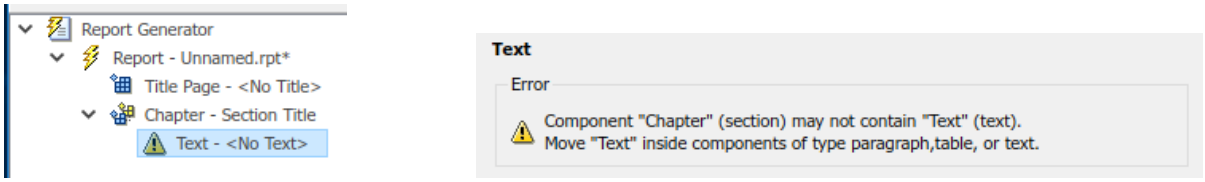
En repartant de zéro (raccourcis classiques : Ctrl+W pour fermer un rapport existant, Ctrl+N pour en créer un nouveau …), on accède dans la partie centrale à l'ensemble des composants qui peuvent être utilisés :



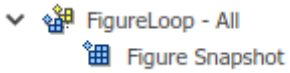
Le plus important est de comprendre que les rapports doivent être construits en respectant une arborescence hiérarchique : un chapitre contient un paragraphe qui contient du texte



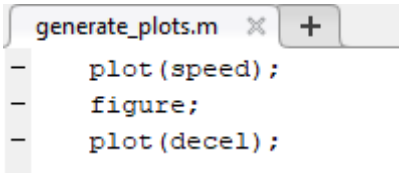
Si on met directement du texte dans un chapitre un message nous l'explique gentiment :



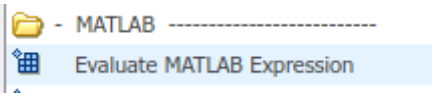
L'autre point à comprendre est la notion de "loops" : on peut inclure une boucle sur toutes les figures (attention, par défaut elle est paramétrée sur "Current figure only"), et à chaque élément de la boucle inclure un snapshot :



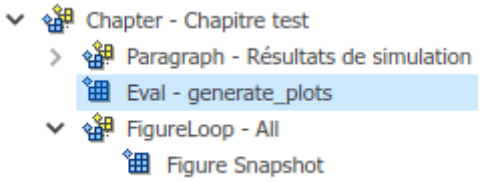
Imaginons que la simu ait permis de créer deux variables speed et decel dans le workspace, que l'on affiche en temps normal par un script



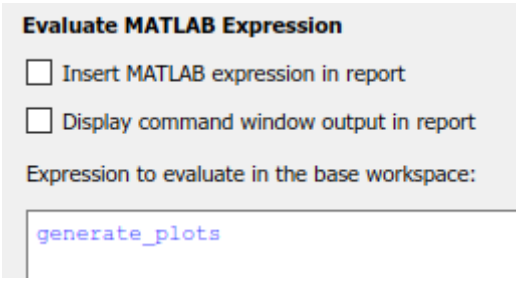
Ce script peut être appelé lors de la génération du rapport grâce au composant suivant :

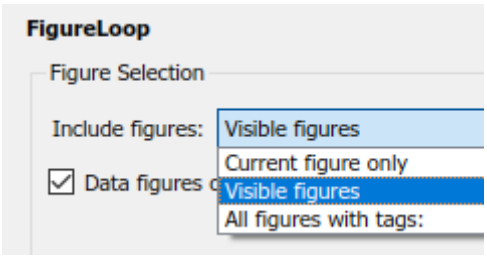


On utilise alors la construction suivante : appel du script puis FigureLoop



Avec les paramétrages suivants de chaque composant :

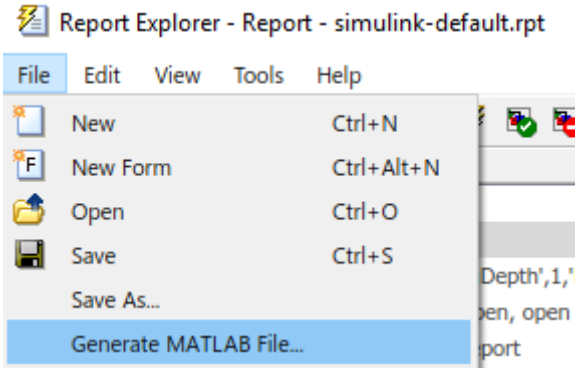




16.3 FONCTIONNALITES UTILES

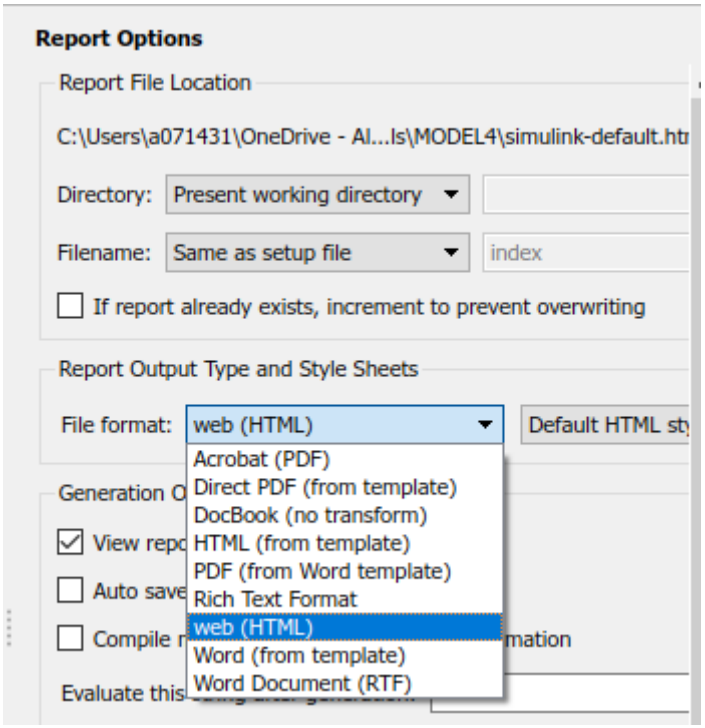
16.3.1 GENERATION DE .M

Intéressant : on peut générer un .m pour pouvoir ensuite scripter plus finement si besoin (et surtout pour connaître l'équivalent "programmatique" de ce qu'on a fait grâce à l'IHM)



16.3.2 FORMATS DE SORTIE

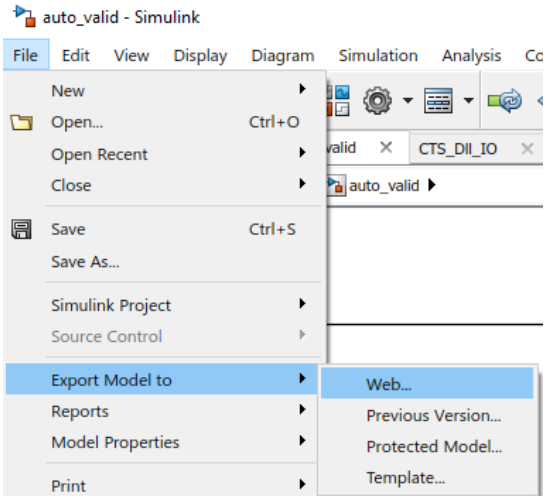
On peut générer des rapports sous différents formats dans la section Report Options:



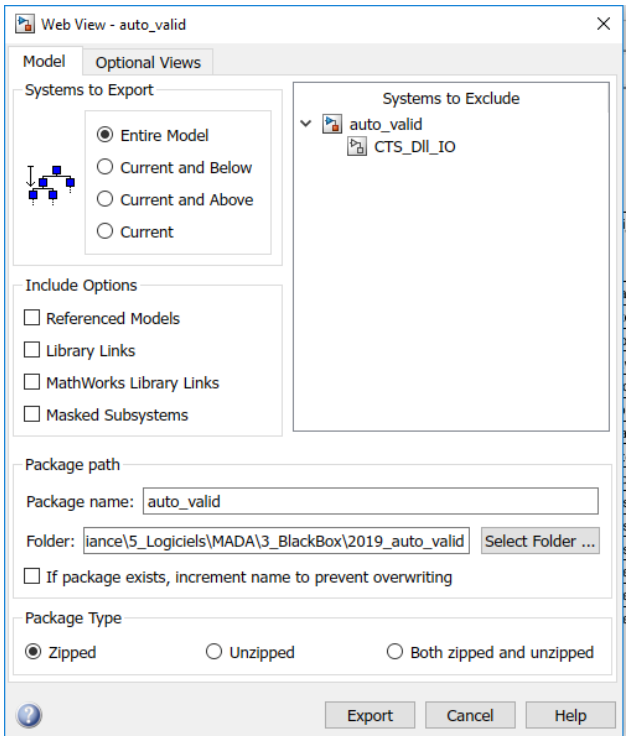
16.3.3 EXPORT MODEL TO / WEB

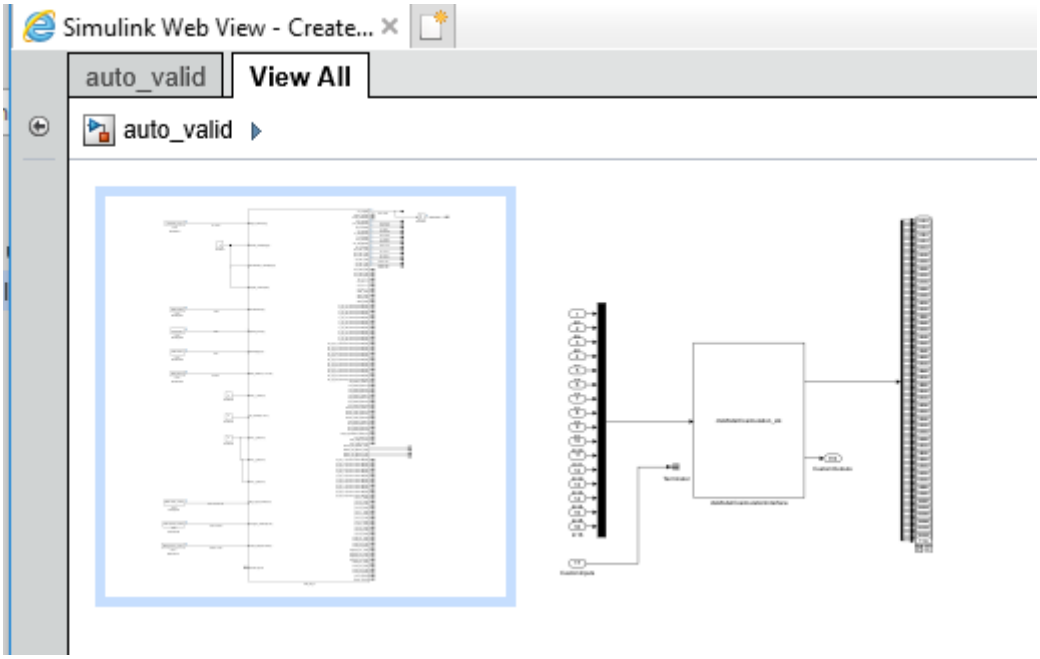
(depuis Simulink, mais également accessible dans le Report Generator / menu Tools)

Permet de générer un fichier html navigable



On peut n'exporter qu'une partie d'un modèle, en exclure certains blocs …





Cela permet de "partager" le contenu d'un modèle (par exemple avec un fournisseur) sans le fournir réellement.

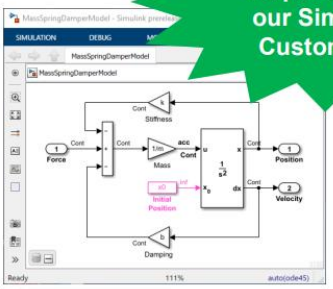
Share Simulink simulations

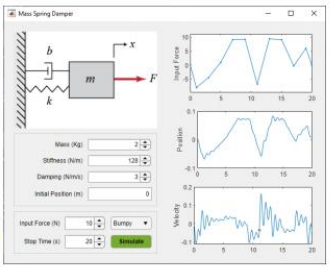
MATLAB WebApp Server supports Simulink

Benefits:

- End users do not require a Simulink license
- End users can use a web app to choose tunable parameters and run the simulation from a browser
  - MATLAB Apps that call sim() can be deployed as web apps
- Simulink Compiler can generate a "Starter" app to get you started

A long-standing request from our Simulink Customers!





R2020a

16

18 SIMULINK ET GIT/GITLAB

18.1 CONTINUOUS INTEGRATION (CI)

- https://fr.mathworks.com/company/newsletters/articles/continuous-integration-for-verification-of-simulink-models.html
- https://fr.mathworks.com/company/newsletters/articles/continuous-integration-for-verification-of-simulink-models-using-gitlab.html

19 BLOCKSETS ET TOOLBOXES

19.1 VEHICLE DYNAMICS BLOCKSET

19.1.1 INTRO

Introduit en R2018a

https://fr.mathworks.com/help/vdynblks/release-notes.html

- Ajout du constant radius test en 2019a

Il y a des blocks pour le chassis seul :

Vehicle Body Total Road Load	Vehicle motion using coast-down testing coefficients
Vehicle Body 1DOF Longitudinal	Two-axle vehicle in forward and reverse motion
Vehicle Body 3DOF Longitudinal	3DOF rigid vehicle body to calculate longitudinal, vertical, and pitch motion
Vehicle Body 3DOF	3DOF rigid vehicle body to calculate longitudinal, lateral, and yaw motion
Vehicle Body 6DOF	Two-axle vehicle body with translational and rotational motion
Vehicle Body 3DOF Three Axles	Three-axle vehicle body with longitudinal, lateral, and yaw motion
Vehicle Body 6DOF Three Axles	Three-axle vehicle tractor body with translational and rotational motion
Trailer Body 3DOF	Trailer body with longitudinal, lateral, and yaw motion
Trailer Body 6DOF	Trailer body with translational and rotational motion

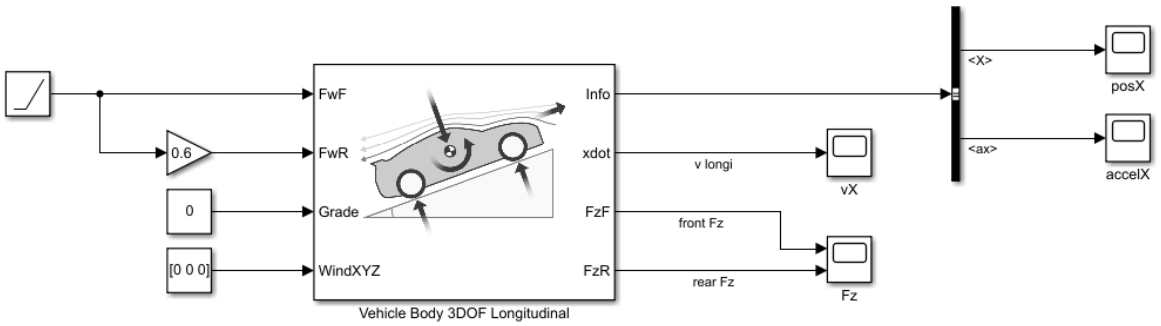
2020b

et des modèles de véhicule complet : 3dof / 7 dof / 14 dof :

<table>
<tr><th>Vehicle Model</th><th>Description</th><th>Vehicle Body Degrees-of-Freedom (DOFs)</th><th>Wheel DOFs</th></tr>
<tr><td>Passenger 14DOF Vehicle</td><td>• Vehicle with four wheels
• Available as model variant in the maneuver reference applications</td><td>Six
<table><tr><th>Translational</th><th></th><th>Rotational</th><th></th></tr><tr><td>Longitudinal</td><td>✓</td><td>Pitch</td><td>✓</td></tr><tr><td>Lateral</td><td>✓</td><td>Yaw</td><td>✓</td></tr><tr><td>Vertical</td><td>✓</td><td>Roll</td><td>✓</td></tr></table></td><td>Two per wheel - eight total
<table><tr><th>Translational</th><th></th><th>Rotational</th><th></th></tr><tr><td>Vertical</td><td>✓</td><td>Rolling</td><td>✓</td></tr></table></td></tr>
<tr><td>Passenger 7DOF Vehicle</td><td>• Vehicle with four wheels
• Available as model variant in the maneuver reference applications</td><td>Three
<table><tr><th>Translational</th><th></th><th>Rotational</th><th></th></tr><tr><td>Longitudinal</td><td>✓</td><td>Pitch</td><td></td></tr><tr><td>Lateral</td><td>✓</td><td>Yaw</td><td>✓</td></tr><tr><td>Vertical</td><td></td><td>Roll</td><td></td></tr></table></td><td>One per wheel - four total
<table><tr><th>Rotational</th><th></th></tr><tr><td>Rolling</td><td>✓</td></tr></table></td></tr>
<tr><td>Passenger 3DOF Vehicle</td><td>• Vehicle with ideal tire</td><td>Three
<table><tr><th>Translational</th><th></th><th>Rotational</th><th></th></tr><tr><td>Longitudinal</td><td>✓</td><td>Pitch</td><td></td></tr><tr><td>Lateral</td><td>✓</td><td>Yaw</td><td>✓</td></tr><tr><td>Vertical</td><td></td><td>Roll</td><td></td></tr></table></td><td>None</td></tr>
</table>

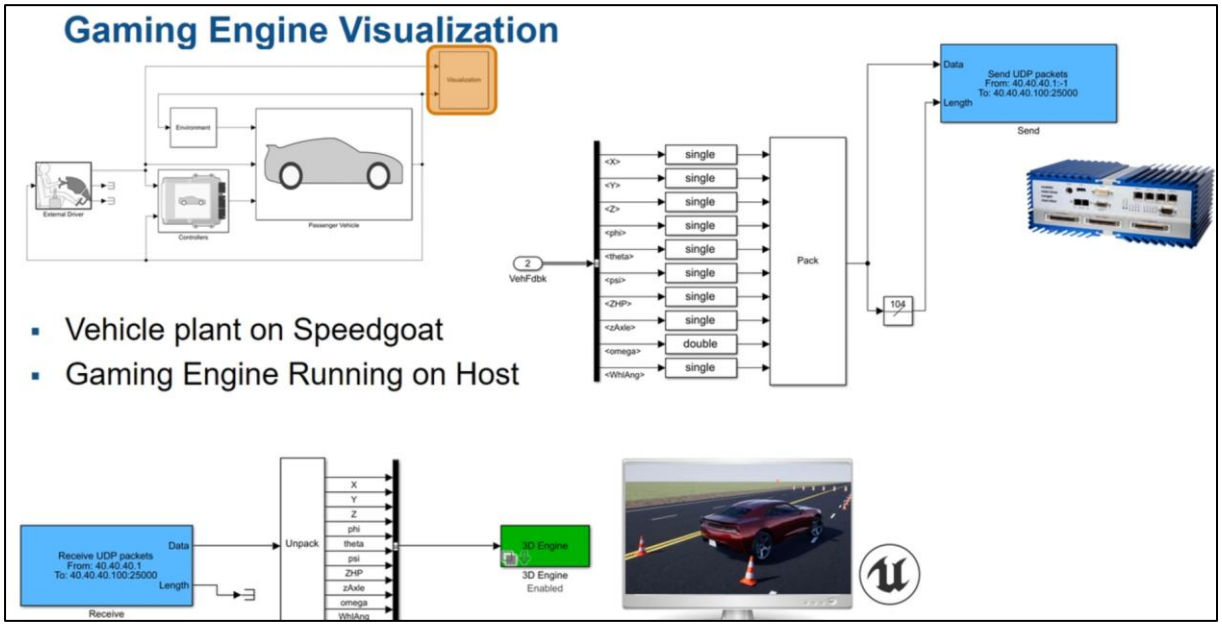
2020b

19.1.2 3DOF LONGI



---

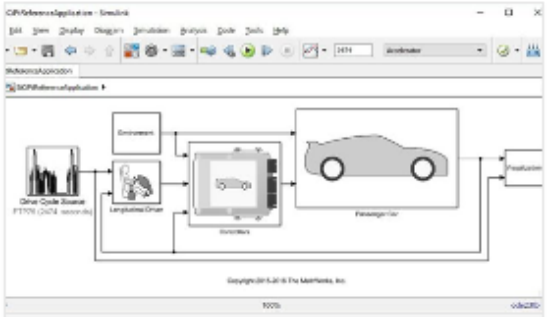
https://fr.mathworks.com/videos/building-real-time-driver-in-the-loop-simulators-1542313377351.html



19.2 POWERTRAIN BLOCKSET

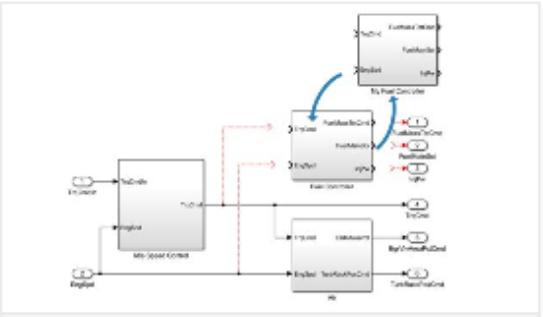
https://fr.mathworks.com/products/powertrain.html

https://fr.mathworks.com/videos/model-and-simulate-automotive-systems-using-powertrain-blockset-1506349847101.html?&t=1695



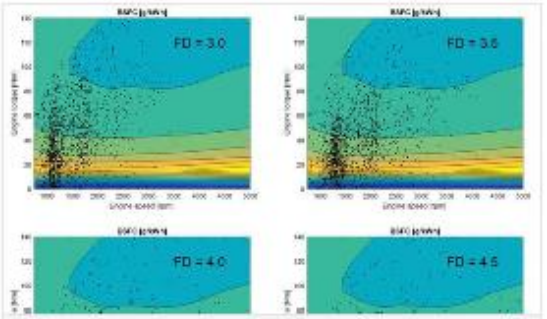
Model a Powertrain System

Use fully assembled reference applications and parameterize the components.



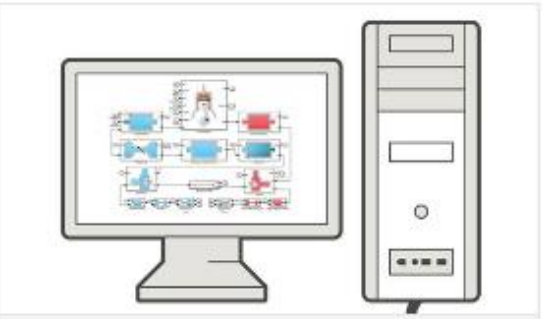
Design and Test a Controller Model

Use built-in controller models, user-defined controller models, and embedded estimators.



Perform System Design Tradeoff

Perform powertrain design tradeoff studies, such as emissions, fuel economy, and performance.

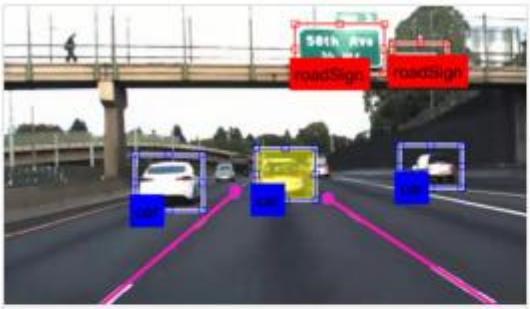


Deploy for Hardware-in-the-Loop Testing

Perform HIL testing on your controller model.

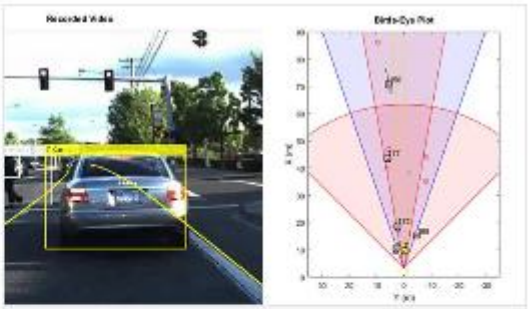
» Learn more

19.3 AD/ADAS TOOLBOX



Ground-Truth Labeling

Automate labeling of ground-truth data, and compare output from the algorithm under test with ground-truth data.



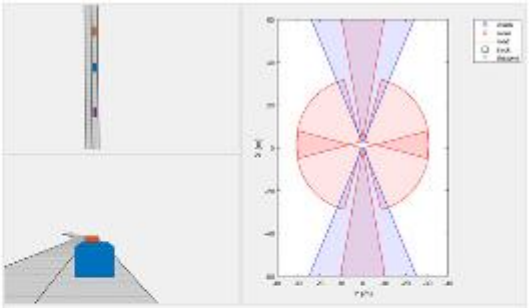
Sensor Fusion and Tracking

Perform multisensor fusion using multiobject tracking framework with Kalman filters.



Vision System Design

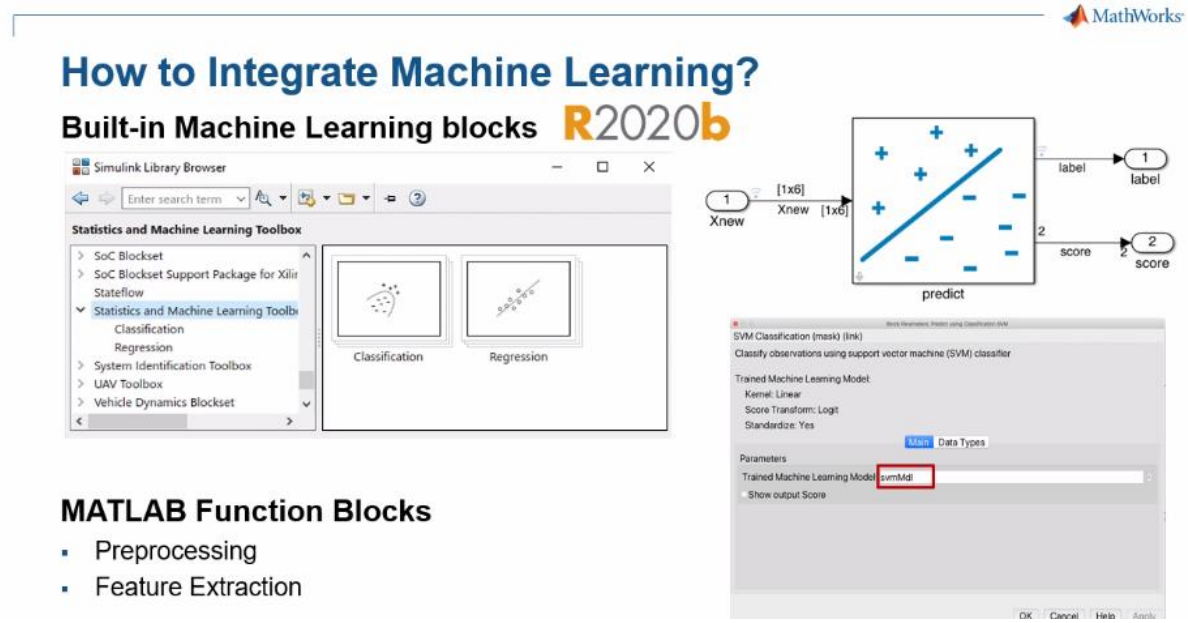
Develop computer vision algorithms for vehicle and pedestrian detection, lane detection, and classification.



Scenario Generation

Generate traffic scenarios and simulate radar and camera sensor outputs to test sensor fusion and control algorithms.

https://www.mathworks.com/videos/ai-for-simulink-users-1607620166106.html



Using AI for Reduced Order Modeling (Lucas Garcia, Mathworks, 2022)

https://blogs.mathworks.com/deep-learning/2022/12/05/using-ai-for-reduced-order-modeling/

https://content.mathworks.com/viewer/63814a32c8ce21bfab25787d

21 COMMENT FAIRE POUR …

21.1 TROUVER DES BLOCS PAR TYPE DE CONNECTION (SLQUERY)

https://blogs.mathworks.com/pick/2019/04/05/do-you-find-yourself-searching-for-connection-in-simulink/

21.2 ACCELERER UNE SIMULATION

Utiliser le Simulink cache (.slxc), cf §10.7

21.3 EXPORTER DES MODELES DANS DES VERSIONS PLUS ANCIENNES



21.4 CHERCHER DANS UN MODELE SIMULINK

21.4.1 AVEC L'ANCIENNE METHODE : FIND_SYSTEM

https://blogs.mathworks.com/simulink/2008/03/04/searching-in-simulink

Exemple :

```
h=find_system(logic_4WS,'SearchDepth',N, 'regexp', 'on',
'FollowLinks','on','Findall','on','LookUnderMasks','all',
'BlockType','Gain');
```

21.4.2 ET MAINTENANT : FINDBLOCKS

```
Simulink.findBlocksOfType('logic_4WS','Gain');
```

https://blogs.mathworks.com/simulink/2022/04/12/time-to-convert-from-find_system-to-simulink-findblocks/

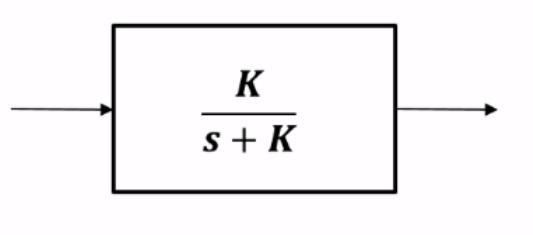
```
opts = Simulink.FindOptions;
opts.FollowLinks = true;
blksFindBlocks = Simulink.findBlocksOfType(mdl,'Math',opts);
getfullname(blksFindBlocks)
```

21.5 FONCTIONS DE TRANSFERT

21.5.1 VIDEO MATHWORKS

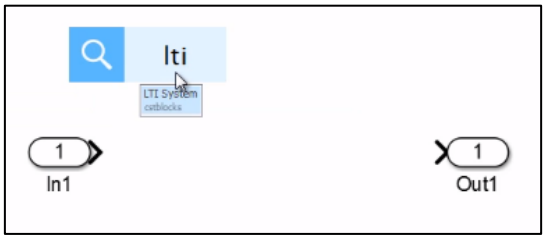
https://fr.mathworks.com/videos/transfer-functions-in-simulink-part-1-creating-and-using-transfer-functions-106146.html

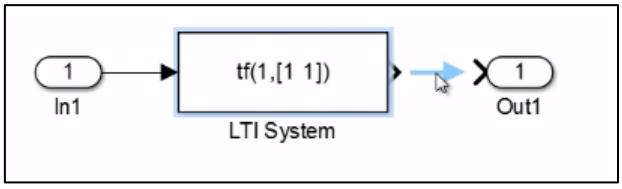
Pour représenter une fonction de transfert du type

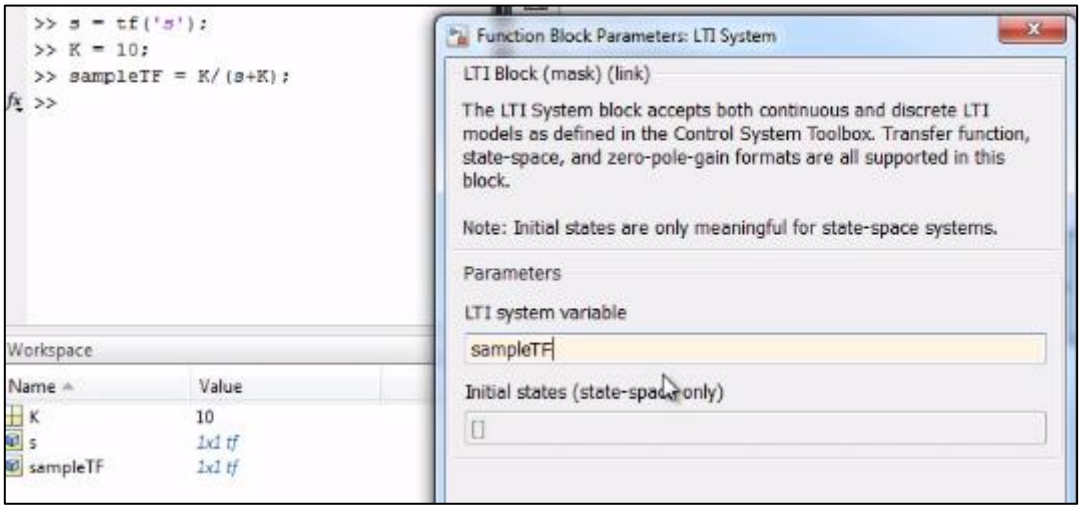


on peut utiliser les solutions suivantes

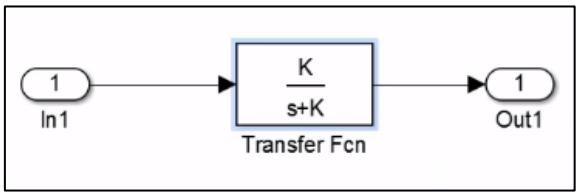
- le bloc LTI



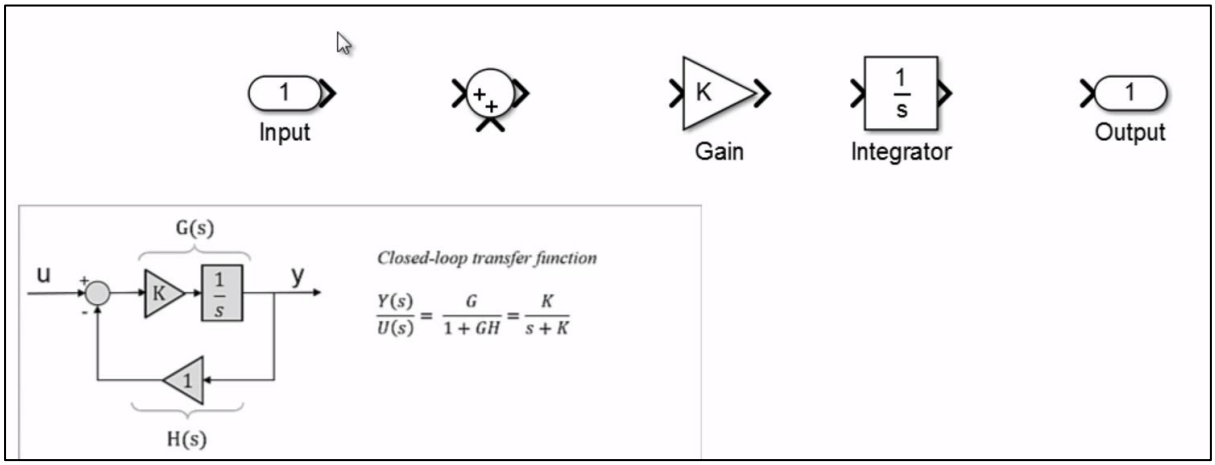




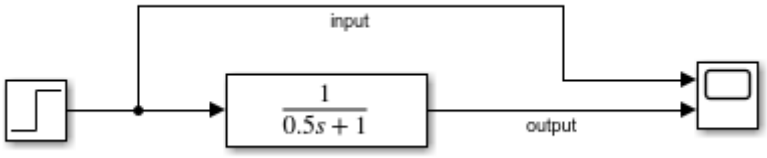
- un bloc TransferFcn :



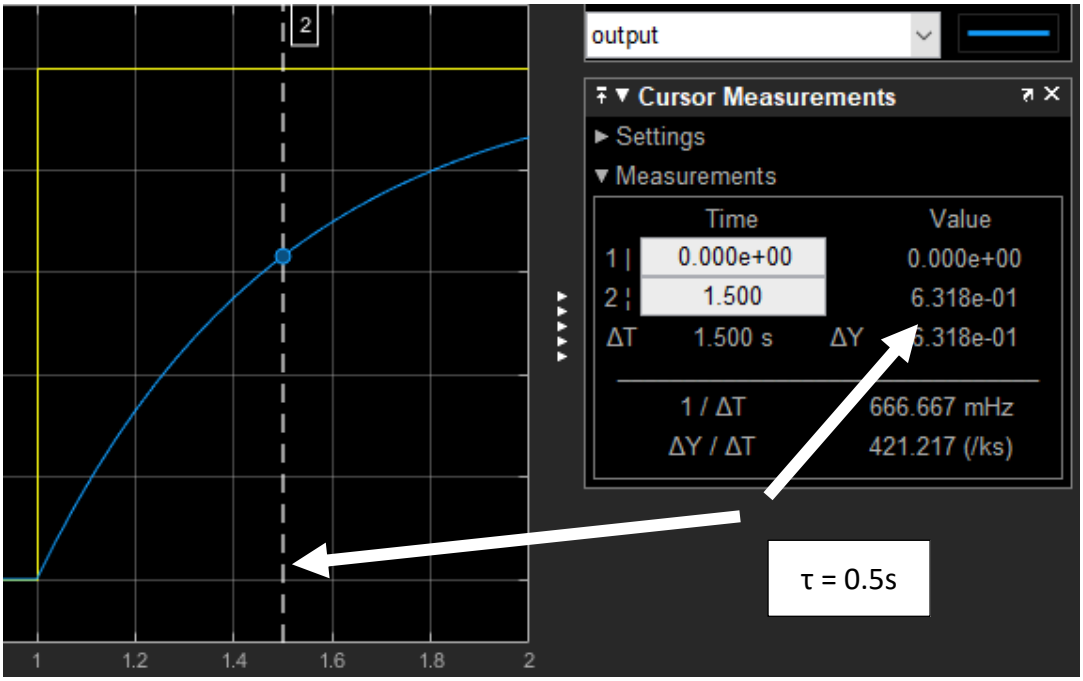
- une construction à base de gains et d'intégrateurs :



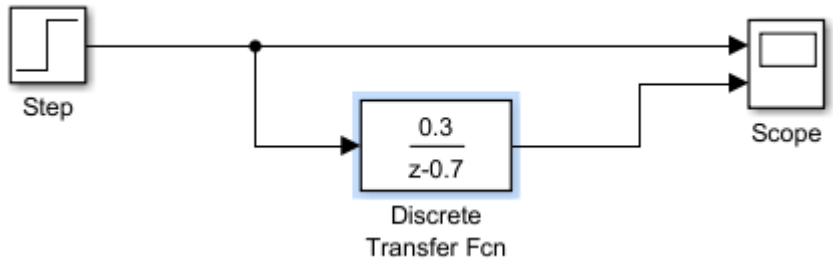
21.5.2 EXEMPLE SIMPLE EN CONTINU



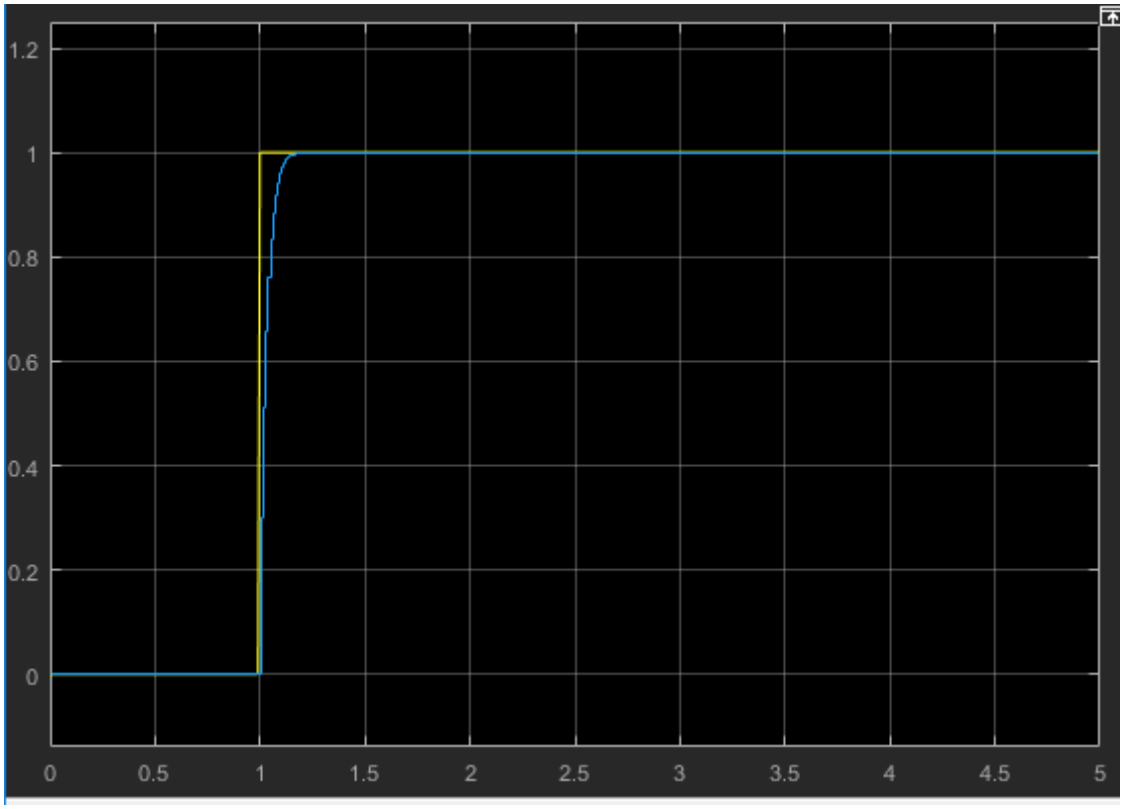
La fonction est de la forme 1/(1+τs), où τ est le temps caractéristique, c'est-à-dire le temps mis par la réponse pour atteindre 63% de la valeur finale.



21.5.3 EXEMPLE SIMPLE EN DISCRET

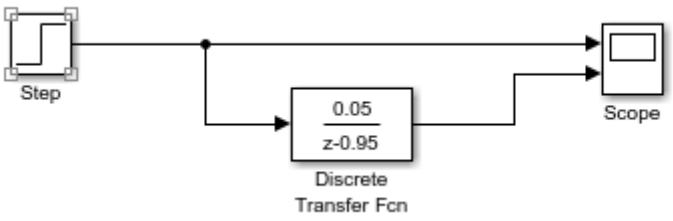


donne

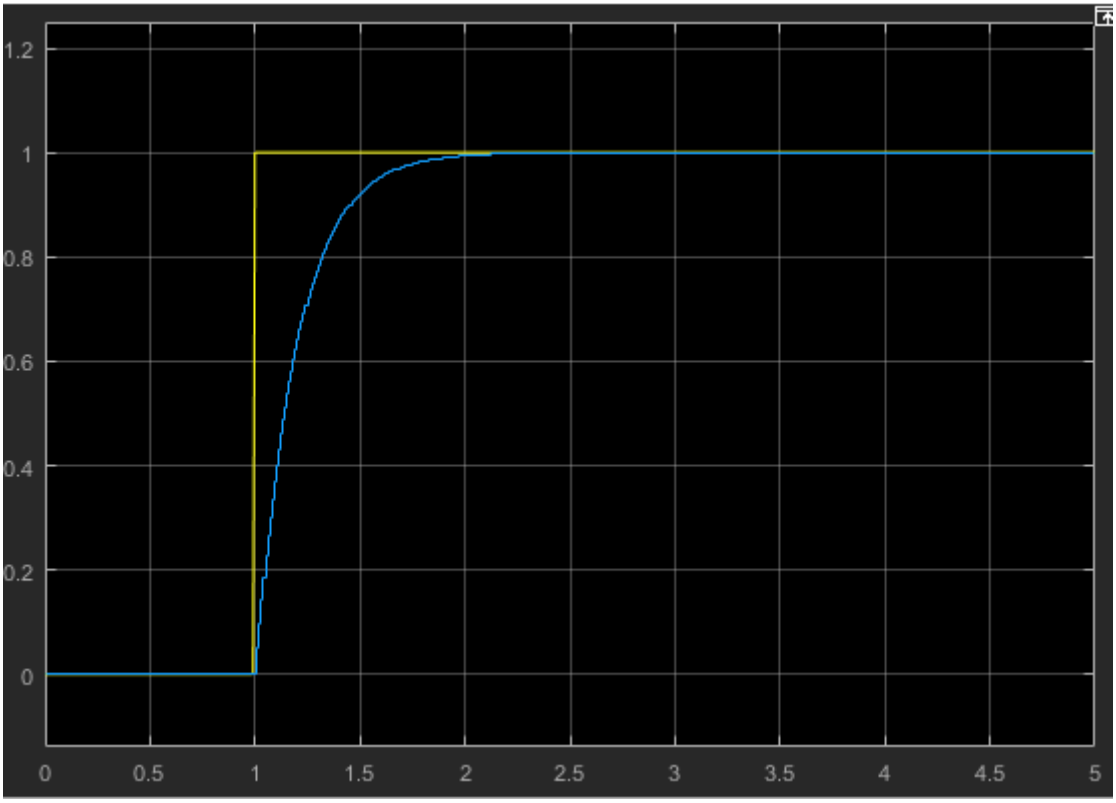


Plus la constante du numérateur est faible, plus le temps de réponse est élevé.

Avec



On obtient


1.2
1
0.8
0.6
0.4
0.2
0
0
0.5
1
1.5
2
2.5
3
3.5
4
4.5
5

22 EVOLUTIONS – RELEASES

22.1 R2016A

22.1.1 SIMULINK

Analyse et performance d'une simulation

- Option de solveur automatique : Configurez et simulez votre modèle plus rapidement grâce aux paramètres de solveur automatiquement sélectionnés
- Affichage en un seul clic : Cliquez sur une ligne de signal au moment d'exécuter la simulation pour afficher la valeur actuelle
- Diagnostic des métadonnées de simulation : Comprenez les raisons de l'arrêt d'une simulation par lots ou individuelle
- Mappage de ports d'entrée à racines multiples : Connectez plusieurs ensembles de signaux d'entrée à votre modèle Simulink pour une simulation interactive ou par lots
- Simulation pour les cibles mixtes : Simulez les designs au niveau système qui intègrent des modèles référencés ciblant un ensemble hétérogène d'appareils embarqués

Modélisation à base de composants

- Propagation des conditions dans les blocs Variant Source et Sink : Concevez des choix de variantes et supprimez automatiquement les fonctions non souhaitées en fonction de la connectivité du bloc
- Détermination des fonctions Simulink Appelez les blocs Simulink Function dans une hiérarchie de sous-systèmes
- Unités Simulink : Spécifiez, visualisez et vérifiez la cohérence des unités sur les interfaces
- Boîtes de dialogue des masques : Créez des masques avec des options de mise en page flexibles et de nouveaux paramètres de contrôle

Gestion de projet et de fichier

- Page de démarrage : Commencez ou reprenez le travail plus rapidement grâce à des modèles de base, des modèles récents et des exemples
- Renommage automatique : Mettez à jour toutes les références d'un projet au moment de renommer les modèles, les bibliothèques ou les fichiers MATLAB

Éditeur Simulink

- Actions de sélection unique : Accédez aux actions de modification fréquemment utilisées au moment de cliquer sur un bloc ou une ligne de signal

Gestion des données

- Enregistrement des signaux et des états dans un fichier : Enregistrez les données directement dans un fichier MAT pour les longues simulations.

22.1.2 TOOLBOXES

SimEvents

Nouveau moteur de simulation et de modélisation d'événements discrets avec des actions d'événement, création d'objets MATLAB pour les systèmes à événements discrets et transitions automatiques de domaine Simulink et Stateflow

Simscape

Nouvelle technologie de réduction et de simulation des équations pour une simulation et un réglage des paramètres d'exécution plus rapides dans le code généré

Simscape Fluids

Bibliothèque de liquides thermiques pour la modélisation de systèmes contenant des liquides dont les propriétés varient en fonction de la température

Simulink Design Optimization

Outil d'analyse de la sensibilité qui utilise un plan d'expérience, les simulations de Monte Carlo et une analyse de corrélation.

Simulink Report Generator

Fusion d'un modèle tridimensionnel afin de résoudre graphiquement les conflits entre les révision au sein d'un projet Simulink

22.2 R2018A

Introduction du blockset Vehicle Dynamics

22.3 R2020A

• Model Comparison: Create PDF model comparison reports
• C Function Block: Integrate and call C code from the model
• Rate Transition block deterministic mode support for concurrent execution

• X3D is now the default file format for Simulink 3D Animation

A lot of evolutions in the **Automated Driving Toolbox**

Control System Designer: Perform automated loop shaping without a Robust Control Toolbox license

MPC Toolbox : Nonlinear MPC Code Generation: Generate code for nonlinear MPC controllers that use default fmincon solver with the SQP algorithm

Powertrain Blockset :

- Transmission Control Module: Optimize shift schedules for algorithm design and performance, fuel economy, and emissions analysis
- Longitudinal Driver Block: Configure inputs to control acceleration and deceleration commands

Vehicle Dynamics Blockset

- Driver Blocks: Configure inputs to control acceleration, deceleration, and steering commands
- Headless Mode: Run 3D simulations without opening the Unreal Engine 3D visualization display
- Inertial Measurement Unit Block: Implement a three-axis inertial block to measure accelerations and angular rates

22.4 JUIN 2020 : SIMULINK ONLINE

https://fr.mathworks.com/products/simulink-online.html

23 EXTENSIONS DE FICHIER

.slxc : Simulink Cache

.sldd : Data Dictionary